

AssureX Claim Engine

Project documentation

Group	NN_DevStorm
Batch	2609E2'
Faculty	Sir Minhaj
Repository	https://github.com/Saba1512006/assurex-claim-engine
Live application	https://assurex.pythonanywhere.com
Date	29 September 2026

Student ID	Name	Role
Student1525913	Saba Noor	Team lead · integration, frontend direction, QA, documentation
Student1524867	Muhammad Sami	Backend and ML pipeline developer · deployment
Student1509222	Muhammad Ghanyan	Frontend design (Figma, colour palette, mobile checks)
Student1505677	Muhammad Sami ur Rehman	Testing (sample claims, Chrome and mobile)

Abstract

AssureX Claim Engine evaluates warranty claims for consumer electronics, home appliances and industrial tools. A customer or service center registers a product from its receipt (read by OCR), files a claim with evidence, and the application pre-processes the claim into model features. A calibrated Python classification model (HistGradientBoosting, 89.3% test accuracy on 225 unseen claims) scores the structured record; the same claim is drawn as a Claim Summary Card and classified by a Google Teachable Machine model (93.8% test accuracy). The two predictions and their confidences are compared (Strong Match, Acceptable Match, Weak Match, Model Disagreement, Uncertain Result), the category's warranty policy is executed from JSON configuration, contradictions, missing documents and duplicates are detected, and a decision table produces Likely Valid, Likely Invalid or Manual Review Required. Anything uncertain goes to a human reviewer, and every step is recorded in an audit trail. The application has four roles (customer, service-center staff, claim reviewer, administrator), 295 automated tests and 34 browser tests.

This document brings every deliverable of the SRS (§1.10) together: the project report, dataset, both models' evidence, the comparison report, warranty policies, test cases and results, installation and execution instructions with screens of the working application, deployment, the demonstration video, the technical blog, the AI usage declaration and the team contribution record.

Contents

1. Project report
2. Dataset
3. Python classification model evidence
4. Google Teachable Machine evidence
5. Model prediction and confidence comparison report
6. Warranty policy files
7. Test cases and results
8. Installation instructions
9. Execution instructions and user guide
10. Deployed application
11. Demonstration video
12. Technical blog
13. Security and access-control design
14. AI tool usage declaration
15. Team contribution record and development log
16. Submission checklist

1. Project report

1. Problem definition

Manufacturers and service centers receive warranty claims that must be checked against purchase proof, product age, warranty terms, exclusions, repair history and supporting evidence. Done by hand this is slow, inconsistent between staff, and easy to defeat with reused receipts, altered dates or mismatched serial numbers. AssureX automates the checking and produces a recommendation — *Likely Valid*, *Likely Invalid* or *Manual Review Required* — that is explained, audited and sent to a person whenever the evidence is uncertain.

2. Background and business necessity

Warranty cost is a direct expense for manufacturers and a satisfaction driver for customers. Typical failure modes of manual review are: approving claims filed after the warranty and its grace period, missing exclusions (drops, liquid damage), accepting the same invoice twice, and inconsistent decisions for identical cases. An automated first pass that applies the policy the same way every time, flags contradictions and duplicates, and routes only the unclear cases to reviewers shortens turnaround and makes decisions defensible.

3. Proposed solution

A Python/Flask web application in which:

1. customers (or service-center staff for walk-in customers) register products and warranties and file claims with receipts, warranty cards, photos, videos and diagnostic reports;
2. documents are validated, fingerprinted (SHA-256) and read by OCR; the user confirms extracted values;
3. the claim is converted into a structured feature record and scored by a **Python classification model**;
4. the same record is drawn as a **Claim Summary Card** image and scored independently by a **Google Teachable Machine** model;
5. a **rule engine** checks the category's warranty policy; detectors look for **contradictions** and **duplicates**;
6. a configurable **decision table** combines rules, both predictions and their **consistency status** into the final recommendation; uncertain claims enter a **manual-review queue**;
7. reviewers decide (with overrides that require a reason), customers track progress through eight stages, and administrators see dashboards, analytics, exports and a full audit trail.

4. Purpose of the document

This report describes the requirements, design, data, models, testing and limitations of AssureX so that evaluators, maintainers and stakeholders share one understanding of what was built and how to verify it.

5. Scope

In scope: registration and authentication for four roles, product/warranty records, receipt upload and OCR, claim intake, evidence management, repair history, validation and pre-processing, the common dataset, both models, comparison, warranty rules, contradiction/duplicate/missing-document detection, claim summary and explanation, manual review, tracking, notifications and alerts,

dashboards, search, analytics, PDF reports, CSV export, audit trail, model versioning, error handling and monitoring. Out of scope (SRS §1.4): live manufacturer databases, payment systems, enterprise warranty platforms.

6. Assumptions

- Claims arrive through the web interface; a claim concerns one registered product with one warranty.
- Service-center staff belong to one service center; products may name a preferred authorised center.
- The warranty policy for a category is the JSON file in [policies/](#); changing it changes future evaluations only.
- Diagnostic confidence (0–1) comes from a technician; if nobody assessed the fault it is 0.5 (neutral).
- The training data is synthetic, generated from the policy with 4% label noise.

7. Constraints

- Quality depends on the completeness and honesty of the submitted evidence (SRS §1.5).
- The dataset size (1,500 claims) limits how much rare scenarios can be learned (e.g. late reporting).
- The two models are trained independently and do not produce identical confidences; the comparison thresholds absorb normal differences.
- Teachable Machine training happens in the browser and must be repeated by hand when cards change.
- Personal data (names, contact details, receipts) must be protected: role-scoped access, no public file URLs, audit logging, and demo data only in the repository.

8. Functional requirements (SRS §1.6) — where each is implemented

#	Requirement	Implementation
i	Registration & authentication, role-based access	src/api/auth.py , src/security/ , config/rbac.json — customer self-sign-up, other roles invited
ii	Profile, unique User ID	/profile , User.user_id (USR-...)
iii	Product registration, Product ID	/products/new , Product.product_id (PRD-...)
iv	Standard & extended warranties, provider, dates, conditions, exclusions, service center; active/expired/expiring view	ProductWarranty , product page, product list filters
v	Upload receipts/invoices/warranty cards (PDF/JPG/JPEG/PNG)	src/services/documents.py (magic-byte validation)
vi	Extract purchase date, invoice, product, model, serial, retailer, amount, warranty length	src/ocr/document_processor.py
vii	Show extracted values, let users correct them	Wizard step 3, product form auto-fill, claim page <i>Extracted details</i> (audited)
viii	Warranty dates, remaining period, active/expired/expiring/extended	ProductWarranty.status , life-used bars
ix	Expiry alerts with admin-configured window	src/services/alert_service.py , Admin > Policies > alerts

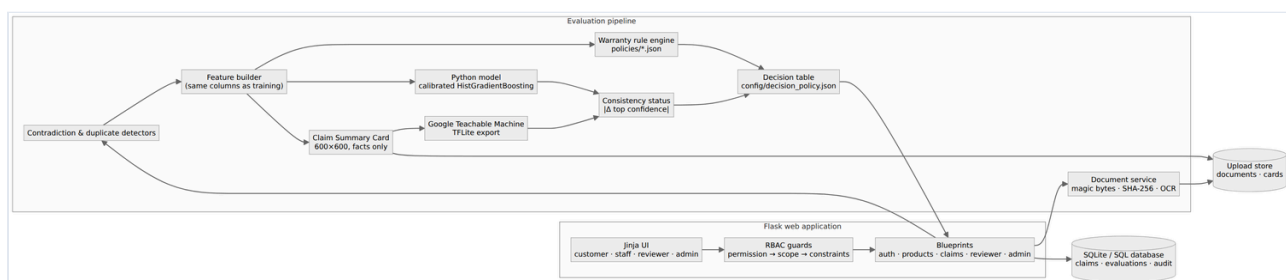
#	Requirement	Implementation
x	Claim registration by users or staff, Claim ID, links	/claims/new , Claim.claim_id (CLM-...)
xi	Claim information (age, purchase, fault date, description, cause, service history, replacement, submission date)	Wizard step 2, Claim model
xii	Photos, damage images, videos, serial photos, diagnostic reports	Eight evidence types incl. MP4 video
xiii	Repair history (dates, center, parts, outcome, cost, authorised or not)	RepairHistory , product page (staff)
xiv	Documents organised per product and claim; view/download/replace/remove by rights	Product and claim evidence panels, document.* permissions
xv	Validation: mandatory fields, dates, numbers, file types, sizes, duplicate IDs	src/rules/validator.py , documents.validate , unique DB constraints
xvi	Pre-processing: missing values, dates, encoding, normalisation, derived fields	src/core/features.py + saved sklearn pipeline
xvii	Common dataset in CSV and card form	data/ , dataset_generator/
xviii	Train & compare algorithms, select best	notebooks/train_python_v2.py
xix	Python confidence for all three classes	Claim page, stored per evaluation
xx	Claim Summary Card without prediction/decision	src/core/card_v2.py
xxi	Teachable Machine classification	src/core/gtm_classifier_v2.py
xxii-xxiv	Class match, Δ confidence , five consistency statuses with configurable thresholds	src/core/consistency.py , config/decision_policy.json
xxv-xxvi	Warranty rule validation, configurable per category	src/rules/policy_engine.py , policies/*.json , Admin > Policies
xxvii	Serial number vs receipt, warranty card, product image, repair records	src/rules/contradiction_detector.py
xxviii	Contradictions (dates, models, serials)	same
xxix	Missing mandatory documents, telling the user which	Policy mandatory_documents / supporting_documents , checklist, notifications
xxx-xxxi	Duplicate claims and documents (IDs, invoices, serials, descriptions, claimant, hashes, history)	src/rules/duplicate_detector.py
xxxii	Claim summary	src/services/explain.py → summarise (template-based, no external AI)
xxxiii	Claim preparation assistance	Live readiness checklist in the wizard
xxxiv	Final decision from all factors	src/core/pipeline.py , decision table
xxxv	Explanation: supporting/opposing factors, rules passed/failed, contradictions, evidence needed	explain.py → explain , claim page
xxxvi-xxxvii	Manual-review queue, approve/reject/request info, comments, override with reason, original AI result kept	src/api/reviewer.py , ReviewerAction
xxxviii	Eight tracked stages	Claim.transition_status , stage bar, tracking page
xxxix	Notifications (expiry, submission, missing docs, info requests, status, review, approval, rejection)	src/services/notifications.py , bell menu, dashboard
xl	User dashboard	/claims/
xli	Admin dashboard (totals, classes, pending, duplicates, disagreements, average confidences, trend)	/admin/dashboard

#	Requirement	Implementation
xlii	Search & filter by claim, product, category, serial, warranty status, claim status, risk, confidence, reviewer, dates	/claims/search
xliii	Analytics (outcomes, faults, rejection reasons, categories, expirations, repairs, model performance, manual-review frequency)	/admin/analytics
xliv	Downloadable claim report	<a href="/claims/<id>/report.pdf">/claims/<id>/report.pdf (src/services/report_generator.py)
xlvi	CSV/Excel export	<a href="/admin/export/<kind>">/admin/export/<kind> (CSV with BOM, or ?format=xlsx workbook); formula-injection guard in both
xlvi	Secure database storage	SQLAlchemy models, SQLite/any SQL
xlvi	Audit trail	AuditLog , src/services/audit.py , Admin > Audit
xlviii	Model version tracking; updates don't alter recorded results	Hash-based versions on every ModelEvaluation row; rows never updated
xlix	Understandable errors	Friendly error pages and flash messages; model/OCR failures degrade gracefully
I	Monitoring & anomaly alerts	alert_service.anomalies() on the admin dashboard; <i>Send due alerts</i>

9. Non-functional requirements

NFR	Result
Performance ≤ 5 s	Full evaluation (rules + Python + card + image model + decision) ~150–300 ms on a laptop; stored per claim (latency_ms) and tested
Scalability (10,000 claims, several centers, concurrent users)	Indexed columns (claim_id, status, reviewer, serial, hashes); scoped SQL queries; stateless workers; any SQL database via DATABASE_URL
Usability and accessibility	Four-step wizard with a live readiness checklist and server-saved drafts, plain-language explanations, responsive layout from 390 px, keyboard shortcuts for reviewers, visible focus, reduced-motion support. axe-core finds no serious or critical WCAG 2 A/AA issue on 21 pages; Lighthouse scores 100 for accessibility, best practices and SEO and 99 for performance on the public pages
Accuracy $\geq 85\%$	Python model 89.3%, Teachable Machine 93.8% on the 225 unseen test claims
Availability $\geq 99\%$	Stateless app behind gunicorn; /healthz for uptime monitors; no runtime dependency on external APIs or CDNs

10. Application architecture



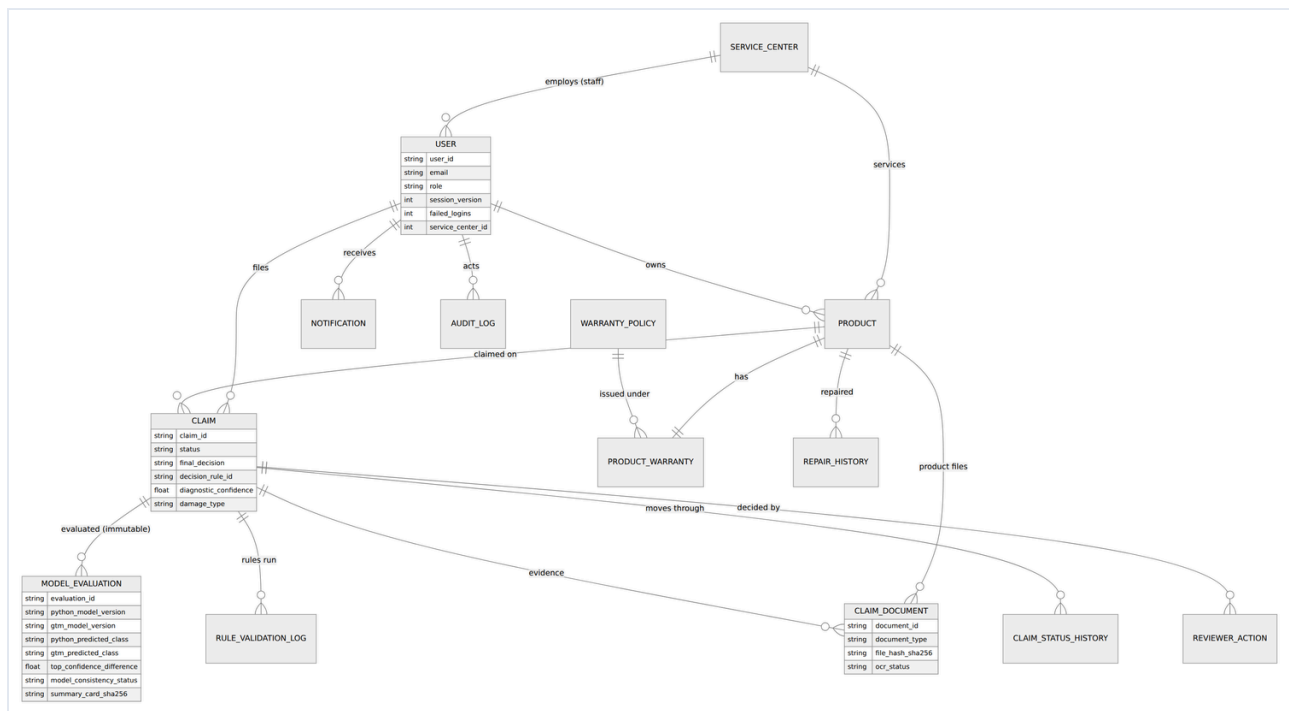
Architecture

Three layers: the **web layer** (Jinja templates on one component library, design tokens in [static/css/tokens.css](#), self-hosted IBM Plex fonts, a shell script plus small page scripts and the Alpine.js CSP build; the Content-Security-Policy allows only same-origin scripts and one nonce'd bootstrap line, and every JSON endpoint answers with one `{success, data, error}` envelope), the **application layer** (Flask blueprints guarded by the RBAC engine; services for the claim lifecycle, documents, alerts, analytics, exports and reports) and the **evaluation layer** (feature builder, rule engine, detectors, both model runtimes, card renderer, consistency and decision table). Persistence is SQLAlchemy over SQLite by default plus a file store for documents and cards.

11. Module descriptions

Module	Responsibility
src/core/vocab.py	Single vocabulary: categories, damage types, faults, classes, document types, date formats
src/core/features.py	Pre-processing and feature engineering for a stored claim
src/core/python_classifier.py	Loads the versioned sklearn pipeline; never retrains itself
src/core/card_v2.py	Deterministic 600×600 card renderer (v3 tile design) with bundled fonts
src/core/gtm_classifier_v2.py	Teachable Machine TFLite runtime, label parsing, version hashing
src/core/consistency.py , decision_table.py	Five statuses; first-match decision table with validation and safe saving
src/core/pipeline.py	Orchestration and immutable persistence of each evaluation
src/core/offline_eval.py	Same pipeline over the val/test splits for evaluation and reports
src/rules/policy_store.py	Loads/validates/saves policy JSON; rule catalogue
src/rules/policy_engine.py	Sixteen rule checks; severity from policy
src/rules/contradiction_detector.py , duplicate_detector.py	Evidence consistency and duplicate indicators
src/rules/validator.py	Form validation and preparation checklist
src/security/rbac.py , guards.py	Permission → scope → constraint engine; request hooks, decorators, scoped queries, login hardening
src/services/*	Claim lifecycle, documents, notifications, audit, alerts, analytics, exports, PDF report, summaries
src/api/*	Public, auth, products, claims, reviewer (queue and workbench), admin, access-control blueprints
src/api/api.py , errors.py	JSON endpoints (stored verdict, live bench, draft autosave, what-if, batch) and the error vocabulary
src/core/explain_models.py	Per-claim explanations: Python feature contributions (replace each input with its training median/mode) and Teachable Machine tile occlusion
src/services/verdict.py	Verdict screen data from the stored payload: agreement-meter geometry, reviewer overrides, customer visibility
src/services/whatif.py , batch.py	Threshold simulator over the labelled test claims and live claims; chunked batch evaluation of uploaded CSVs
src/services/model_card_service.py	Public model card: metrics, calibration (reliability bins, ECE), per-category accuracy, label-noise analysis

12. Database design



Entity relationships

Sixteen tables. Key points: public identifiers (USR-, PRD-, CLM-, DOC-, EVL-) are random and unique; **claims** carries status, recommendation, decision rule and flags; **model_evaluations** stores the exact feature record, both predictions with all three confidences, model versions, card hash, thresholds and the decision trace — one row per run, never updated; **rule_validation_logs** stores every rule with its severity and message plus contradictions, duplicate indicators and missing documents; **claim_status_histories** and **audit_logs** are append-only; **users** hold security state (session version, failed logins, logout, forced password change, service center).

13. Data dictionary

Dataset data dictionary ([data/data_dictionary.json](#))

Column	Description
claim_id	Claim identifier CLM-00001..CLM-01500, assigned after shuffling (never encodes the class). Not a feature.
product_id	Product identifier PRD-xxxxx. Not a feature.
product_category	Consumer Electronics Home Appliances Industrial Tools (src/core/vocab.py). Feature (one-hot).
product_brand	Descriptive. Not a feature.
product_model	Descriptive. Not a feature.
product_serial	Descriptive. Not a feature.
retailer	Descriptive. Not a feature.
invoice_number	Descriptive. Not a feature.
fault_category	Fault from the category's covered-fault list. Used by the rule engine (UNCOVERED_FAULT), not a model feature.
fault_description	Free text written from fault + cause, class-independent. Never a feature.
damage_type	Customer-describable cause (7 values, vocab.DAMAGE_TYPES). Feature (one-hot). Not a relabelled class.

Column	Description
purchase_date	YYYY-MM-DD.
warranty_start_date	Equal to purchase date.
warranty_expiry_date	purchase_date + round(months * 30.44) days.
fault_occurrence_date	YYYY-MM-DD; before purchase_date when claim_date_conflict_flag = 1.
claim_submission_date	YYYY-MM-DD within 2026.
warranty_duration_months	Standard term of the category (+12 when extended). Feature.
is_extended_warranty	1 if an extended term was bought. Feature.
purchase_price	Log-normal price. Feature.
product_age_days	claim date - purchase date. Feature.
days_to_expiry	Coverage days - age; negative once expired. Feature.
remaining_warranty_days	max(0, days_to_expiry). Display only.
reporting_delay_days	claim date - fault date (0 if negative). Feature.
diagnostic_confidence	Technician's confidence (0-1) that the stated cause is right; 0.5 = not assessed. Feature.
previous_repairs_count	Earlier repairs on record. Feature.
unauthorized_repair_flag	1 if any earlier repair was by an unauthorised center. Feature.
serial_number_match	0 if the registered serial differs from the evidence. Feature.
duplicate_invoice_flag	1 if the invoice/receipt was already used elsewhere. Feature.
claim_date_conflict_flag	1 if the dates contradict (fault before purchase, etc.). Feature.
has_receipt	Purchase receipt present (0/1). Feature.
has_warranty_card	Warranty card present. Feature.
has_damage_photo	Damage/fault photo present. Feature.
has_serial_photo	Serial photo present. Feature.
has_repair_report	Diagnostic/repair report present. Feature.
missing_document_count	4 - (receipt + warranty card + damage photo + serial photo). Feature.
scenario	Why the ground-truth policy gave this label (e.g. expired, grace_boundary, covered_defect); '+noise' marks the 4% flipped labels. Never a feature.
claim_class	Target: Valid Claim Invalid Claim Manual Review.

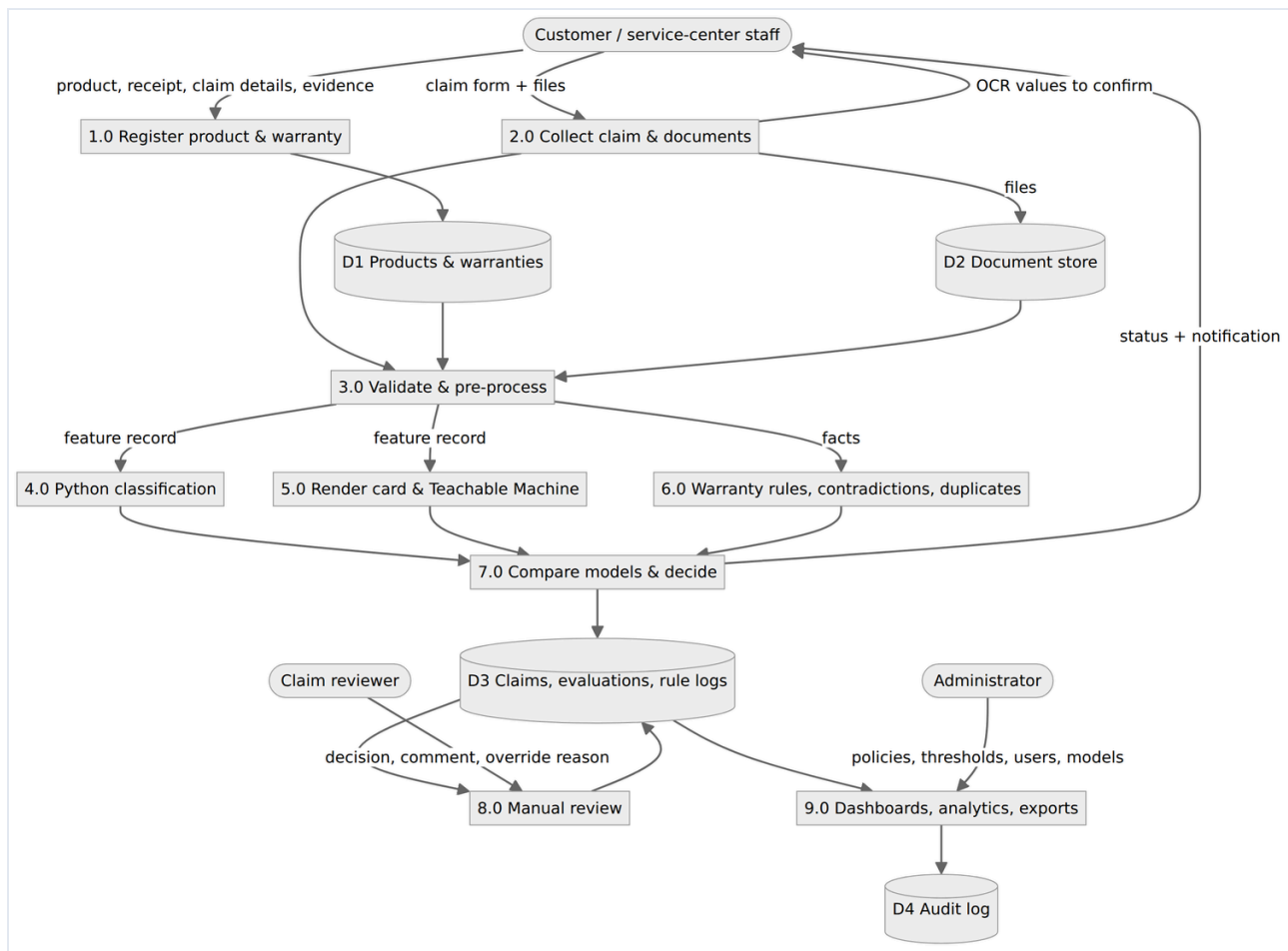
Database tables (16, [src/models/entities.py](#))

Table	Columns	Fields (PK primary key, FK foreign key)
service_centers	6	id (PK), center_id, name, city, phone, created_at
users	17	id (PK), user_id, email, password_hash, full_name, role, phone_number, address, is_active, session_version, failed_logins, locked_until, must_change_password, last_login_at, service_center_id (FK → service_centers), created_at, updated_at
products	14	id (PK), product_id, user_id (FK → users), service_center_id (FK → service_centers), product_name, category, brand, model_number, serial_number, purchase_date, purchase_price, retailer, invoice_number, created_at
warranty_policies	10	id (PK), policy_id, category, policy_name, coverage_duration_months, grace_period_days, claim_reporting_period_days, authorized_service_center_required, policy_rules_json, created_at
product_warranties	14	id (PK), warranty_id, product_id (FK → products), policy_id (FK → warranty_policies), warranty_provider, start_date, expiry_date, duration_months, is_extended, extended_months, coverage_conditions, exclusions, service_center_name, created_at

Table	Columns	Fields (PK primary key, FK foreign key)
claims	28	id (PK), claim_id, user_id (FK → users), created_by_id (FK → users), product_id (FK → products), warranty_id (FK → product_warranties), service_center_id (FK → service_centers), fault_occurrence_date, fault_description, fault_category, damage_type, diagnostic_confidence, diagnosis_source, previous_replacement_details, claim_submission_date, status, risk_level, final_decision, decision_reason, decision_rule_id, decision_policy_version, is_duplicate_flag, contradiction_flag, missing_document_flag, assigned_reviewer_id (FK → users), reviewer_notes, created_at, updated_at
claim_documents	16	id (PK), document_id, claim_id (FK → claims), product_id (FK → products), uploaded_by_id (FK → users), document_type, file_path, original_filename, mime_type, file_size_bytes, file_hash_sha256, ocr_status, ocr_extracted_text, ocr_data_json, verified_by_user, created_at
repair_histories	14	id (PK), repair_id, product_id (FK → products), claim_id (FK → claims), recorded_by_id (FK → users), repair_date, repair_center, replaced_parts, outcome, repair_cost, is_authorized_center, serial_number_seen, notes, created_at
notifications	10	id (PK), notification_id, user_id (FK → users), notification_type, title, message, related_claim_id, related_product_id, is_read, created_at
claim_status_histories	7	id (PK), claim_id (FK → claims), previous_status, new_status, changed_by_user_id (FK → users), reason_comment, created_at
model_evaluations	30	id (PK), evaluation_id, claim_id (FK → claims), features_json, python_available, python_model_version, python_predicted_class, python_conf_valid, python_conf_invalid, python_conf_manual, gtm_available, gtm_model_version, gtm_predicted_class, gtm_conf_valid, gtm_conf_invalid, gtm_conf_manual, model_errors_json, is_class_match, top_confidence_difference, model_consistency_status, consistency_thresholds_json, decision, decision_rule_id, decision_trace_json, decision_policy_version, summary_card_image_path, summary_card_sha256, latency_ms, payload_json, evaluation_timestamp
rule_validation_logs	10	id (PK), claim_id (FK → claims), evaluation_id (FK → model_evaluations), policy_name, rules_json, contradictions_json, duplicate_flags_json, missing_documents_json, overall_rule_status, execution_timestamp
reviewer_actions	10	id (PK), claim_id (FK → claims), reviewer_id (FK → users), previous_status, previous_recommendation, reviewer_decision, is_override, override_reason, comments, timestamp
audit_logs	9	id (PK), user_id (FK → users), user_role, action, entity_type, entity_id, ip_address, details_json, timestamp
system_settings	5	id (PK), setting_key, setting_value, description, updated_at
batch_runs	12	id (PK), batch_id, created_by_id (FK → users), filename, status, total, processed, rows_json, results_json, error, created_at, finished_at

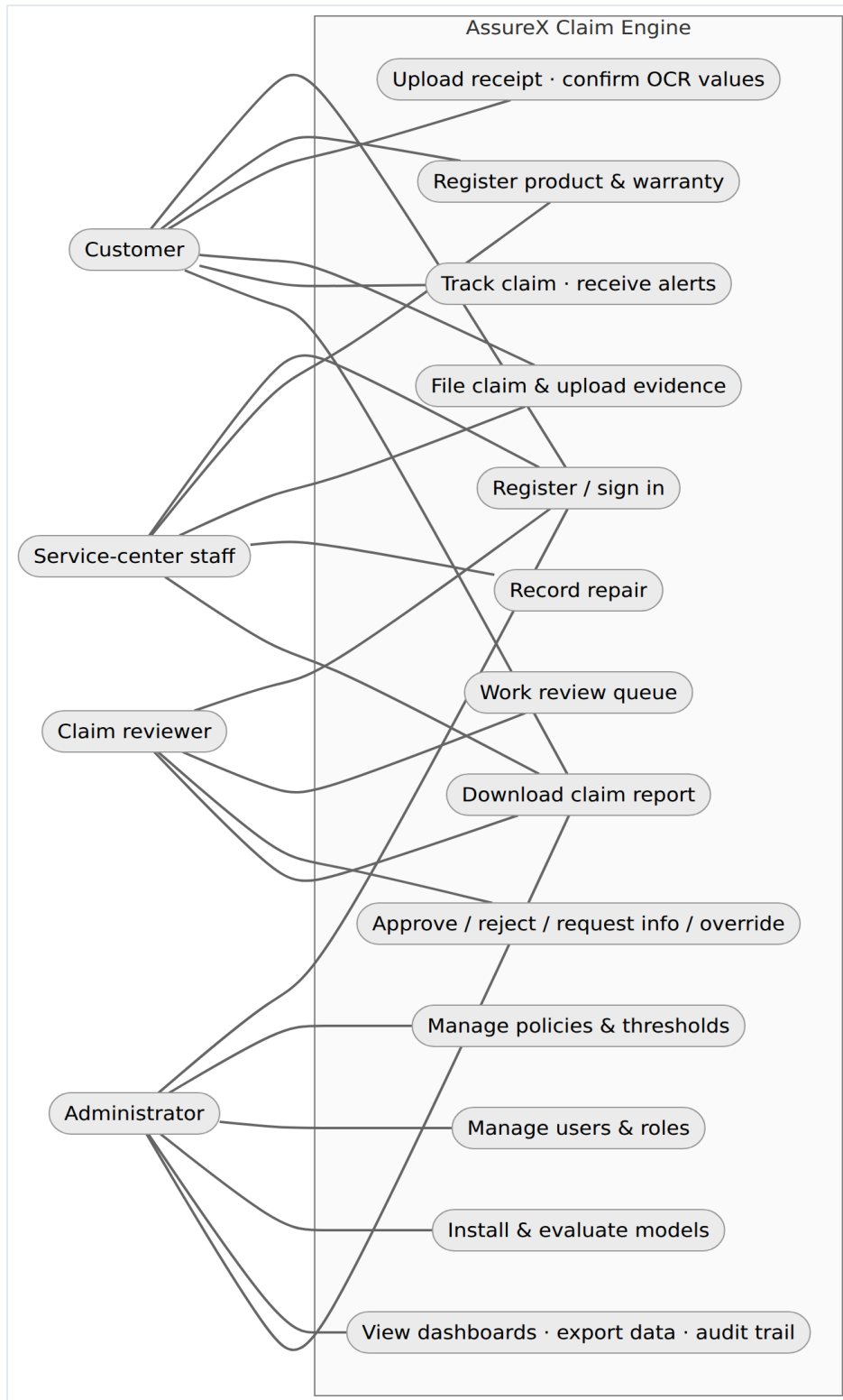
The dataset columns are documented in [data/data_dictionary.json](#) (37 columns: identifiers, descriptive fields, 20 model features, scenario and label). Database fields are documented in [src/models/entities.py](#).

14. Data flow diagram



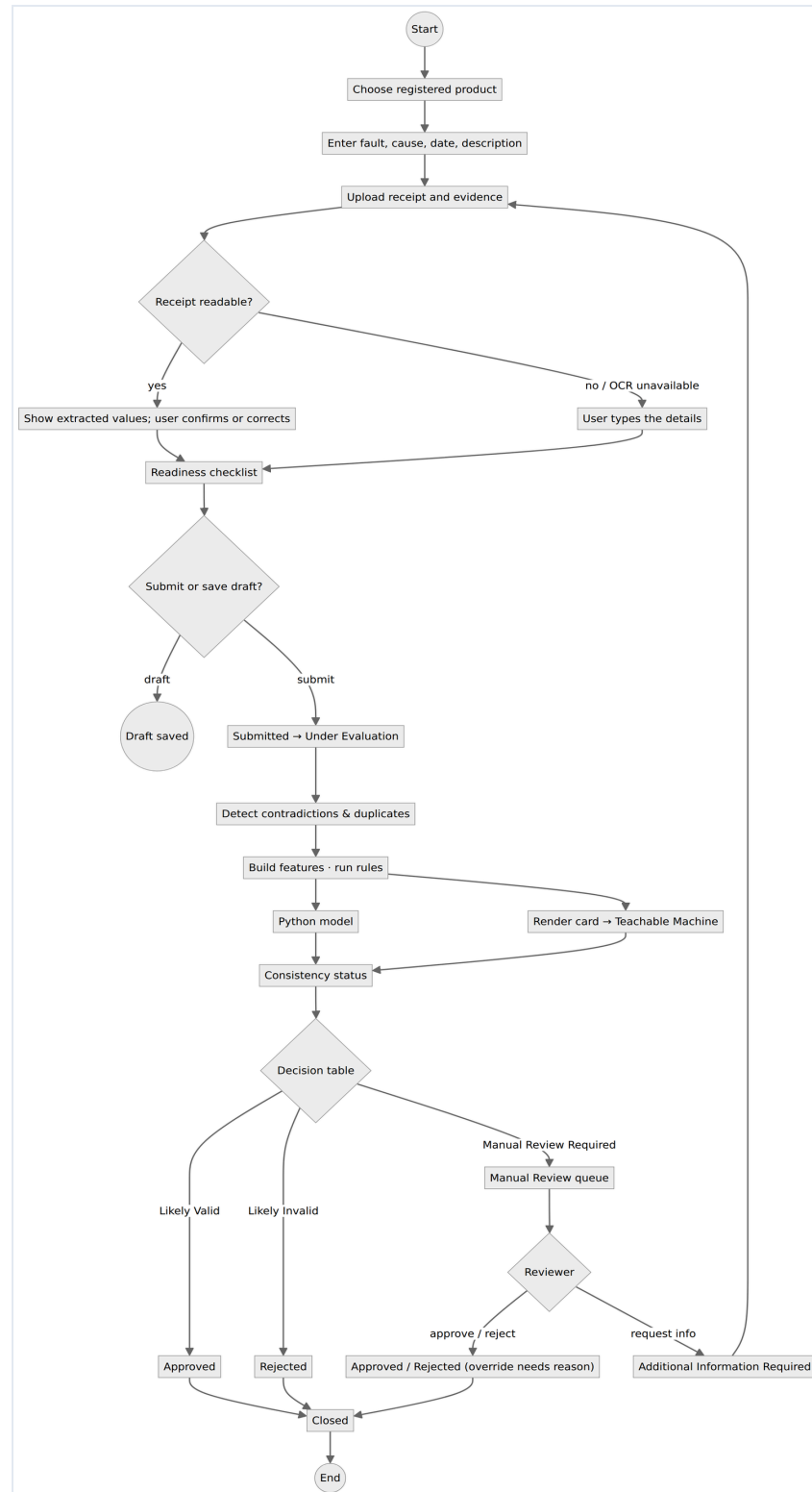
Level-1 DFD

15. Use case diagram



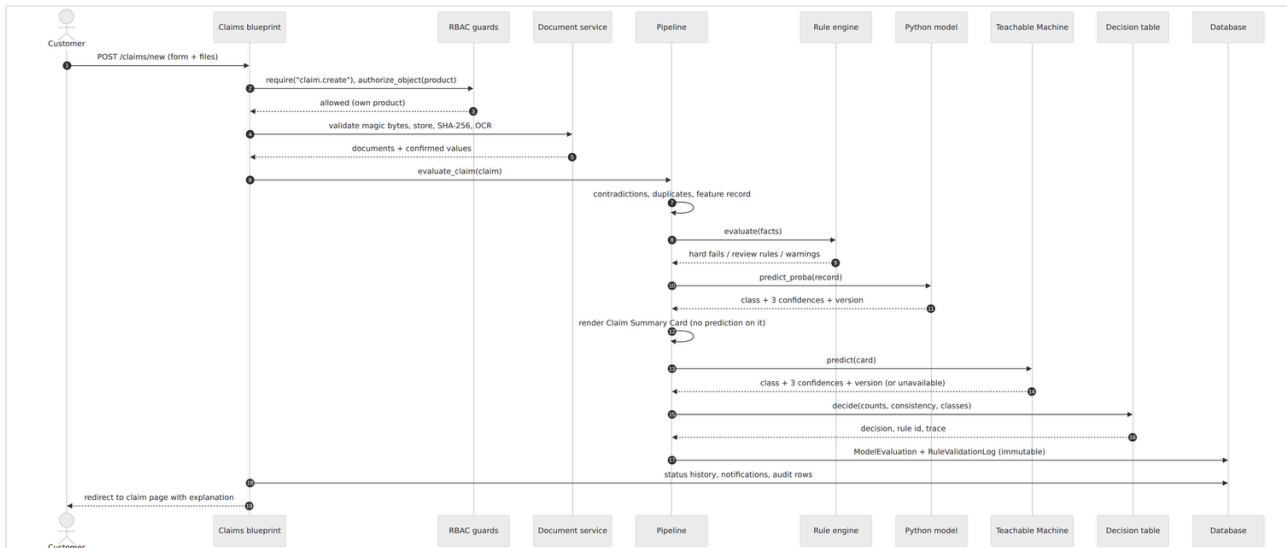
Use cases

16. Activity diagram



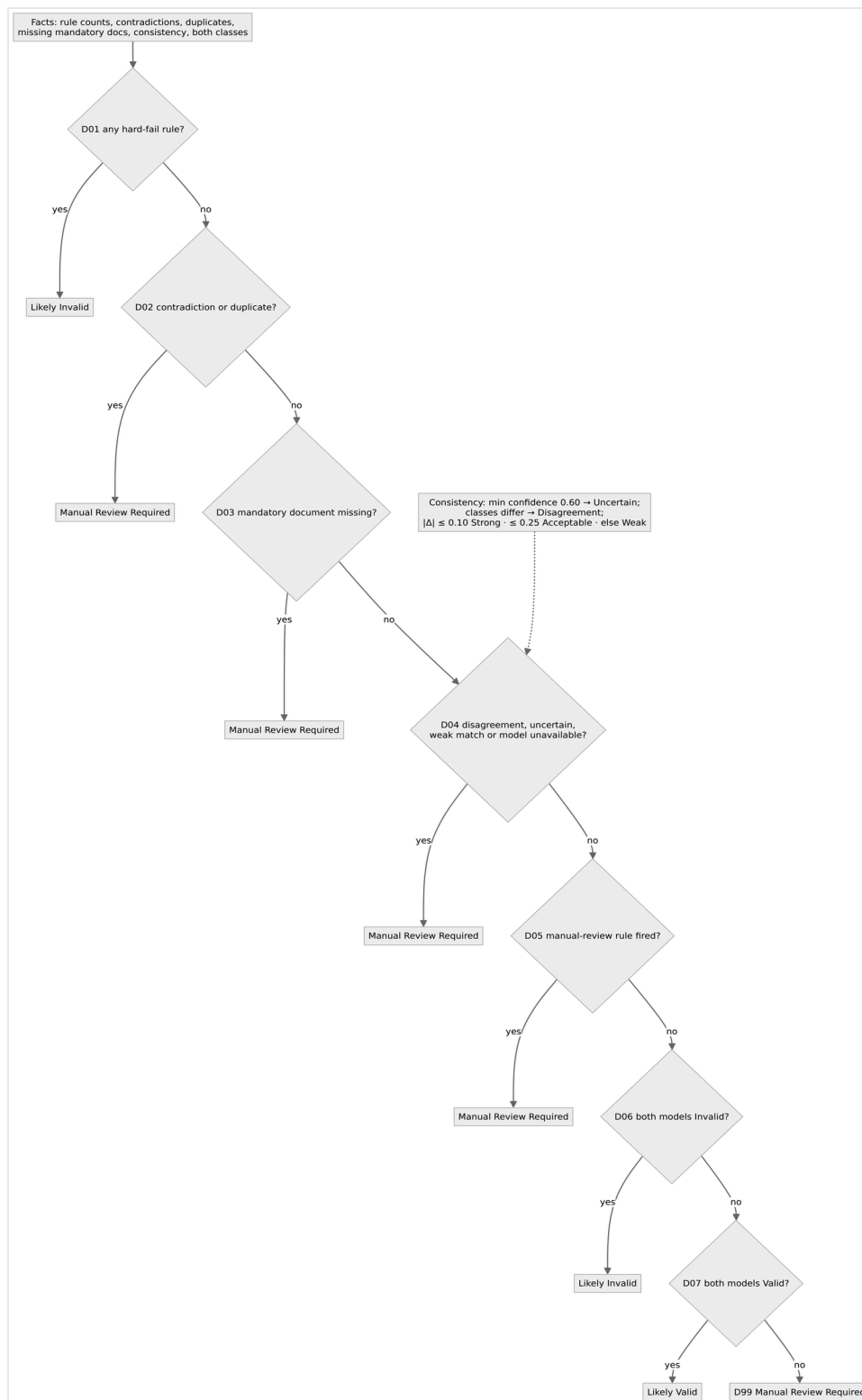
Claim activity

17. Sequence diagram



Evaluation sequence

18. Decision flow diagram



Decision flow

19. Rule-engine design

Each category has a JSON policy with coverage, standard terms, extension length, start conditions, grace period, reporting period, covered faults, excluded damage causes (plus human-readable exclusions), diagnosis threshold for confirming an exclusion, repeat-repair threshold, repair and replacement conditions, authorised-center requirement, mandatory and supporting documents, and three rule lists: `hard_fail_rules`, `manual_review_rules`, `warning_rules`. The engine has sixteen checks

(e.g. WARRANTY_EXPIRED, GRACE_PERIOD, EXCLUDED_DAMAGE, SERIAL_MISMATCH, LATE_REPORTING, INCOMPLETE_EVIDENCE); which list a rule is in decides its severity, and a rule in no list is off. Every result — fired or passed — is stored with a message, so the explanation can show rules passed and failed. Files are validated against the vocabulary on load and on every admin save (atomic write, audited).

20. Python classification model design

Feature record (8 numeric, 10 binary, 2 categorical) → [ColumnTransformer](#) (scaler for the linear model, one-hot with fixed categories) → candidate classifier. Three candidates compared with stratified 5-fold CV and a validation split; the best on validation macro-F1 is wrapped in sigmoid calibration (5-fold) and saved as one pipeline artefact with its feature list and library version. Full evidence:

[documentation/PYTHON_MODEL_EVIDENCE.md](#).

21. Google Teachable Machine model design

Standard image model trained on 2,100 Claim Summary Cards (700 per class; two variations of each training claim), exported as TensorFlow Lite. The card (v3) shows each claim fact against its policy limit as one of nine large colour-and-glyph tiles in a fixed 3 × 3 grid (coverage, reporting time, damage cause, receipt, supporting evidence, serial, dates, invoice, repairs), because Teachable Machine classifies with frozen ImageNet features and cannot read small text. Procedure and status:

[documentation/GTM_EVIDENCE.md](#).

22. Dataset description

1,500 claims, exactly 500 per class, across three categories (Consumer Electronics 464, Home Appliances 514, Industrial Tools 522). Scenarios (from the ground-truth policy): covered defect 487, expired 223, missing documents 144, ambiguous cause 127, unauthorised repair 98, grace boundary 80, excluded damage 76, serial mismatch 60, contradiction 51, repeat repairs 35, possible duplicate 31, late reporting 23, plus 63 noise-flipped labels. This covers the SRS's normal, incomplete, contradictory, complex and borderline cases.

23. Dataset-generation method

[dataset_generator/generator_v2.py](#): attributes are sampled from realistic, overlapping distributions independently of the class (where in the warranty life the claim falls, reporting delay, damage cause by category, diagnosis, repairs, documents, serial match, duplicate invoice, date conflict). The label is then **derived** by an adjudication function that mirrors the policy files, and 4% of labels are flipped to model reviewer disagreement. Records are drawn until each class has 500, shuffled, and only then given claim IDs. A stratified 70/15/15 split follows, before any card is rendered. v1 had a [damage_type](#) column that was a renamed label and claim IDs ordered by class; both are fixed and tested.

24. Data pre-processing and feature engineering

Dates parsed with seven accepted formats; derived fields (product age, days to expiry, remaining warranty, reporting delay, missing-document count, repair count, unauthorised-repair / serial-mismatch / duplicate-invoice / date-conflict flags); missing documents → 0; unassessed diagnosis → 0.5; categorical values validated against the vocabulary and one-hot encoded with fixed categories; numeric scaling inside the pipeline for the linear candidate.

25. Python model-training procedure

`python notebooks/train_python_v2.py`: leakage audit (fails above 90%) → 5-fold CV and validation metrics for each candidate → select on validation → permutation importance → calibrate → one-time test evaluation (accuracy, macro precision/recall/F1, ROC-AUC, log-loss, ECE, confusion matrix, per-class report, 30 sample predictions) → save pipeline and model card. Asserts accuracy ≥ 85%.

26. Google Teachable Machine training procedure

See `documentation/GTM_EVIDENCE.md` §“Training procedure”. After export the model is uploaded in Admin > Models, validated, versioned, and evaluated on the 450 held-out cards.

27. Model evaluation

Model	Accuracy	Macro precision	Macro recall	Macro F1	ROC-AUC
Python (HistGradientBoosting, calibrated), test	89.3%	89.5%	89.3%	89.2%	0.948
Teachable Machine (<code>gtm-514702678b8c</code>), test	93.8%	94.0%	93.8%	93.8%	—

28. Confusion matrix (Python model, test)

actual \ predicted	Invalid	Manual Review	Valid
Invalid Claim	61	8	6
Manual Review	2	69	4
Valid Claim	3	1	71

29. Accuracy, precision, recall and F1 by class (Python model, test)

Class	Precision	Recall	F1
Invalid Claim	92.4%	81.3%	86.5%
Manual Review	88.5%	92.0%	90.2%
Valid Claim	87.7%	94.7%	91.0%

30. Model prediction and confidence comparison

`reports/generate_comparison_report.py` produces `reports/model_comparison_report.csv/.md` for the test claims with every SRS column (IDs, actual class, both predictions and confidences, card filename, match, difference, status, rule result, missing documents, contradictions, duplicates, final decision, explanation of disagreements, summary). Consistency rules: minimum top confidence 0.60 (else *Uncertain Result*); different classes → *Model Disagreement*; $|\Delta| \leq 0.10$ *Strong*, ≤ 0.25 *Acceptable*, otherwise *Weak Match*; the latter three statuses and disagreement all route to manual review. On the test claims the two models agree on 92.9%. If the Teachable Machine export is missing, the report states that its columns are unavailable rather than estimating them.

Every stored verdict also carries its explanation: the three inputs that moved the Python model most (each replaced by its typical training value) and the Claim Summary Card tiles that changed the image model's answer most when blanked out. Administrators can replay all 225 test claims under different agreement thresholds in the what-if simulator before changing them; applying a change saves a new decision-policy version with its simulated effect in the audit trail.

Design check with a simulated agreeing image model (not a reported metric): the rules + decision table map 93.3% of test claims to their labelled class, and 10 of the 15 remaining differences are the deliberately noisy labels — i.e. the combination is sound once the second model is in place.

31. Testing strategy

295 automated pytest tests across unit (rules, detectors, OCR patterns, consistency, RBAC policy), integration (full HTTP journeys through the real app, DB and file store), boundary, negative, security, database, model and demonstration cases; surprise-modification scenarios are tests too. A generated route × role matrix calls every guarded route (68 route/method pairs) as each role and signed out and compares the answer with [config/rbac.json](#). Accuracy guards re-render the test cards with the current code and fail if the image model's accuracy could drop. A separate browser suite (`python -m pytest -m e2e tests/e2e`, 34 tests) runs the main journeys on a live seeded server with the real models, checks accessibility with axe, keyboard use, reduced motion, phone width and session revocation, and fails on any console error. Catalogue: [documentation/TEST_CASES.md](#); screenshots in [docs/screenshots/](#).

32. Security considerations

Deny-by-default RBAC with record scopes and separation of duties; out-of-scope records return 404; user state reloaded per request with session versioning (role change/disable signs out everywhere); idle timeout; generic login errors, five-attempt lockout (one minute, with a countdown on the sign-in page), session rotation, rate-limited sign-in; server-side input rules (no digits in names, no letters in phone numbers, strict email format); public sign-up limited to customers; forced password change for invited users; CSRF tokens on all forms; strict CSP, clickjacking and MIME-sniffing protection; uploads validated by magic bytes, stored under random names outside the web root and served only through permission checks; CSV formula-injection guard; parameterised queries via the ORM; secrets from the environment; append-only audit including denied access. Design and v1 loophole list: [documentation/RBAC_DESIGN.md](#).

33. Privacy considerations

Only data needed to decide a claim is collected. Each role sees only its scope; documents are never publicly addressable; exports require an explicit permission and are audited. The repository contains only fictitious demo data and synthetic images. Deployments should use HTTPS (secure cookies are on by default), restrict database backups, and define a retention period for closed claims and uploaded files.

34. Limitations

- The Teachable Machine model must be retrained in the browser whenever the card design or policy limits change.
- Synthetic training data; rare scenarios (late reporting) are learned less well; real data needs retraining.
- Image OCR requires Tesseract on the server.
- Duplicate text similarity is lexical ($\geq 90\%$ sequence match), not semantic.
- SQLite and in-memory rate limits suit a single server; use PostgreSQL/Redis for multiple workers.
- The saved Python model needs scikit-learn 1.9.1 (Python 3.11+). On the free PythonAnywhere account used for the live demo (Python 3.10, 512 MB disk) the models can be unavailable, and claims then go to manual review.

35. Future enhancements

Retrain on real claims with periodic recalibration and drift monitoring; feed the reviewer-decision export (Admin > Overview) back into training; image-based damage detection on customer photos; semantic duplicate detection; email/SMS notifications; multi-tenant manufacturer support; REST API for service-center systems; database migrations with Alembic.

2. Dataset

One common dataset of synthetic warranty claims in two linked forms: structured CSV records for the Python model and Claim Summary Card images for Google Teachable Machine. Each CSV record and its card carry the same Claim ID, claim details and class label. Splits are made before any card is rendered, so a validation or test claim never appears in training in either form.

Item	File
Raw dataset (1,500 claims)	data/raw/common_warranty_claims_1500.csv
Training / validation / test CSV	data/splits/train.csv, val.csv, test.csv (1,050 / 225 / 225)
Model-ready feature matrices	data/processed/{train,val,test}_features.csv
Card images	data/summary_cards/train/{valid,invalid>manual_review}/ (700 each), val/ (225), test/ (225)
Claim ID ↔ image mapping	data/claim_id_to_card_mapping.csv
Labels	data/labels.csv
Scenario definitions	data/claim_scenarios.json
Statistics	data/dataset_statistics.json
Data dictionary	data/data_dictionary.json
Generation scripts	dataset_generator/generator_v2.py, dataset_generator/build_cards_v2.py

Dataset statistics

Class	Claims
Manual Review	500
Valid Claim	500
Invalid Claim	500

By category

Value	Count
Industrial Tools	522
Home Appliances	514
Consumer Electronics	464

Claim scenarios and labelling rules

rules are applied in this order; the first that matches decides the label. 65 claims (4.3%) carry a deliberately flipped label (scenario suffix '+noise') to simulate reviewer disagreement.

Scenario	Class	Definition	Claims
expired	Invalid Claim	Claim filed after the warranty and its grace period ended.	237
unauthorized_repair	Invalid Claim	The product was previously repaired by an unauthorised center.	102
excluded_damage	Invalid Claim	Damage cause is excluded by the category policy and the technician's diagnosis confirms it (confidence at or above the policy threshold).	81

Scenario	Class	Definition	Claims
serial_mismatch	Invalid Claim	Serial number differs from the receipt and the serial photo.	63
contradiction	Manual Review	Dates contradict each other (fault reported before purchase).	53
late_reporting	Invalid Claim	Fault reported later than the category's reporting period.	24
grace_boundary	Manual Review	Warranty ended but the claim is inside the grace period.	83
possible_duplicate	Manual Review	The invoice was already used on another claim.	31
missing_documents	Manual Review	Receipt missing, or two or more supporting documents missing.	149
serial_unverifiable	Manual Review	Serial mismatch that cannot be verified (evidence incomplete).	1
ambiguous_cause	Manual Review	Excluded cause without a confirming diagnosis, or unknown cause with low diagnostic confidence.	130
repeat_repairs	Manual Review	Previous repairs at or above the policy's repeat-repair threshold.	36
covered_defect	Valid Claim	Covered fault, inside the warranty, complete evidence, consistent data.	510

Claim ID and image filename mapping (first rows)

claim_id	split	claim_class	variation	filename	relative_path
CLM-00055	train	Invalid Claim	1	CLM-00055_v1.jpg	train/invalid/CLM-00055_v1.jpg
CLM-00055	train	Invalid Claim	2	CLM-00055_v2.jpg	train/invalid/CLM-00055_v2.jpg
CLM-00910	train	Manual Review	1	CLM-00910_v1.jpg	train/manual_review/CLM-00910_v1.jpg
CLM-00910	train	Manual Review	2	CLM-00910_v2.jpg	train/manual_review/CLM-00910_v2.jpg
CLM-01375	train	Invalid Claim	1	CLM-01375_v1.jpg	train/invalid/CLM-01375_v1.jpg
CLM-01375	train	Invalid Claim	2	CLM-01375_v2.jpg	train/invalid/CLM-01375_v2.jpg
CLM-00083	train	Valid Claim	1	CLM-00083_v1.jpg	train/valid/CLM-00083_v1.jpg
CLM-00083	train	Valid Claim	2	CLM-00083_v2.jpg	train/valid/CLM-00083_v2.jpg

Sample Claim Summary Cards

One training card per class. A card shows claim facts only — never the Python prediction, a confidence score or the final decision.

Claim summary
CLM-00004 · Consumer Electronics · Motherboard failure

✓ Within coverage 18 days left	✓ Reported in time 28 days · limit 30	✓ Manufacturing Defect covered cause · diag 0.75
✓ Receipt on file	✓ Evidence 3 of 3 supporting	✓ Serial matches registered = evidence
✓ Dates consistent purchase < fault < claim	✓ Invoice unique not seen before	✓ Repairs 0 previous

Purchased 15/02/2024 · claim 28/01/2026
Claim facts vs policy limits. Contains no model output or decision.

Valid Claim: CLM-00004_v1.jpg

Claim summary
CLM-00006 · Home Appliances · Thermostat failure

✗ Coverage ended 243 days ago (grace 14)	✓ Reported in time 4 days · limit 45	! Accidental Drop excluded cause · diag 0.19
✓ Receipt on file	✓ Evidence 3 of 3 supporting	✓ Serial matches registered = evidence
✓ Dates consistent purchase < fault < claim	✓ Invoice unique not seen before	✓ Repairs 1 previous

Purchased 26/06/2023 · claim 24/02/2026
Claim facts vs policy limits. Contains no model output or decision.

Invalid Claim: CLM-00006_v1.jpg

Claim summary
CLM-00001 · Consumer Electronics · Wi-Fi/Bluetooth failure

✓ Within coverage 826 days left	✓ Reported in time 5 days · limit 30	✓ Manufacturing Defect covered cause · diag 0.79
✗ Receipt missing	✓ Evidence 3 of 3 supporting	✓ Serial matches registered = evidence
✓ Dates consistent purchase < fault < claim	✓ Invoice unique not seen before	✓ Repairs 1 previous

Purchased 24/10/2025 · claim 21/07/2026
Claim facts vs policy limits. Contains no model output or decision.

Manual Review: CLM-00001_v1.jpg

3. Python classification model evidence

Source of every number below: [model/python_model/model_card_v2.json](#), written by [notebooks/train_python_v2.py](#) (seed 42, deterministic — re-running gives the same artefact hash 514e2fb68286...). The live values are also shown in the app under Admin > Analytics.

Data

Item	Value
Dataset	data/raw/common_warranty_claims_1500.csv — 1,500 claims, 500 per class
Split (stratified, before any card was rendered)	train 1,050 · validation 225 · test 225, 350/75/75 per class
Isolation	a claim ID belongs to exactly one split (asserted in the generator and by tests/test_ml_integrity.py)
Label noise	4% of labels flipped on purpose (reviewer disagreement) → realistic ceiling ≈ 96%

Features (20) and pre-processing

Group	Features	Handling
Numeric (8)	purchase_price, warranty_duration_months, product_age_days, days_to_expiry, reporting_delay_days, diagnostic_confidence, previous_repairs_count, missing_document_count	Standardised (StandardScaler) for the linear model; raw for tree models
Binary (10)	is_extended_warranty, has_receipt, has_warranty_card, has_damage_photo, has_serial_photo, has_repair_report, serial_number_match, unauthorized_repair_flag, duplicate_invoice_flag, claim_date_conflict_flag	Passed through as 0/1
Categorical (2)	product_category (3 values), damage_type (7 values)	One-hot with the fixed vocabulary from src/core/vocab.py ; an unseen value raises an error instead of being silently zeroed

Never features: claim_id, scenario, claim_class, fault_description, identifiers (asserted in the trainer).

Missing values. In the web app, absent documents become 0, no repairs 0, and an unassessed diagnosis takes the neutral prior 0.5 (the mean of the non-defect training distribution). Categories outside the vocabulary are rejected with a message. **Derived fields:** product age, days to expiry, remaining warranty, reporting delay, missing-document count, conflict / mismatch / duplicate flags — [src/core/features.py](#) builds them with exactly the training column names.

Encoder / pre-processing files. The application loads one artefact, [model/python_model/claim_classifier_v2.joblib](#), which contains the pre-processing and the model together so they can never drift apart. [python notebooks/export_model_artifacts.py](#) extracts the pieces from that saved model (nothing is refitted) and a test checks they match it exactly:

File	Content
model/python_model/preprocessing_pipeline.joblib	ColumnTransformer : numeric and binary passthrough, one-hot for category and damage type with the fixed vocabulary
model/python_model/label_encoder.joblib	LabelEncoder for Invalid Claim / Manual Review / Valid Claim (ids 0 / 1 / 2)

File	Content
model/python_model/feature_schema.json	Input columns and types, vocabularies, missing-value and encoding rules, the 30 encoded output columns
data/processed/{train,val,test}_features.csv	Model-ready matrices: claim ID, 30 encoded features, class and label id
data/labels.csv, data/claim_scenarios.json	Label, split, scenario and card files per claim; scenario definitions and counts

Leakage audit

A depth-3 decision tree trained on **one feature at a time** (validation accuracy; chance = 33%):

Feature	Accuracy alone
days_to_expiry	52.9%
product_age_days	49.8%
missing_document_count	45.3%
has_receipt	42.7%
unauthorized_repair_flag	40.9%

The trainer refuses to continue if any single feature exceeds 90% (v1's `damage_type` was a renamed label and scored 100%).

Algorithms tested (5-fold stratified CV on train, then the validation split)

Algorithm	Hyper-parameters	CV macro-F1	Val accuracy	Val precision	Val recall	Val macro-F1
Logistic regression	C=1.0, max_iter=2000, scaled inputs	0.739 ± 0.026	72.9%	73.3%	72.9%	72.6%
Random forest	400 trees, min_samples_leaf=2, class_weight=balanced	0.922 ± 0.018	94.2%	94.2%	94.2%	94.2%
HistGradientBoosting (selected)	max_iter=300, learning_rate=0.06, max_leaf_nodes=15, l2=1.0	0.913 ± 0.008	94.7%	94.7%	94.7%	94.7%

Selection uses the **validation** macro-F1 only; the test split is touched once, at the end. The selected pipeline is then wrapped in `CalibratedClassifierCV(method="sigmoid", cv=5)` so its probabilities are usable as confidence scores (the consistency statuses depend on them).

Test results (225 unseen claims)

Metric	Value
Accuracy	89.3% (SRS NFR-4 target 85%)
Macro precision / recall / F1	89.5% / 89.3% / 89.2%
ROC-AUC (one-vs-rest, macro)	0.948
Log-loss	0.406
Expected calibration error	0.082
Mean top-class confidence	82.6%

Confusion matrix (rows actual, columns predicted):

	Invalid	Manual Review	Valid
Invalid Claim	61	8	6
Manual Review	2	69	4
Valid Claim	3	1	71

Class	Precision	Recall	F1	Support
Invalid Claim	92.4%	81.3%	86.5%	75
Manual Review	88.5%	92.0%	90.2%	75
Valid Claim	87.7%	94.7%	91.0%	75

The weakest cell is Invalid → Manual Review (8). Checking the 14 missed Invalid claims by scenario: 5 are *late reporting* (a rare scenario — 23 of 1,500 claims — so the model has seen few examples), 4 carry deliberately flipped labels (noise), 3 are *excluded damage* with a diagnosis near the 0.55 threshold, and 2 are expiry/grace boundary cases. All of these are claims the rule engine decides deterministically (LATE_REPORTING, EXCLUDED_DAMAGE, WARRANTY_EXPIRED are hard-fail rules), which is why the application never relies on the model alone.

Feature importance (permutation, validation split, drop in macro-F1)

days_to_expiry 0.279 · damage_type 0.161 · unauthorized_repair_flag 0.098 · diagnostic_confidence 0.081 · has_receipt 0.076 · serial_number_match 0.057 · missing_document_count 0.056 · reporting_delay_days 0.054

These match the warranty policy: expiry, excluded causes, unauthorised repairs and proof of purchase drive the outcome.

Model version and sample predictions

- Version string stored with every prediction: `v2.0.0+<first 12 hex of the file's SHA-256>`.
- scikit-learn 1.9.1 (pinned in `requirements.txt` so the pickle loads identically everywhere).
- 30 sample test predictions with all three probabilities: `model_card_v2.json` → `test.sample_predictions`; all 225 in `model/python_model/sample_test_predictions.csv` and `reports/model_comparison_report.csv`.

4. Google Teachable Machine evidence

Status: trained and installed. Model [gtm-514702678b8c](#), trained in the browser on the 2,100 v3 training cards: **93.8% test accuracy** (macro F1 93.8%), 97.3% validation accuracy, agreeing with the Python model on 92.9% of test claims. A first model trained on the v2 cards scored 54.2% and was replaced (see *Card design iterations*). The earlier [gtm_classifier.joblib](#) / [weights.bin](#) / [metadata.json](#) files were a scikit-learn stand-in, not a Teachable Machine export, and were removed.

Project evidence (in place of a project link)

The SRS accepts "Google Teachable Machine project link **or** permitted project evidence". The model was trained in the browser and exported directly; the browser project was not saved to Google Drive, so there is no shareable project link. The following evidence is submitted instead, and every item can be checked against the others:

Evidence	Where
Exported Teachable Machine model (TensorFlow Lite, float)	model/teachable_machine/model_unquant.tflite ; its SHA-256 begins 514702678b8c , which is the model version gtm-514702678b8c shown in the application and stored with every prediction
Label file (the three classes, in Teachable Machine's order)	model/teachable_machine/labels.txt
Screenshots of the three classes, training and in-browser testing	screenshots/23_gtm_v3_training.png (the three classes, 700 samples each, model trained), screenshots/24_gtm_v3_preview_test.png (preview test), screenshots/25_gtm_v3_evaluation.png (evaluation); first run: screenshots/20_gtm_training.png , screenshots/22_gtm_evaluation.png
Class samples (Valid Claim, Invalid Claim, Manual Review)	the training cards in data/summary_cards/train/{valid,invalid,manual_review}/ (700 per class) and the validation and test cards in data/summary_cards/val/ and data/summary_cards/test/
Training configuration and retraining details	<i>Training record</i> below (epochs 50, batch 16, learning rate 0.001; run 1 on v2 cards, run 2 on v3 cards)
Evaluation on unseen cards	model/teachable_machine/evaluation.json (225 validation and 225 test cards: 97.3% and 93.8%)
Incorrectly classified samples	reports/gtm_misclassified_test.csv , reports/gtm_misclassified_val.csv
Comparison with the Python model	reports/model_comparison_report.md and .csv (225 test claims)

Reproduce the figures: [python notebooks/evaluate_gtm.py](#) re-scores the installed model on the 225 validation and 225 test cards, or sign in as the administrator and use **Admin > Models > Run evaluation**. The test suite also re-scores the model on every run and fails if its accuracy differs from the recorded 93.8% ([tests/test_ml_integrity.py](#)).

Claim Summary Cards

Item	Value
Renderer	src/core/card_v2.py (same code for training cards and live claims)
Size	600 × 600 px (square, so Teachable Machine's centre crop keeps every element)
Fonts	DejaVu Sans shipped in static/fonts/ (identical rendering on Windows / Linux / servers)

Item	Value
Content (v3)	Claim facts compared with the category's policy limits, as a fixed 3×3 grid of large tiles: coverage (within / grace / ended), reporting time (in time / late), damage cause (covered / excluded, with diagnostic confidence), receipt, supporting evidence (3 of 3 ... 0 of 3), serial (matches / differs / unverified), dates (consistent / conflict), invoice (unique / reused), repairs (count / repeat / unauthorised). Each state has its own colour and glyph (✓ ! × ↔); a colour band shows the category. Limits come from policies/*.json , the same source as the rule engine
Never on a card	Python prediction, any confidence score, final decision, class label
Training images	2 variations per training claim (colour jitter, date format, ±1.5° rotation, blur, JPEG quality) → 2,100 images: 700 per class in data/summary_cards/train/{valid,invalid,manual_review}/
Validation / test	one canonical card per claim (<claim_id>_v0.jpg) in data/summary_cards/{val,test}/ — never uploaded for training
Mapping	data/claim_id_to_card_mapping.csv (claim ID, split, class, variation, filename, path)

Samples: Admin > Models shows one training card per class; any card can be opened from a claim page.

Card design iterations (measured)

Card	How measured	Validation	Test	Notes
v2 (grey bars, small tiles and pips)	Real Teachable Machine export gtm-ec95b4aa49b5 , evaluated in the app	52.9%	54.2% (macro F1 45.1%)	agreed with the Python model on 55.1% of test claims; below the SRS 85% target
v2	notebooks/tm_replica_check.py , 2 seeds	63.1–74.7%	58.2–65.8%	the replica reproduces the failure, so the card, not the training run, was the problem
v3 (policy-relative colour tiles)	notebooks/tm_replica_check.py , 3 seeds	95.1–97.3%	92.9–94.2% (mean 93.6%)	pre-check before the browser run
v3	Real Teachable Machine export gtm-514702678b8c , evaluated in the app	97.3%	93.8% (macro F1 93.8%)	agrees with the Python model on 92.9% of test claims; ceiling ≈ 96% (4% deliberate label noise)

Why v2 failed: Teachable Machine does not fine-tune the image network; it trains a small head on frozen ImageNet (MobileNet) features. Thin grey bars and 20-pixel pips at 224×224 produce almost identical features for all three classes. v3 makes every fact a large region whose colour and glyph change with the fact's value relative to the policy limit, which those frozen features separate easily. The card still shows facts only — no class, decision, rule outcome or model confidence.

The replica estimate (92.9–94.2%) matched the real v3 export (93.8%), so the offline check is a useful way to test a card design before spending a browser training run.

Training procedure

1. Admin > Models > *Download training cards* (zip with three class folders), or use the folders above.
2. [teachablemachine.withgoogle.com](#) → *Image Project* → *Standard image model*.
3. Rename the classes exactly **Valid Claim**, **Invalid Claim**, **Manual Review**; upload 700 images each.
4. *Advanced*: start with epochs 50, batch size 16, learning rate 0.001. Train.
5. *Export Model* → *Tensorflow Lite* → *Floating point* → download; unzip.

6. Admin > Models > upload `model_unquant.tflite` and `labels.txt` (checked: TFLite header, exactly the three labels, the model loads and runs; the previous model is archived under `model/teachable_machine/versions/`).
7. Run *evaluation* (or `python notebooks/evaluate_gtm.py`) and `python reports/generate_comparison_report.py`.

Confusion matrix (test, 225 cards; rows = actual, columns = predicted)

	Valid Claim	Invalid Claim	Manual Review
Valid Claim	73	1	1
Invalid Claim	5	68	2
Manual Review	4	1	70

Most errors predict *Valid Claim* for claims labelled Invalid or Manual Review (9 of 14). 12 of the 14 errors are claims whose label was deliberately flipped, where the model's answer is the policy's answer (see *Training observations*); in the application no claim is marked *Likely Valid* unless the rule engine and the Python model agree as well.

Training observations

- **Run 1 (v2 cards).** Training finished normally (epochs 50, batch 16, learning rate 0.001), but on our 225 test cards the model reached only 54.2% and agreed with the Python model on 55.1% of claims: a card that looks clear to a person is not necessarily clear to frozen ImageNet features at 224 × 224 pixels.
- **Diagnosis.** An offline replica of the trainer (`notebooks/tm_replica_check.py`) reproduced the failure (58–66%), so the problem was the input representation, not the training run or its settings.
- **Run 2 (v3 cards).** Same settings. Validation 97.3%, test 93.8%; the preview in Teachable Machine gave *Invalid Claim* 99% for an unseen card with a date conflict, late reporting and an excluded cause (`screenshots/24_gtm_v3_preview_test.png`).
- **Where it still errs.** 12 of its 14 test errors are claims whose label was deliberately flipped to simulate reviewer disagreement (the test split has 12 such claims): on every one of them the image model gave the answer the warranty policy gives. Its only 2 real errors are a grace-period claim called *Invalid* (should be reviewed) and a late-reporting claim called *Manual Review* (should be invalid). Measured against the policy instead of the noisy labels it agrees on 223 of 225 test claims (99.1%). The misclassified cards are listed with their scenario in `reports/gtm_misclassified_test.csv`.
- **Browser behaviour.** With 2,100 images Chrome showed "Page unresponsive" during "Preparing training data"; choosing *Wait* and keeping the tab in front let training finish (about 20–25 minutes).
- **Deployment.** TensorFlow 2.20 no longer ships `tf.lite.Interpreter`; the app uses `ai-edge-litert` (Windows and Linux). Models are loaded from bytes so a new export can replace a running one on Windows.

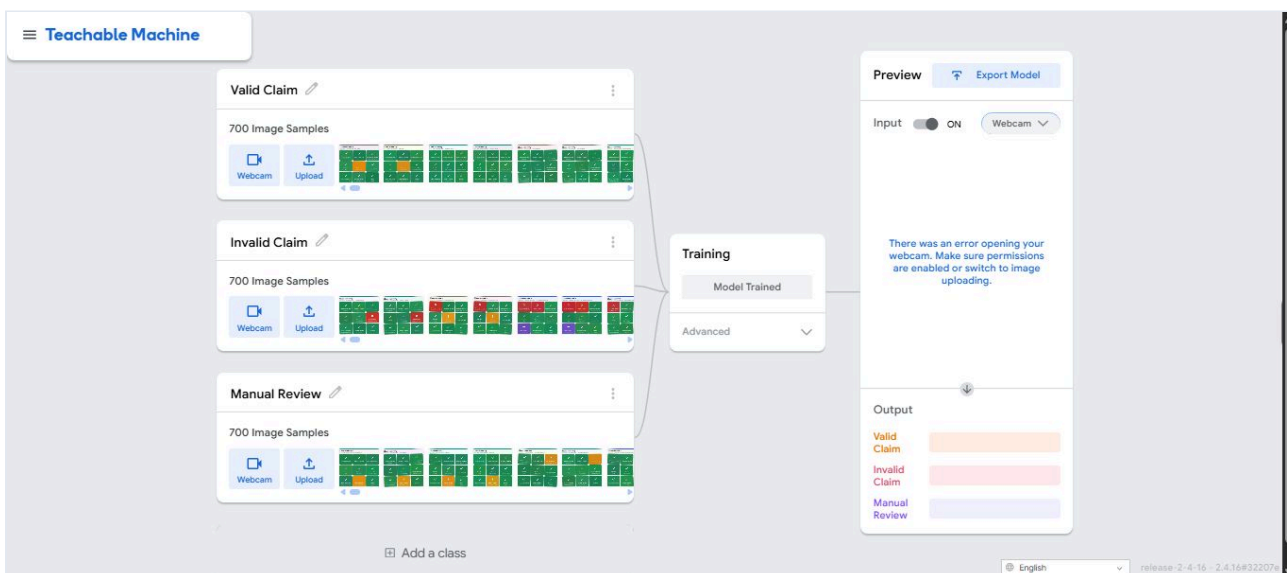
Runtime

`src/core/gtm_classifier_v2.py` — TensorFlow Lite interpreter (`ai-edge-litert`), preprocessing identical to the Teachable Machine export sample (ImageOps.fit to the model input, scaled to [-1, 1]; quantized exports are supported). The model version is `gtm-<SHA-256 prefix of the .tflite file>` and is stored with every prediction. If the file is missing or broken the prediction is marked *unavailable* and the claim goes to manual review — there is no fallback model.

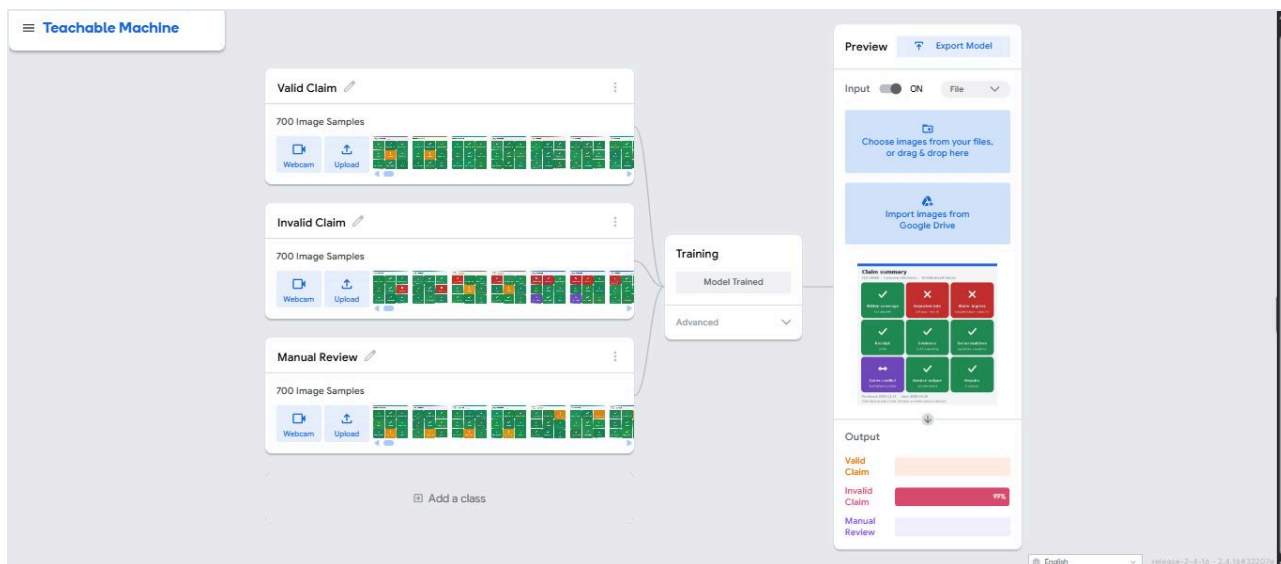
Training record

Item	Value
Model version	gtm-514702678b8c (model/teachable_machine/model_unquant.tflite, float, input 224×224)
Classes	Valid Claim · Invalid Claim · Manual Review (labels.txt)
Images per class	700 / 700 / 700 v3 training cards (two variations of each of the 1,050 training claims)
Training configuration	Standard image model; epochs 50, batch size 16, learning rate 0.001
Validation accuracy (225 cards)	97.3%
Test accuracy (225 cards) · macro F1	93.8% · 93.8%
Agreement with the Python model (test)	92.9% of claims
Macro precision / recall (test)	94.0% / 93.8%
Per class (test): precision / recall	Valid 89.0% / 97.3% · Invalid 97.1% / 90.7% · Manual Review 95.9% / 93.3%
Full evaluation	model/teachable_machine/evaluation.json; comparison with the Python model in reports/model_comparison_report.md (application decision accuracy 92.0%)
Incorrectly classified samples	reports/gtm_misclassified_test.csv (written by the evaluation)
Retraining details	Run 1 on v2 cards → 54.2% test (archived as gtm-ec95b4aa49b5). Cause found with notebooks/tm_replica_check.py; card redesigned (v3); run 2 on v3 cards → 93.8%
Screenshots	Run 1: screenshots/20_gtm_training.png, 22_gtm_evaluation.png. Run 2: 23_gtm_v3_training.png, 24_gtm_v3_preview_test.png, 25_gtm_v3_evaluation.png

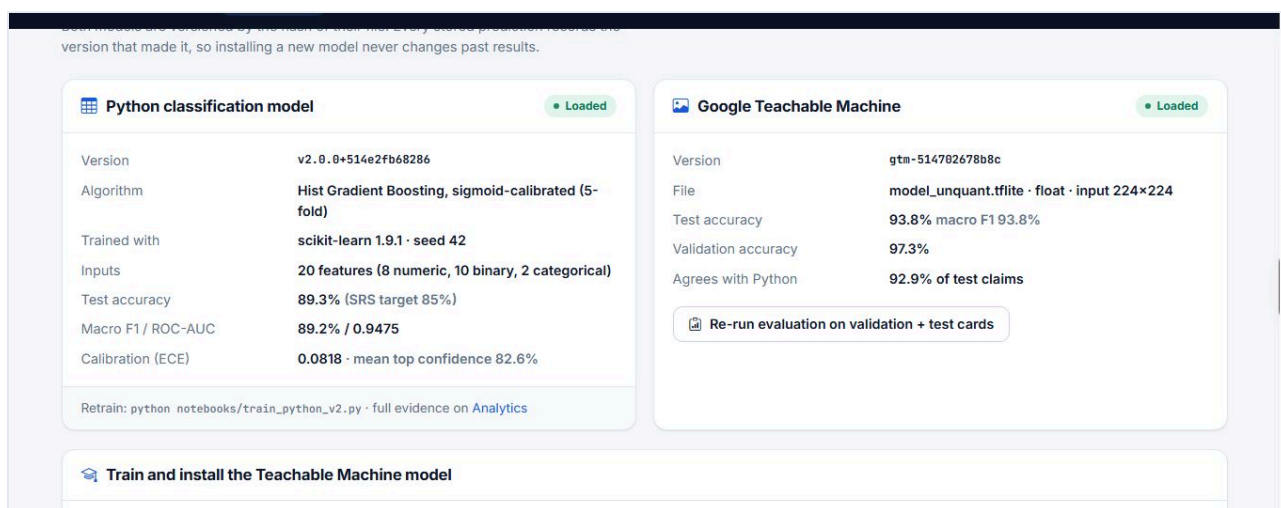
Teachable Machine screenshots



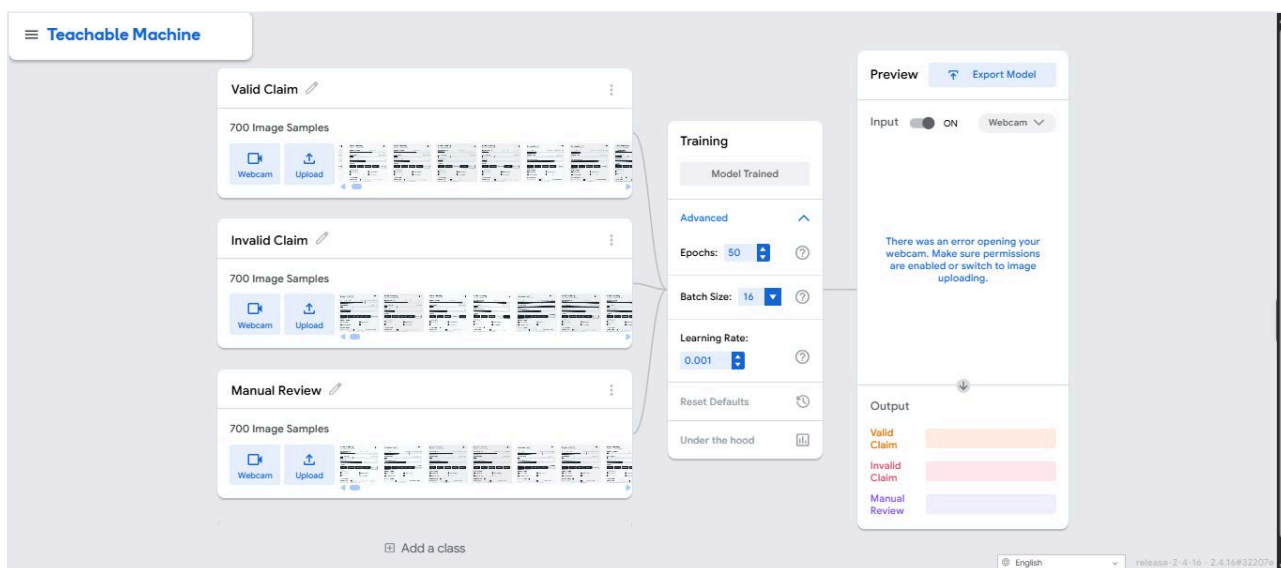
Run 2 (v3 cards): the three classes, 700 images each, model trained



Run 2: preview test on an unseen card



Run 2: evaluation of the installed model in the application



Run 1 (v2 cards): training

CLAIM ENGINE

Administration

Both models are versioned by the hash of their file. Every stored prediction records the version that made it, so installing a new model never changes past results.

Python classification model

Loaded

Version	v2.0.0+514e2fb68286
Algorithm	Hist Gradient Boosting, sigmoid-calibrated (5-fold)
Trained with	scikit-learn 1.9.1 · seed 42
Inputs	20 features (8 numeric, 10 binary, 2 categorical)
Test accuracy	89.3% (SRS target 85%)
Macro F1 / ROC-AUC	89.2% / 0.9475
Calibration (ECE)	0.0818 · mean top confidence 82.6%

Retrain: python · notebooks/train_python_v2.py · full evidence on [Analytics](#)

Google Teachable Machine

Loaded

Version	gtm-ec95b4aa49b5
File	model_unquant.tflite · float · input 224×224
Test accuracy	54.2% macro F1 45.1%
Validation accuracy	52.9%
Agrees with Python	55.1% of test claims

Re-run evaluation on validation + test cards

Train and install the Teachable Machine model

Run 1: evaluation (54.2% test accuracy, replaced)

5. Model prediction and confidence comparison report

225 unseen claims. Python model on the structured record; Google Teachable Machine on the claim's canonical Claim Summary Card; rules and decision table as in production.

Overall comparison summary

Metric	Value
Python model accuracy	89.3%
Python model macro F1	89.2%
Teachable Machine accuracy	93.8%
Teachable Machine macro F1	93.8%
Predicted-class agreement	92.9%
Application decision accuracy (decision → class)	92.0%

Consistency statuses: Strong Match 117, Acceptable Match 68, Uncertain Result 15, Weak Match 13, Model Disagreement 12

Final decisions: Manual Review Required 81, Likely Valid 74, Likely Invalid 70

Major disagreements

- **CLM-00016** (actual Manual Review): Python says Manual Review (0.61); Teachable Machine says Invalid Claim (0.64). Rules: GRACE_PERIOD, REPEAT_REPAIRS.
- **CLM-00106** (actual Valid Claim): Python says Invalid Claim (0.66); Teachable Machine says Valid Claim (0.77). Rules: none fired.
- **CLM-00433** (actual Valid Claim): Python says Invalid Claim (0.84); Teachable Machine says Valid Claim (0.94). Rules: none fired.
- **CLM-01370** (actual Invalid Claim): Python says Manual Review (0.69); Teachable Machine says Invalid Claim (0.92). Rules: LATE_REPORTING, UNKNOWN_CAUSE.
- **CLM-00309** (actual Invalid Claim): Python says Manual Review (0.75); Teachable Machine says Invalid Claim (0.97). Rules: WARRANTY_EXPIRED.
- **CLM-00189** (actual Manual Review): Python says Invalid Claim (0.51); Teachable Machine says Manual Review (0.52). Rules: GRACE_PERIOD.
- **CLM-00517** (actual Valid Claim): Python says Invalid Claim (0.63); Teachable Machine says Valid Claim (0.82). Rules: none fired.
- **CLM-00419** (actual Invalid Claim): Python says Invalid Claim (0.91); Teachable Machine says Valid Claim (0.93). Rules: none fired.
- **CLM-00809** (actual Valid Claim): Python says Valid Claim (0.73); Teachable Machine says Invalid Claim (0.99). Rules: EXCLUDED_DAMAGE.
- **CLM-01313** (actual Invalid Claim): Python says Manual Review (0.69); Teachable Machine says Invalid Claim (0.99). Rules: LATE_REPORTING, GRACE_PERIOD, EXCLUDED_DAMAGE_UNCONFIRMED.
- **CLM-01456** (actual Invalid Claim): Python says Manual Review (0.55); Teachable Machine says Invalid Claim (0.97). Rules: LATE_REPORTING, GRACE_PERIOD, EXCLUDED_DAMAGE_UNCONFIRMED.
- **CLM-00632** (actual Invalid Claim): Python says Manual Review (0.87); Teachable Machine says Invalid Claim (0.90). Rules: EXCLUDED_DAMAGE.

- **CLM-01040** (actual Invalid Claim): Python says Valid Claim (0.45); Teachable Machine says Invalid Claim (0.98). Rules: EXCLUDED_DAMAGE.
- **CLM-01479** (actual Invalid Claim): Python says Valid Claim (0.60); Teachable Machine says Invalid Claim (0.96). Rules: EXCLUDED_DAMAGE.
- **CLM-00077** (actual Invalid Claim): Python says Manual Review (0.62); Teachable Machine says Invalid Claim (0.90). Rules: LATE_REPORTING, REPEAT_REPAIRS.
- **CLM-01474** (actual Manual Review): Python says Invalid Claim (0.57); Teachable Machine says Manual Review (0.90). Rules: GRACE_PERIOD.

Per-claim results (first 30 of 225 unseen test claims)

Every column, including the warranty-rule result, missing documents, contradictions, duplicate indicators and the explanation of each disagreement, is in [reports/model_comparison_report.csv](#).

Claim ID	Actual class	Python predicted class	Python conf. Valid	Python conf. Invalid	Python conf. Manual	Card filename	GTM predicted class	GTM conf. Valid	GTM conf. Invalid	GTM conf. Manual	Classes match	Top-confidence difference	Consistency status	Final application decision
CLM-01428	Manual Review	Manual Review	0.02	0.0393	0.9407	CLM-01428_v0.jpg	Manual Review	0.0077	0.1115	0.8807	True	0.06	Strong Match	Manual Review Required
CLM-00367	Valid Claim	Valid Claim	0.7903	0.0957	0.114	CLM-00367_v0.jpg	Valid Claim	0.9529	0.0462	0.0009	True	0.1626	Acceptable Match	Likely Valid
CLM-00654	Valid Claim	Valid Claim	0.73	0.0355	0.2345	CLM-00654_v0.jpg	Valid Claim	0.8961	0.0996	0.0043	True	0.1661	Acceptable Match	Likely Valid
CLM-00539	Invalid Claim	Invalid Claim	0.0119	0.8961	0.092	CLM-00539_v0.jpg	Invalid Claim	0.0002	0.9925	0.0073	True	0.0964	Strong Match	Likely Invalid
CLM-00816	Manual Review	Manual Review	0.0271	0.2208	0.7521	CLM-00816_v0.jpg	Manual Review	0.0064	0.25	0.7435	True	0.0086	Strong Match	Manual Review Required
CLM-01383	Valid Claim	Valid Claim	0.8138	0.0412	0.1449	CLM-01383_v0.jpg	Valid Claim	0.9699	0.0299	0.0002	True	0.1561	Acceptable Match	Likely Valid
CLM-00508	Valid Claim	Valid Claim	0.6899	0.0157	0.2943	CLM-00508_v0.jpg	Valid Claim	0.8816	0.1168	0.0017	True	0.1917	Acceptable Match	Likely Valid
CLM-00850	Valid Claim	Valid Claim	0.8673	0.0182	0.1145	CLM-00850_v0.jpg	Valid Claim	0.9098	0.0866	0.0036	True	0.0425	Strong Match	Likely Valid
CLM-00535	Invalid Claim	Invalid Claim	0.0478	0.8358	0.1165	CLM-00535_v0.jpg	Invalid Claim	0.0128	0.7717	0.2155	True	0.0641	Strong Match	Likely Invalid
CLM-00988	Invalid Claim	Valid Claim	0.9376	0.0305	0.0319	CLM-00988_v0.jpg	Valid Claim	0.8571	0.1418	0.0011	True	0.0805	Strong Match	Likely Valid
CLM-00458	Manual Review	Manual Review	0.0167	0.0853	0.8981	CLM-00458_v0.jpg	Manual Review	0.0012	0.0009	0.9979	True	0.0998	Strong Match	Manual Review Required
CLM-00266	Manual Review	Manual Review	0.1796	0.071	0.7494	CLM-00266_v0.jpg	Manual Review	0.009	0.4777	0.5133	True	0.2361	Uncertain Result	Manual Review Required
CLM-00159	Valid Claim	Valid Claim	0.8131	0.0949	0.0919	CLM-00159_v0.jpg	Valid Claim	0.8727	0.0059	0.1214	True	0.0596	Strong Match	Likely Valid
CLM-00925	Invalid Claim	Invalid Claim	0.0123	0.8131	0.1746	CLM-00925_v0.jpg	Invalid Claim	0.0056	0.9368	0.0575	True	0.1237	Acceptable Match	Likely Invalid
CLM-01117	Valid Claim	Valid Claim	0.7763	0.0452	0.1785	CLM-01117_v0.jpg	Valid Claim	0.888	0.1112	0.0008	True	0.1117	Acceptable Match	Likely Valid
CLM-01444	Invalid Claim	Invalid Claim	0.0244	0.8322	0.1434	CLM-01444_v0.jpg	Invalid Claim	0.0237	0.9598	0.0165	True	0.1276	Acceptable Match	Likely Invalid
CLM-01196	Valid Claim	Valid Claim	0.7772	0.0127	0.2102	CLM-01196_v0.jpg	Valid Claim	0.9563	0.0417	0.002	True	0.1791	Acceptable Match	Likely Valid

Claim ID	Actual class	Python predicted class	Python conf. Valid	Python conf. Invalid	Python conf. Manual	Card filename	GTM predicted class	GTM conf. Valid	GTM conf. Invalid	GTM conf. Manual	Classes match	Top-confidence difference	Consistency status	Final application decision
CLM-01441	Invalid Claim	Invalid Claim	0.0086	0.7925	0.1989	CLM-01441_v0.jpg	Invalid Claim	0.0011	0.9562	0.0426	True	0.1637	Acceptable Match	Likely Invalid
CLM-00906	Valid Claim	Valid Claim	0.8365	0.0215	0.142	CLM-00906_v0.jpg	Valid Claim	0.9508	0.011	0.0382	True	0.1143	Acceptable Match	Likely Valid
CLM-00627	Invalid Claim	Invalid Claim	0.0055	0.9561	0.0385	CLM-00627_v0.jpg	Invalid Claim	0.0028	0.9819	0.0153	True	0.0258	Strong Match	Likely Invalid
CLM-00016	Manual Review	Manual Review	0.0064	0.3826	0.611	CLM-00016_v0.jpg	Invalid Claim	0.0195	0.6429	0.3376	False	0.0319	Model Disagreement	Manual Review Required
CLM-01154	Valid Claim	Valid Claim	0.9429	0.0238	0.0333	CLM-01154_v0.jpg	Valid Claim	0.8931	0.1062	0.0007	True	0.0498	Strong Match	Likely Valid
CLM-00104	Manual Review	Manual Review	0.1524	0.0086	0.8391	CLM-00104_v0.jpg	Manual Review	0.0348	0.0059	0.9592	True	0.1201	Acceptable Match	Manual Review Required
CLM-00066	Valid Claim	Valid Claim	0.7844	0.0144	0.2011	CLM-00066_v0.jpg	Valid Claim	0.9116	0.0872	0.0012	True	0.1272	Acceptable Match	Likely Valid
CLM-00078	Invalid Claim	Invalid Claim	0.0369	0.9013	0.0618	CLM-00078_v0.jpg	Invalid Claim	0.0	0.9994	0.0006	True	0.0981	Strong Match	Likely Invalid
CLM-01425	Invalid Claim	Invalid Claim	0.0167	0.9616	0.0218	CLM-01425_v0.jpg	Invalid Claim	0.0003	0.997	0.0027	True	0.0354	Strong Match	Likely Invalid
CLM-01421	Valid Claim	Valid Claim	0.9415	0.0185	0.0401	CLM-01421_v0.jpg	Valid Claim	0.9019	0.0947	0.0033	True	0.0396	Strong Match	Likely Valid
CLM-00050	Invalid Claim	Invalid Claim	0.0287	0.8274	0.1439	CLM-00050_v0.jpg	Invalid Claim	0.0006	0.9989	0.0004	True	0.1715	Acceptable Match	Likely Invalid
CLM-00003	Valid Claim	Valid Claim	0.915	0.0172	0.0677	CLM-00003_v0.jpg	Valid Claim	0.8721	0.1263	0.0016	True	0.0429	Strong Match	Likely Valid
CLM-00777	Manual Review	Manual Review	0.0077	0.051	0.9413	CLM-00777_v0.jpg	Manual Review	0.0011	0.004	0.9948	True	0.0535	Strong Match	Manual Review Required

6. Warranty policy files

Warranty rules are configuration, not code: one JSON file per category in [policies/](#), loaded by [src/rules/policy_store.py](#) and editable (with preview and version history) in Admin > Policies.

Consumer Electronics — [policies/consumer_electronics.json](#)

Field	Value
category	Consumer Electronics
policy name	Standard Consumer Electronics Warranty Policy
coverage duration months	12
standard terms months	12, 24
extended extra months	12
start conditions	Coverage starts on the purchase date shown on the verified invoice.
grace period days	7
claim reporting period days	30
covered faults	Screen flickering, Motherboard failure, Battery not charging, Speaker malfunction, Touch panel unresponsive, Wi-Fi/Bluetooth failure
excluded damage types	Accidental Drop, Water Ingress, Electrical Surge
exclusions	Liquid ingress or immersion, Screen shattering from drops, Power surge / overvoltage damage, Cosmetic casing wear, Commercial rental use
exclusion min diagnostic confidence	0.55
repeat repair review threshold	2
authorized service center required	True
repair conditions	Repairs must be carried out by an authorised service center using genuine parts.
replacement conditions	Unit replacement is considered only after a repeated failure of the same component.
mandatory documents	receipt
supporting documents	warranty_card, serial_photo, damage_photo
hard fail rules	WARRANTY_EXPIRED, UNAUTHORIZED_REPAIR, EXCLUDED_DAMAGE, SERIAL_MISMATCH, LATE_REPORTING
manual review rules	GRACE_PERIOD, EXCLUDED_DAMAGE_UNCONFIRMED, SERIAL_UNVERIFIED, INCOMPLETE_EVIDENCE, UNKNOWN_CAUSE, REPEAT_REPAIRS, PRIOR_REPLACEMENT, UNCOVERED_FAULT
warning rules	NEAR_REPORTING_DEADLINE, WARRANTY_ENDING_SOON, SUPPORTING_DOCUMENT_MISSING

Home Appliances — [policies/home_appliances.json](#)

Field	Value
category	Home Appliances

Field	Value
policy name	Major Home Appliances Comprehensive Protection Policy
coverage duration months	24
standard terms months	24, 36
extended extra months	12
start conditions	Coverage starts on the purchase date shown on the verified invoice.
grace period days	14
claim reporting period days	45
covered faults	Compressor failure, Motor burnout, Thermostat failure, Drum spin malfunction, PCB failure, Water pump leakage
excluded damage types	Water Ingress, Accidental Drop
exclusions	External water / flood damage, Impact damage during relocation, Improper electrical supply rating, Pest infestation, Commercial heavy utilisation
exclusion min diagnostic confidence	0.55
repeat repair review threshold	2
authorized service center required	True
repair conditions	Repairs must be carried out by an authorised service center using genuine parts.
replacement conditions	Unit replacement is considered only after a repeated failure of the same component.
mandatory documents	receipt
supporting documents	warranty_card, serial_photo, damage_photo
hard fail rules	WARRANTY_EXPIRED, UNAUTHORIZED_REPAIR, EXCLUDED_DAMAGE, SERIAL_MISMATCH, LATE_REPORTING
manual review rules	GRACE_PERIOD, EXCLUDED_DAMAGE_UNCONFIRMED, SERIAL_UNVERIFIED, INCOMPLETE_EVIDENCE, UNKNOWN_CAUSE, REPEAT_REPAIRS, PRIOR_REPLACEMENT, UNCOVERED_FAULT
warning rules	NEAR_REPORTING_DEADLINE, WARRANTY_ENDING_SOON, SUPPORTING_DOCUMENT_MISSING

Industrial Tools — [policies/industrial_tools.json](#)

Field	Value
category	Industrial Tools
policy name	Heavy Duty Industrial Equipment & Power Tools Policy
coverage duration months	24
standard terms months	24, 36
extended extra months	12
start conditions	Coverage starts on the purchase date shown on the verified invoice.

Field	Value
grace period days	14
claim reporting period days	30
covered faults	Armature burnout, Seal failure, Gearbox seizure, Chuck bearing failure, Trigger switch failure
excluded damage types	Accidental Drop, Water Ingress
exclusions	Tool dropped from height, Submersion in water, slurry or solvent, Overload beyond rated torque, Consumable wear (brushes, chuck teeth, cords)
exclusion min diagnostic confidence	0.55
repeat repair review threshold	2
authorized service center required	True
repair conditions	Repairs must be carried out by an authorised service center using genuine parts.
replacement conditions	Unit replacement is considered only after a repeated failure of the same component.
mandatory documents	receipt
supporting documents	warranty_card, serial_photo, damage_photo
hard fail rules	WARRANTY_EXPIRED, UNAUTHORIZED_REPAIR, EXCLUDED_DAMAGE, SERIAL_MISMATCH, LATE_REPORTING
manual review rules	GRACE_PERIOD, EXCLUDED_DAMAGE_UNCONFIRMED, SERIAL_UNVERIFIED, INCOMPLETE_EVIDENCE, UNKNOWN_CAUSE, REPEAT_REPAIRS, PRIOR_REPLACEMENT, UNCOVERED_FAULT
warning rules	NEAR_REPORTING_DEADLINE, WARRANTY_ENDING_SOON, SUPPORTING_DOCUMENT_MISSING

7. Test cases and results

Run everything: `python -m pytest -q` → **295 passed** (about 60 s); the browser suite `python -m pytest -m e2e tests/e2e` → **34 passed** (about 140 s). Every test is automated; the table maps each SRS test category to the functions that cover it (`file :: function`).

SRS category	What is checked	Tests
Functional	Register product with receipt; full claim journey (submit → evaluate → approve, notifications, audit, card, PDF, tracking); drafts; request-info loop; reviewer take/decide; admin pages & exports	<code>test_workflow ::</code> <code>test_customer_registers_product_with_receipt,</code> <code>test_full_claim_journey_with_agreeing_models,</code> <code>test_draft_edit_then_submit,</code> <code>test_request_info_loop,</code> <code>test_reviewer_queue_and_take,</code> <code>test_admin_tools,</code> <code>test_public_pages</code>
Integration	Real Flask app + DB + file store + OCR + both model runtimes + rule engine end to end	<code>test_workflow :: *</code> , <code>test_pipeline ::</code> <code>test_both_models_agree_valid_is_approved,</code> <code>test_detectors ::</code> <code>test_repair_serial_feeds_contradictions</code>
Boundary	Last day of cover, first/last day of grace, first day after grace; reporting deadline ± 1 day; exclusion at 0.54 / 0.55 diagnosis; consistency at $\Delta = 0.10$ / 0.25 and confidence = 0.60	<code>test_rules ::</code> <code>test_warranty_boundaries,</code> <code>test_reporting_window_boundaries,</code> <code>test_exclusion_needs_diagnosis_to_hard_fail,</code> <code>test_pipeline ::</code> <code>test_consistency_boundaries,</code> <code>test_ml_integrity ::</code> <code>test_consistency_matrix</code>
Negative	Bad product form, future dates, unknown damage type, fake/oversized/mismatched files, non-receipt PDF, non-TFLite model upload, invalid policy values, invalid decision thresholds	<code>test_workflow ::</code> <code>test_product_form_validation,</code> <code>test_wizard_rejects_bad_input,</code> <code>test_gtm_upload_rejects_non_tflite,</code> <code>test_policy_editor_validates_and_audits,</code> <code>test_documents ::</code> <code>test_invalid_files_are_refused,</code> <code>test_oversized_file_is_refused,</code> <code>test_rules ::</code> <code>test_policy_validation_rejects_bad_files,</code> <code>test_decision_policy_save_rejects_invalid</code>
Security	No unguarded route; registration can't create admins; generic login errors; logout; session revoked on role change/disable; IDOR returns 404; staff/reviewer scope; separation of duties; admin self-protection; CSRF; CSP & headers; open redirect; POST-only logout; denials audited; CSV formula injection	<code>test_rbac_routes :: *</code> , <code>test_rbac_policy :: *</code> , <code>test_workflow ::</code> <code>test_csrf_is_enforced,</code> <code>test_security_headers_and_csp,</code> <code>test_open_redirect_blocked,</code> <code>test_logout_is_post_only_and_clears_session,</code> <code>test_admin_tools</code>

SRS category	What is checked	Tests
Database	Unique public IDs enforced; every evaluation a new immutable row; status history and audit rows written with each change	<pre>test_workflow :: test_database_constraints, test_pipeline :: test_reevaluation_keeps_earlier_results, test_workflow :: test_full_claim_journey_with_agreeing_models</pre>
OCR	PDF receipt read (invoice, serial, date, amount); entity patterns; words not mistaken for identifiers; OCR engine missing → accept & type; user corrections saved and audited	<pre>test_documents :: test_pdf_receipt_is_read_by_ocr, test_entity_patterns, test_words_are_not_mistaken_for_identifiers, test_image_receipt_accepted_when_ocr_engine_missing, test_workflow :: test_scan_receipt_endpoint, test_ocr_values_confirmed_by_user_are_saved_and_audited</pre>
Python model	Accuracy $\geq 85\%$ and not suspiciously perfect; no single-feature leakage; three calibrated probabilities; unknown category rejected; failure handled	<pre>test_ml_integrity :: test_model_meets_srs_accuracy_and_is_not_suspiciously_perfect, test_unknown_category_is_rejected_not_silently_zeroed, test_pipeline :: test_python_model_scores_all_three_classes, test_unknown_category_is_rejected_by_model, test_python_model_failure_is_handled</pre>
Teachable Machine	Missing model → <i>Uncertain Result</i> + manual review (no fallback); label-file parsing incl. folder names; installer rejects non-TFLite files; card is 600×600 and prediction-free	<pre>test_pipeline :: test_without_teachable_machine_claim_goes_to_manual_review, test_teachable_machine_label_file_parsing, test_card_stored_and_contains_no_prediction, test_workflow :: test_gtm_upload_rejects_non_tflite, test_gtm_upload_replaces_and_archives_running_model</pre>
Accuracy guards	The installed Teachable Machine model is re-scored on test cards rendered with the current card code and policies ($\geq 85\%$ and equal to the recorded 93.8%); the saved Python model is re-scored on the test split ($\geq 85\%$ and equal to the recorded 89.3%); editing a policy limit shown on the card warns that the image model needs retraining	<pre>test_ml_integrity :: test_teachable_machine_keeps_its_accuracy_on_freshly_rendered_cards, test_installed_teachable_machine_model_matches_its_evaluation, test_saved_python_model_still_scores_its_recorded_accuracy, test_workflow :: test_changing_a_card_limit_warns_that_teachable_machine_needs_retraining</pre>

SRS category	What is checked	Tests
Model artefacts & exports	Exported label encoder, preprocessing pipeline and processed CSVs reproduce the saved model exactly; Excel export is a valid workbook and neutralises formulas	<code>test_ml_integrity ::</code> <code>test_exported_encoder_preprocessing_and_processed_data_match_the_model,</code> <code>test_workflow ::</code> <code>test_excel_export_is_a_real_workbook_and_neutralises_formulas</code>
Model comparison	All five consistency statuses; disagreement; low confidence; hard fail beats agreeing models	<code>test_ml_integrity ::</code> <code>test_consistency_matrix,</code> <code>test_pipeline ::</code> <code>test_model_disagreement_goes_to_manual_review,</code> <code>test_low_confidence_image_model_is_uncertain,</code> <code>test_hard_fail_wins_over_agreeing_models</code>
Rule engine	Every rule fires and passes; per-category exclusions; severities from config; policy files complete (SRS deliverable 7)	<code>test_rules :: *</code>
Contradiction detection	Claim before purchase, fault before purchase, fault after claim, repair before purchase, model/invoice/purchase-date mismatch	<code>test_detectors ::</code> <code>test_date_contradictions,</code> <code>test_model_invoice_and_purchase_date_mismatch,</code> <code>test_consistent_claim_has_no_contradictions</code>
Missing documents	Mandatory vs supporting documents; product-level receipt counts; D03 routing; evidence rules	<code>test_detectors ::</code> <code>test_missing_documents_follow_policy,</code> <code>test_product_level_receipt_counts_for_claims,</code> <code>test_rules ::</code> <code>test_evidence_rules</code>
Duplicate claims	Same file hash on another product; invoice reused; open claim on the same serial; near-identical description; closed history is not a duplicate	<code>test_detectors ::</code> <code>test_duplicate_document_hash_across_products,</code> <code>test_invoice_reuse_and_open_claim_on_same_serial,</code> <code>test_closed_claim_on_same_unit_is_history_not_duplicate</code>
Serial-number mismatch	Checked against receipt, warranty card, serial photo, diagnostic report and repair records; hard fail vs manual review depends on evidence	<code>test_detectors ::</code> <code>test_serial_checked_against_every_source,</code> <code>test_repair_serial_feeds_contradictions,</code> <code>test_rules ::</code> <code>test_serial_mismatch_severity_depends_on_evidence</code>
Low confidence	Confidence < 0.60 → <i>Uncertain Result</i> → manual review	<code>test_pipeline ::</code> <code>test_low_confidence_image_model_is_uncertain,</code> <code>test_ml_integrity ::</code> <code>test_consistency_matrix</code>
Model disagreement	Different classes → <i>Model Disagreement</i> → D04	<code>test_pipeline ::</code> <code>test_model_disagreement_goes_to_manual_review,</code> <code>test_demonstration_cases ::</code> <code>11_model_disagreement_claim</code>
Demonstration cases	The 11 SRS cases from sample_claims/	<code>test_demonstration_cases :: *</code>

SRS category	What is checked	Tests
Surprise modifications	New exclusion, rule severity change, confidence threshold, extra date format, decision-logic (routing) change	<code>test_rules :: test_surprise_*</code>
Performance	Both predictions + rules within 5 s (SRS NFR-1); text responses gzipped	<code>test_pipeline ::</code> <code>test_evaluation_is_fast_enough,</code> <code>test_frontend_contracts ::</code> <code>test_text_responses_are_gzipped</code> <code>_only_when_accepted</code>
Access matrix	Every guarded route (68 route/method pairs) called as each role and signed out; the answer must match <code>config/rbac.json</code> (403 with the JSON envelope under <code>/api/</code> , sign-in redirect or 401 when signed out)	<code>test_route_matrix :: *</code>
Frontend contracts	Stored verdict payload complete and consistent (fixed class order, scores sum to 1, decision re-derivable); JSON envelope and error codes; cross-customer 404 on every claim URL; verdict animates once; no <code>\ safe</code> on data, inline styles or un-nonce'd scripts in any template; icon subset complete	<code>test_frontend_contracts :: *</code>
Live bench, what-if, batch	Demo endpoint runs the real pipeline, writes nothing, 400 on unknown cases, 429 on the 11th call; what-if is read-only, reproduces all 225 stored decisions at current thresholds, apply saves a new audited version; batch validates rows, runs a chunk per call, writes no claims, exports CSV, 429 on the 4th upload	<code>test_frontend_contracts ::</code> <code>test_demo_*, test_what_if_*,</code> <code>test_batch_*</code>
Wizard autosave and workbench	Draft created once and reused on submit; another customer's draft is never reused; workbench only for reviewers, next-claim order, decisions return to the workbench through the open-redirect guard	<code>test_frontend_contracts ::</code> <code>test_wizard_autosave_*,</code> <code>test_workbench_*</code>
Input rules	Names, cities, brands, retailers and providers take no digits; phone numbers take no letters; emails are well formed (server-side, 20+ good/bad cases); malformed login email is refused without counting as an attempt; the account pauses for one minute after five wrong passwords and the page counts down; the rate-limit page shows a countdown	<code>test_input_rules :: *</code>
Hero engine (landing)	Layout math (zone inflation, Bézier-rect intersection, rerouting, tiers, outcome mapping) as Node unit tests; in the browser: no engine part overlaps text or the card at 1440/1280/1024/768/390 px, foreground opacity stays 1, text contrast $\geq 4.5:1$ over the rendered background, foreground pixels identical with and without the layer, the lit outcome follows the live check, reduced motion is still	<code>test_hero_engine_math :: *</code> (runs <code>tests/js/*.test.mjs</code>), <code>tests/e2e/test_hero_engine.py</code> <code>:: *</code>
Browser journeys (e2e)	Landing bench, persona sign-in, receipt prefill, wizard with OCR split view, reviewer keyboard flow and double-submit guard, what-if and batch, session revoked on role change, reduced motion, 390 px without sideways scroll, axe WCAG 2 A/AA on 21 pages; any console error fails the test	<code>tests/e2e/test_journeys.py :: *</code>

Hidden-test readiness checklist

- [x] No logic keyed on claim IDs, names or demo values; OCR has no hard-coded retailer/product lists.

- [x] Every categorical value comes from one vocabulary ([src/core/vocab.py](#)); the UI options are tested to equal the training values.
- [x] Unknown categories fail loudly instead of being silently encoded as zeros.
- [x] Model selected on validation, test used once; calibrated probabilities; leakage audit < 90%.
- [x] Dates accepted in seven formats; adding one is a one-line change (tested).
- [x] Rules, thresholds and routing are configuration, editable in the admin UI and validated on save.
- [x] A missing or broken model can never crash a request or silently decide a claim.
- [x] Every prediction is stored with both model versions, the card hash and the decision trace.
- [x] 295 tests pass on a clean checkout (`python -m pytest -q`), and the 34 browser tests pass (`python -m pytest -m e2e tests/e2e`).

Manual checks performed

- Every page rendered for all four roles (customer, staff, reviewer, administrator) with the seeded data: no server errors; out-of-scope records return 404 and forbidden pages 403.
- Browser screenshots at 1440, 1024 and 390 px; the e2e suite writes the current ones to [docs/screenshots/](#).
- Lighthouse (landing, sign-in, model card, blog): performance 99, accessibility 100, best practices 100, SEO 100.
- `python database/seed.py` submits the 11 demonstration claims through the live pipeline.

Latest test run

```
AssureX Claim Engine – automated test results
Command : python -m pytest -q -rA
Date    : 2026-09-28 (UTC)
Python  : 3.11.15 · scikit-learn 1.9.1 · Linux
Result  : 295 passed, 34 deselected in 59.86s
Browser : python -m pytest -m e2e tests/e2e -rA
Result  : 34 passed in 140.65s (0:02:20)
```

Test file	Passed	Failed
tests/e2e/test_hero_engine.py	21	0
tests/e2e/test_journeys.py	13	0
tests/test_demonstration_cases.py	12	0
tests/test_detectors.py	11	0
tests/test_documents.py	19	0
tests/test_frontend_contracts.py	25	0
tests/test_hero_engine_math.py	1	0
tests/test_input_rules.py	62	0
tests/test_ml_integrity.py	37	0
tests/test_navigation.py	3	0
tests/test_notifications.py	3	0
tests/test_pipeline.py	17	0
tests/test_rbac_policy.py	32	0
tests/test_rbac_routes.py	21	0
tests/test_route_matrix.py	2	0
tests/test_rules.py	27	0
tests/test_workflow.py	23	0

Per-test results: [reports/test_results.txt](#).

8. Installation instructions

Prerequisites

Item	Requirement
Operating system	Windows 10/11, macOS 12+, Ubuntu 20.04+ (any OS with Python)
Python	3.11 or 3.12 (3.11 recommended). <code>scikit-learn==1.9.1</code> is pinned so the saved model loads, and it is not published for Python 3.10
Git	any recent version
Optional	Tesseract OCR (reads <i>image</i> receipts); Chromium via Playwright (only to re-render report diagrams)

Setup

```
git clone https://github.com/Saba1512006/assurex-claim-engine.git
cd assurex-claim-engine
python -m venv venv
# Windows (PowerShell): .\venv\Scripts\Activate.ps1      macOS/Linux: source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Environment variables

`python database/seed.py` creates a `.env` file with a random `SECRET_KEY` the first time it runs. You can also set these yourself (in `.env` or the environment):

Variable	Default	Purpose
<code>SECRET_KEY</code>	<i>(required)</i>	Signs sessions and CSRF tokens. The app refuses to start without it.
<code>DATABASE_URL</code>	<code>sqlite:///database/assurex.db</code>	Any SQLAlchemy URL, e.g. PostgreSQL
<code>UPLOAD_DIR</code>	<code>data/uploads</code>	Where documents and Claim Summary Cards are stored
<code>WARRANTY_EXPIRY_ALERT_DAYS</code>	<code>30</code>	Initial alert window (admins change it in the app)
<code>RATELIMIT_STORAGE_URI</code>	<code>memory://</code>	Use <code>redis://...</code> when running several workers
<code>FLASK_DEBUG</code>	<code>0</code>	<code>1</code> for the debug server

Database

```
python database/seed.py          # drop + recreate tables, add demo users, products and 12 evaluated claims
python database/seed.py --if-empty # only seed an empty database (used on deploy)
```

Tables are created from the SQLAlchemy models (`src/models/entities.py`). The seed prints the status mix it produced. The **default administrator** is `admin@assurex.local` / `AdminPass123!` — change the password from *Profile* after the first sign-in on any shared deployment, and invite real users from *Access*.

Models

Model	Location	How it gets there
Python classification model	<code>model/python_model/claim_classifier_v2.joblib</code> + <code>model_card_v2.json</code>	Included. Rebuild with <code>python notebooks/train_python_v2.py</code>
Google Teachable Machine	<code>model/teachable_machine/model_unquant.tflite</code> + <code>labels.txt</code>	Train in the browser on <code>data/summary_cards/train/</code> , export <i>Tensorflow Lite</i> → <i>Floating point</i> , then upload in Admin > Models (or copy the two files into the folder). Run the evaluation afterwards.

`labels.txt` may use the canonical names (0 Valid Claim) or the folder names (0 valid, 2 manual_review).

OCR

PDF receipts with a text layer are read by `pdfplumber` (installed from requirements). To read photos and scans, install Tesseract:

- Windows: install from the UB Mannheim build, keep the default path `C:\Program Files\Tesseract-OCR\`, and add it to `PATH`.
- macOS: `brew install tesseract` · Ubuntu/Debian: `sudo apt install tesseract-ocr`

Without Tesseract, image receipts are still accepted and the user types the details (the app says so).

Run

```
python src/app.py                # development server: http://127.0.0.1:5000
gunicorn wsgi:app --workers 2    # production (Linux/macOS)
python -m pytest -q              # 295 automated tests (+ 34 browser tests: -m e2e
tests/e2e)
```

PythonAnywhere: point the WSGI file at the project and `from wsgi import application`; set `SECRET_KEY` in the WSGI file or a `.env`, then run `python database/seed.py --if-empty` once in a console. Use a **Python 3.11** web app and virtualenv: the saved Python model needs `scikit-learn==1.9.1`, which is not published for Python 3.10. On Linux the image model needs `ai-edge-litert` or `tflite-runtime`. Free accounts have a 512 MB disk quota; install with `pip install --no-cache-dir` and keep `numpy<2` if the system packages were built against NumPy 1.x. Render: `render.yaml` builds, seeds an empty database and generates `SECRET_KEY` automatically.

Folder setup

Everything needed is in the repository. Created at runtime: `data/uploads/` (documents, cards), `database/assurex.db`, `.env`. None of them are committed.

Troubleshooting

Symptom	Fix
<code>RuntimeError: SECRET_KEY is not set</code>	Run <code>python database/seed.py</code> once, or set <code>SECRET_KEY</code>
Sign-in works but you're immediately signed out (local http)	Use <code>python src/app.py</code> (it disables the secure-cookie flag for http). Behind https nothing needs changing.
"Python model could not be loaded" / models <i>Unavailable</i> on the live site	Install the pinned <code>scikit-learn==1.9.1</code> (Python 3.11+), or retrain with <code>python notebooks/train_python_v2.py</code>

Symptom	Fix
Every claim ends in Manual Review	The Teachable Machine model isn't installed yet (Admin > Models shows the status)
"No TensorFlow Lite runtime installed"	<code>pip install ai-edge-litert</code> (or <code>tflite-runtime</code> / <code>tensorflow</code>)
Receipt photo: "no text found"	Install Tesseract or upload a PDF; you can also enter the details by hand
<code>sqlite3.OperationalError: no such column</code> after pulling	The schema changed: <code>python database/seed.py</code> (demo) or migrate your data (see <code>src/security/MIGRATION.md</code>)
"This form expired"	The CSRF token is tied to your session — reload the page and submit again
Too many attempts / account locked	Wait 1 minute (the sign-in page shows a countdown), or an admin uses <i>Sign out</i> on the Access page to clear the lock

9. Execution instructions and user guide

Quick start

Requirements: Python 3.11 or 3.12 (3.11 recommended), Git. Works on Windows, macOS and Linux.

```
git clone https://github.com/Saba1512006/assurex-claim-engine.git
cd assurex-claim-engine
python -m venv venv
source venv/bin/activate           # Windows: .\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python database/seed.py           # creates the database + demo data, writes .env with a
random SECRET_KEY
python src/app.py                 # http://127.0.0.1:5000
```

Run the tests: `python -m pytest -q` (295 tests, plus 34 browser tests; latest results in `reports/test_results.txt`). Browser suite (13 journeys with accessibility checks): `pip install -r requirements-dev.txt && python -m playwright install chromium && python -m pytest -m e2e tests/e2e`. Production: `gunicorn wsgi:app` (Render uses `render.yaml`; PythonAnywhere's WSGI file imports `application` from `wsgi.py`). Set `SECRET_KEY` in the environment — the app refuses to start without one.

How to use it

Task	Where
Register / sign in	/register , /login
Register a product and its warranty	<i>Products > Register product</i> . Upload the receipt first — it is read by OCR and fills the form; check every value. Choose standard or extended cover.
Add warranty information / documents	Product page > <i>Product documents</i> (receipt, warranty card, serial photo) — used by every claim on that product
Create a claim	<i>New claim</i> — 4 steps: product → what happened → evidence → review
Upload documents	Wizard step 3, or the claim page > <i>Evidence</i> while the claim is a draft, in review, or waiting for information
Verify extracted information	Wizard step 3 shows the values read from the receipt in editable fields; on the claim page each document has <i>Extracted details</i> . Corrections are audited.
Submit	Wizard <i>Submit claim</i> (or <i>Save as draft</i> then <i>Submit claim</i> on the claim page)
Python prediction & confidence	Claim page > verdict, left column: predicted class and probability for all three classes (calibrated: 0.80 means right about 80% of the time), plus the inputs that moved it most
Claim Summary Card	Claim page > <i>What the image model saw</i> , with the tiles that drove its answer outlined
Teachable Machine prediction	Claim page > verdict, right column (or “Model unavailable” until the export is installed)
Compare both models	The agreement gauge between the two columns: both needles, the gap Δ , the minimum-confidence mark and the consistency status
Warranty-rule results	Claim page > <i>Why this decision</i> : failed and review rules first, passed rules underneath
Contradictions / duplicates	Same list; duplicates also flag the claim in lists
Manual-review queue	Reviewer or admin > <i>Review queue</i> , then the workbench (/reviewer/claim/<id>)
Administrator dashboard	Admin > <i>Overview</i> (filters by category, status and date) and <i>Analytics</i>
Try other thresholds safely	Admin > <i>What-if simulator</i>

Task	Where
Evaluate many records at once	Admin > <i>Batch evaluation</i> (CSV in the training-split format)
Choose what customers see	Admin > <i>Policies</i> > <i>What customers see</i> (model probabilities on or off)
Track a claim	Claim page > <i>Track progress</i> , or the 8-stage bar at the top of the claim page
Export a claim report	Claim page > <i>Download report (PDF)</i> ; bulk CSV or Excel (.xlsx) from Admin > <i>Overview, Analytics, Audit</i> and claim <i>Search</i>

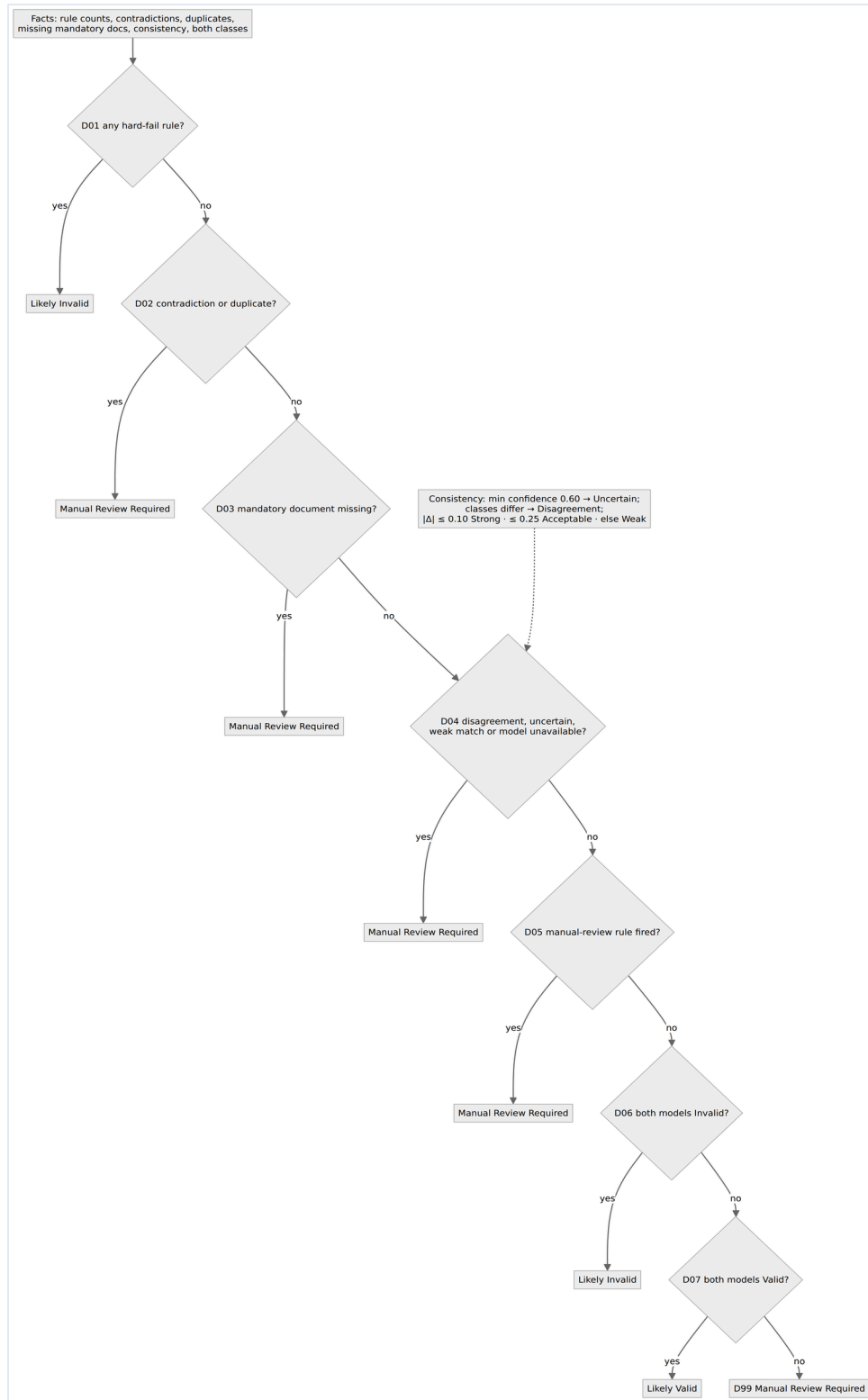
The 11 SRS demonstration cases

`python database/seed.py` creates one real claim for each case, submitted through the live pipeline: valid, invalid (excluded damage), manual review (unknown cause), expired warranty, missing receipt, duplicate (reused receipt and invoice), contradictory dates, serial mismatch, unauthorised repair, boundary date (inside the grace period) and a model disagreement (*KitchenPro Oven 45L*: the image model says Valid, the Python model Invalid, so the claim goes to a reviewer). The same cases exist as feature records in `sample_claims/` and are pinned by `tests/test_demonstration_cases.py`.

How a claim is decided

1. **Evidence** — files are checked by magic bytes (not the extension), stored under random names, SHA-256 fingerprinted, and OCR-read (PDF text via pdfplumber; images via Tesseract when installed).
2. **Contradictions** — claim date vs purchase date, fault date vs purchase/claim dates, repair dates, and the registered serial / model / invoice / purchase date vs every value read from the evidence.
3. **Duplicates** — the same file hash on another product or claim, an invoice number registered to another product, another open claim on the same serial, a near-identical fault description, the same claimant repeating the same fault within 30 days.
4. **Features** — `src/core/features.py` builds exactly the 20 columns the model was trained on (product age, days to expiry, reporting delay, missing-document count, flags ...).
5. **Rules** — `policies/<category>.json` decides each rule's severity: *hard fail*, *manual review*, *warning* (advisory) or off. Coverage, grace period, reporting deadline, excluded causes, evidence rules and repair history are all in the file.
6. **Python model** → class + 3 probabilities. **Claim Summary Card** (600×600, facts only, never a prediction) → **Teachable Machine** → class + 3 probabilities.
7. **Consistency** (`config/decision_policy.json`): top confidence below 0.60 → *Uncertain Result*; different classes → *Model Disagreement*; $|\Delta| \leq 0.10$ *Strong*, ≤ 0.25 *Acceptable*, otherwise *Weak Match*.
8. **Decision table** — first matching row wins: D01 hard fail → Likely Invalid; D02 contradiction or duplicate, D03 missing mandatory document, D04 disagreement / uncertain / weak / model unavailable, D05 manual-review rule → Manual Review Required; D06 both Invalid → Likely Invalid; D07 both Valid → Likely Valid; otherwise D99 Manual Review Required. `routing` maps each recommendation to a status.

Every evaluation is a new, immutable database row holding the exact inputs, both model versions (file hashes), the card hash, the thresholds and the decision trace — updating a model never changes a recorded result.



Decision flow

Security

Deny-by-default access control ([config/rbac.json](#)): permission → record scope (*own, service center, assigned or queue, all*) → constraints (reviewers can't decide claims they filed, a claim can only move along allowed transitions, overrides need a 20-character reason, admins can't demote themselves or the last admin). Out-of-scope records return 404. Users are reloaded from the database on every request and a role change or disable signs them out everywhere. Also: generic login errors, 5-attempt lockout, session rotation, CSRF on every form, rate-limited sign-in, strict Content-Security-Policy (no

inline or third-party scripts), magic-byte upload validation, CSV formula-injection protection and an append-only audit trail. Details: [documentation/RBAC_DESIGN.md](#). Landing hero background: [documentation/HERO_ENGINE.md](#).

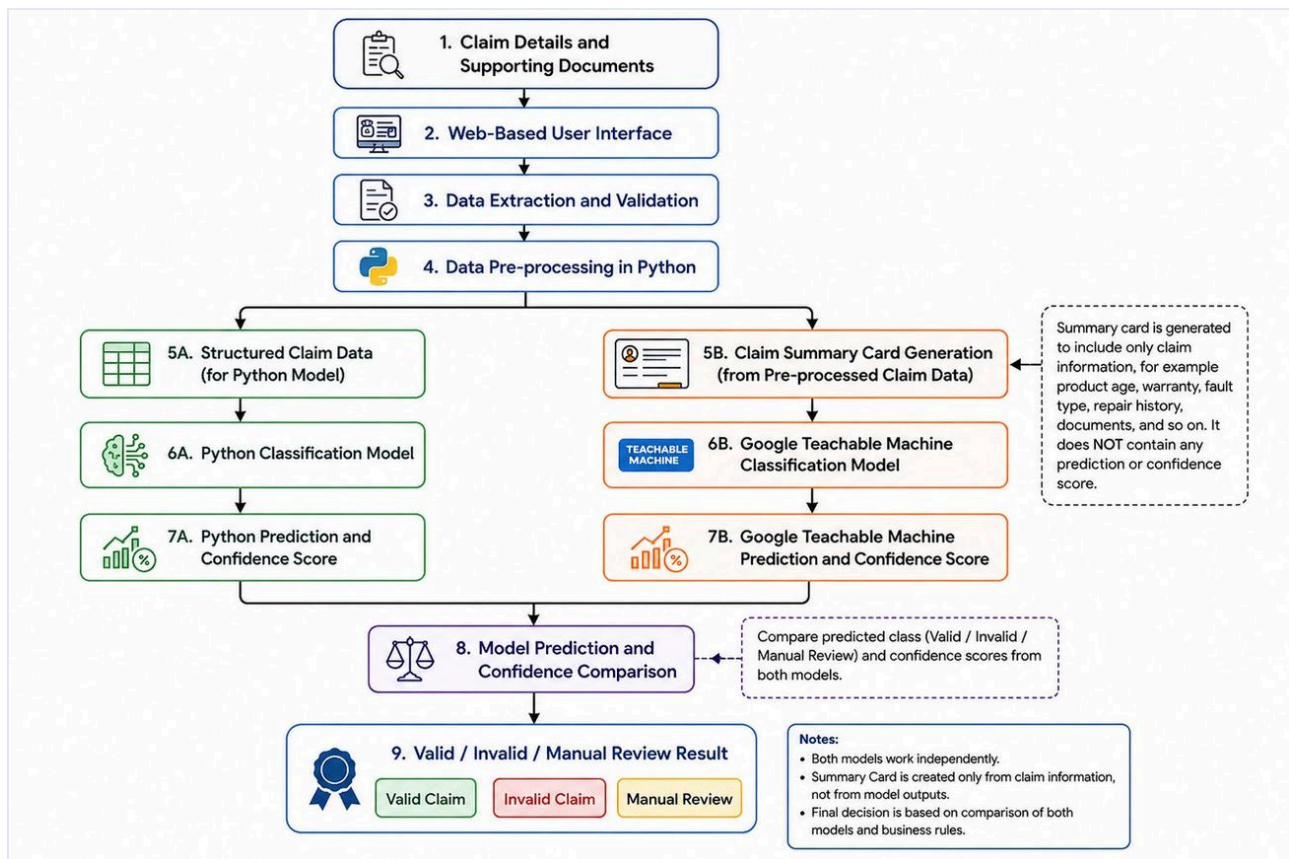
Repository layout

config/	rbac.json, decision_policy.json, config.py		
policies/	one warranty policy per category (JSON)		
data/	raw + split CSVs, summary_cards/{train,val,test}, mapping file, statistics, data dictionary		
dataset_generator/	generator_v2.py (records), build_cards_v2.py (cards)		
notebooks/	train_python_v2.py, evaluate_gtm.py		
model/	python_model/ (pipeline + model card), teachable_machine/ (export goes here)		
src/core/	vocab, features, python + TM runtimes, card renderer, consistency, decision table, pipeline		
src/rules/	policy store, rule engine, contradiction + duplicate detectors, validation		
src/security/	RBAC policy engine and Flask guards		
src/services/	claim lifecycle, documents, OCR glue, alerts, analytics, exports, PDF report, explanations		
src/api/	Flask blueprints	templates/, static/	UI
database/	seed.py	tests/	pytest suites (tests/e2e:
browser journeys)			
scripts/	subset_icons.py (icon-font subset)	docs/screenshots/	images produced by the e2e suite
reports/	comparison report, project report builder		
documentation/	project report, blog, evidence, installation, test cases, diagrams		

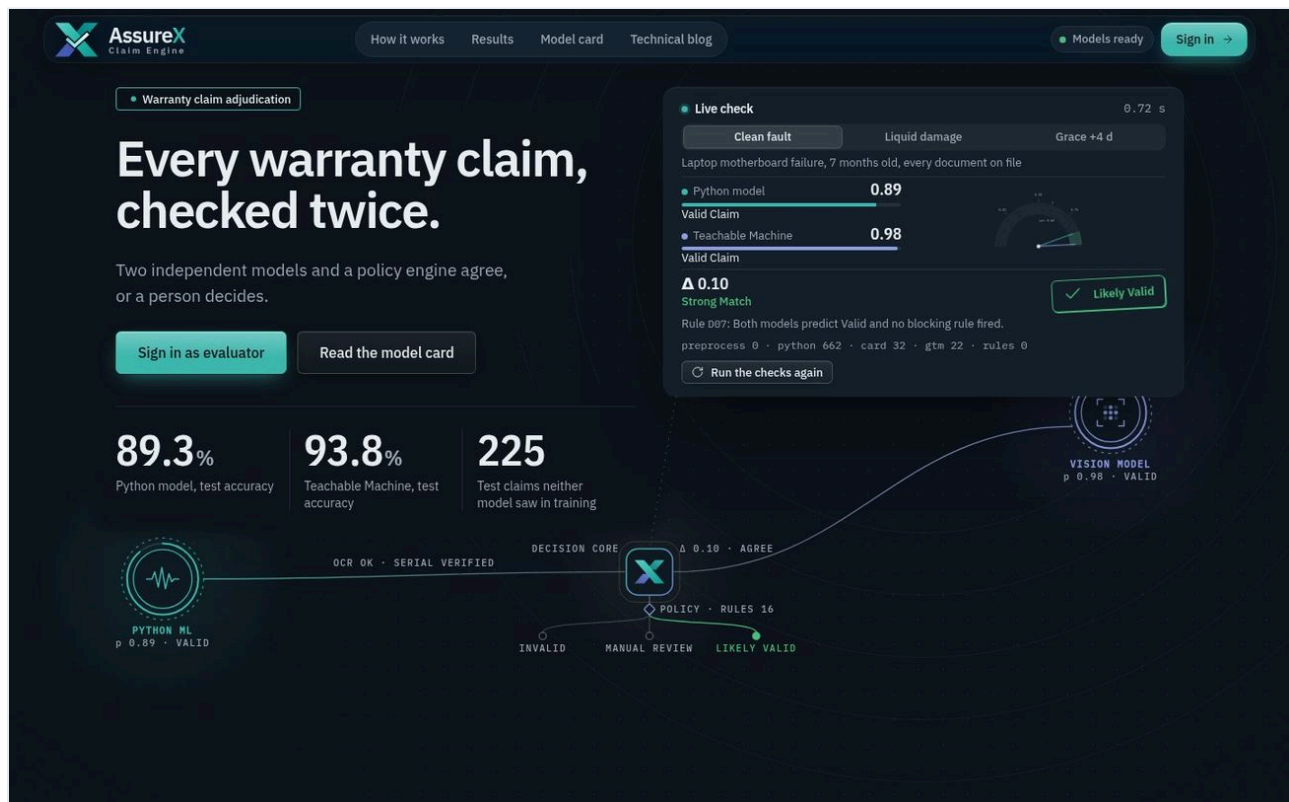
Known limitations

- The Teachable Machine model is trained in the browser and must be retrained by hand whenever the card design or the policy limits shown on the card change.
- The dataset is synthetic (generated from the policy with 4% label noise). Real claims will need retraining and re-calibration.
- OCR of **image** receipts needs the Tesseract binary; without it, images are accepted and the user types the details. PDF receipts with a text layer are read without Tesseract.
- SQLite is the default database; use [DATABASE_URL](#) (e.g. PostgreSQL) and a shared rate-limit store ([RATELIMIT_STORAGE_URI](#)) for multi-worker production.
- The saved Python model needs scikit-learn 1.9.1, which needs Python 3.11 or newer. The PythonAnywhere free account used for the live demo runs Python 3.10 (scikit-learn $\leq 1.7.2$) with a 512 MB disk quota, so both models can show as *Unavailable* there and claims then go to manual review, as designed. The full pipeline runs locally with the pinned requirements (Python 3.11, see [documentation/INSTALLATION.md](#)).

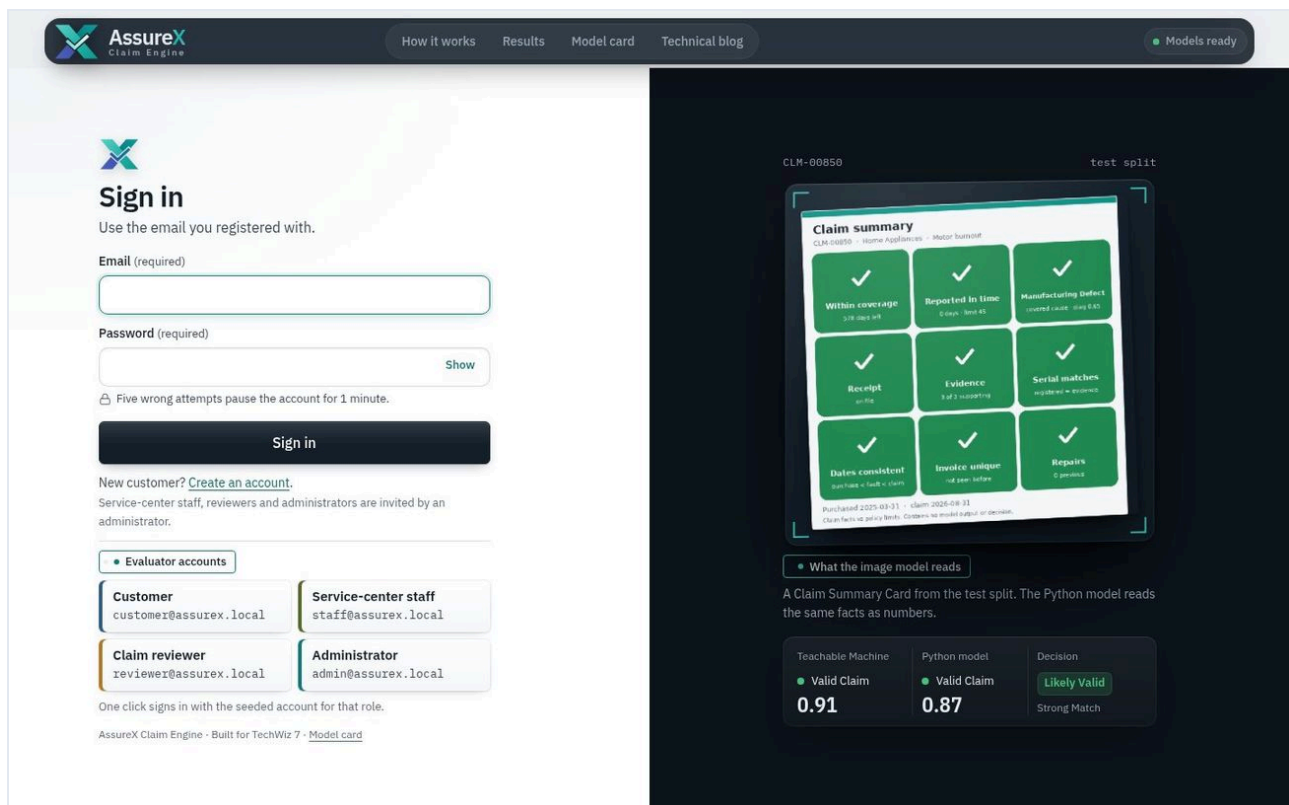
Screens of the working application



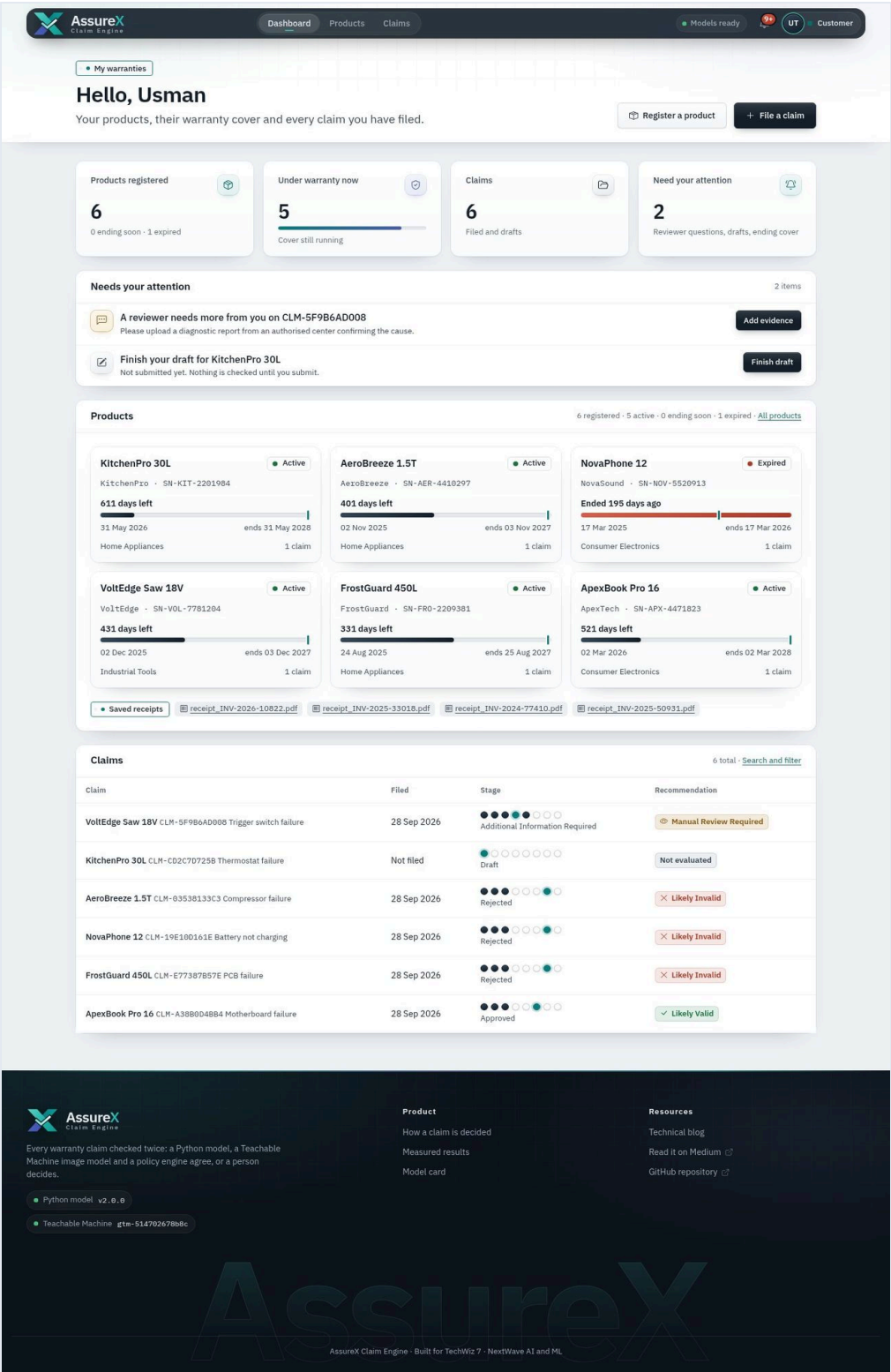
00_system_architecture.jpg — System architecture



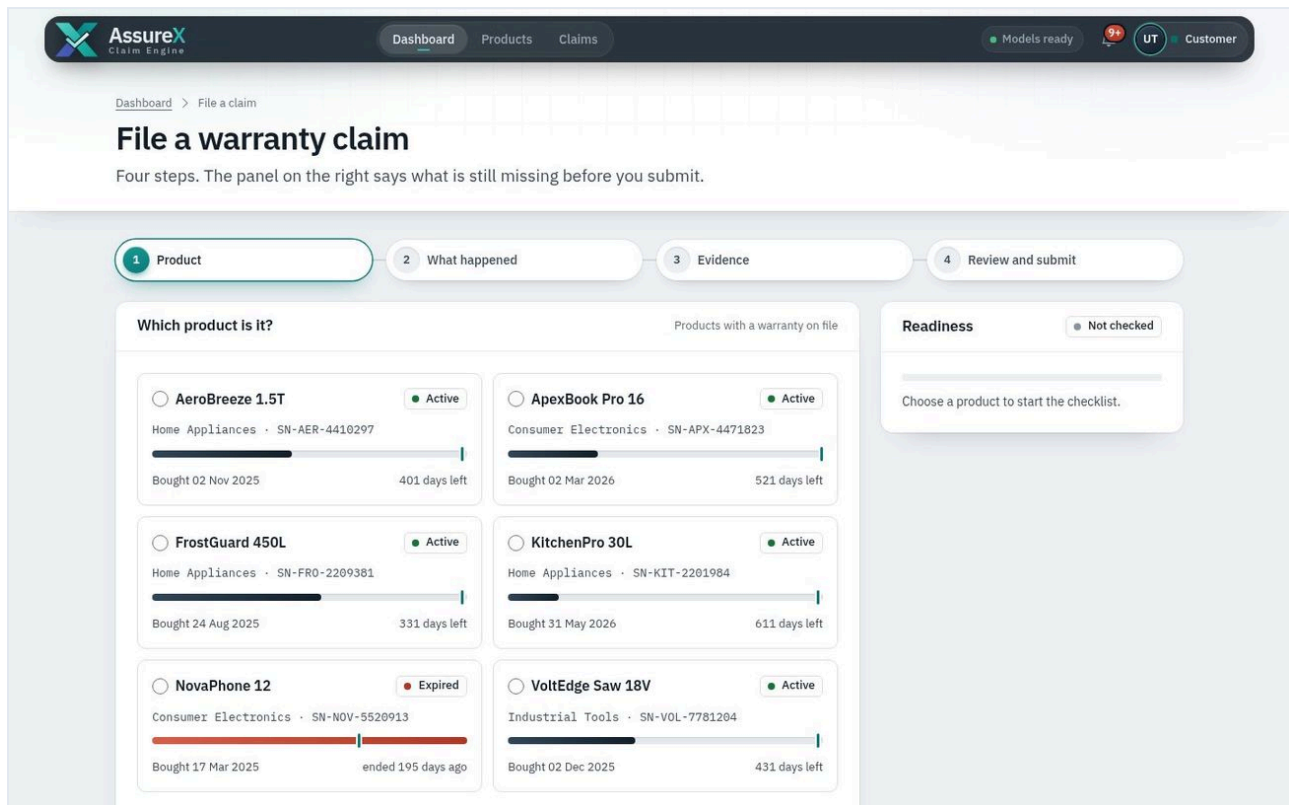
01_landing.jpg — Landing page: headline, measured results, the live check deciding a real sample claim, and the dual-engine background (Python ML and vision model feeding the decision core, policy engine and outcomes)



02_login.jpg — Sign-in beside a scanned test-split Claim Summary Card, with one-click evaluator accounts



03_customer_dashboard.jpg — Customer dashboard: at-a-glance KPIs, pending actions, products and claims



Overview

Claims

Review queue

Analytics

Policies

Access

Audit

Models ready

AK

Administrator

Claims / CLM-26CC70D39B

NovaPods Max

Speaker malfunction · filed for Hira Siddiqui

Manual Review

Medium risk

Track progress

Download report (PDF)

CLM-26CC70D39B NovaPods Max Consumer Electronics

Decided in 0.25 s

Python model

v2.0.0

Manual Review 0.94

Valid 0.03

Invalid 0.04

Manual Review 0.94

Teachable Machine

gtm-514702678b8c

Manual Review 0.55

Valid 0.11

Invalid 0.34

Manual Review 0.55

0.50

0.25

0.75

0.60

0.39

Uncertain Result

Lowest top-class confidence 0.55 is below the 0.60 minimum.

Manual review required

Why this decision

The models disagree, are not confident, or one is unavailable. Rule D04 matched (D01-D03 did not).

D01

D02

D03

D04

Claim filed inside the post-expiry grace period. Claim filed 4 days after expiry, inside the 7-day grace period.

15 warranty checks passed

Evidence still needed

None.

What moved the Python model

Each input replaced by its typical training value; bar = change in the probability of "Manual Review".

Days to warranty end: -4 (typical 221)

+0.88

Purchase price: 329 (typical 619.22)

+0.14

Product category: Consumer Electronics (typical Industrial Tools)

-0.04

What the image model saw

Claim summary

CLM-26CC70D39B · Consumer Electronics · Speaker malfunction

Grace period

Reported in time

Manufacturing Defect

Receipt on file

Evidence

Serial matches

Dates consistent

Invoice unique

Repairs

Purchased 2025-09-24 · Claim 2026-09-28

Claim facts vs policy limits. Contains no model output or decision.

The card holds claim facts only — no prediction, confidence or decision. Teal outlines mark the tiles that most drove the image model's answer.

Serial matches (registered = evidence)

-0.26

Grace period (4 of 7 grace days)

-0.02

Pipeline timings (ms): preprocess 5 · python 124 · card 27 · gtm 91 · rules 0 · total 250

Evidence

Each file is fingerprinted (SHA-256); receipts are read by OCR

Receipt

Warranty card

Damage photo

Serial photo

Repair report missing

Warranty card

warranty_card.png · 10.3 KB · 4bc65a02771a · 28 Sep 2026

Open

Damage / fault photo

damage_photo.png · 10.7 KB · 6ba8fc72e06f · 28 Sep 2026

Open

Serial-number photo

serial_photo.png · 12.4 KB · 3a4c4e45e07b · 28 Sep 2026

Open

Reviewer decision

Unassigned

Assign reviewer

Claim details

Product

NovaPods Max

PRD-0BEDBF91BC

Category

Consumer Electronics

Serial

SN-NOV-8830154

05_claim_manual_review_d04.jpg — Claim page, *Manual Review Required* (rule D04): Δ 0.39, *Uncertain Result* because the image model's top confidence is below 0.60; the decision trace, the Claim Summary Card the image model saw and what moved each model (top of the page)

AssureX Claim Engine — Project Documentation · Group NN_DevStorm

Page 57 of 106

Dashboard
Products
Claims

Models ready
99
UT
Customer

Claims / CLM-E77387857E

FrostGuard 450L

PCB failure · filed for Usman Tariq by Sana Malik

Rejected
High risk

Track progress
Download report (PDF)

CLM-E77387857E

FrostGuard 450L

Home Appliances

Decided in 0.24 s

Python model

v2.0.0

Invalid Claim 0.86

Valid 0.04

Invalid 0.86

Manual Review 0.10

Teachable Machine

gtm-514702678b8c

Invalid Claim 0.99

Valid 0.00

Invalid 0.99

Manual Review 0.00

0.50

0.25

0.75

min 0.60

0

1

Δ 0.13

Acceptable Match

Both predict Invalid Claim; confidence difference 0.13.

Likely invalid

Why this decision

A policy hard-fail rule was triggered. Rule D01 matched.

D01

✗ Damage cause is excluded by the policy and confirmed by diagnosis. 'Water Ingress' is excluded for Home Appliances and the diagnosis confirms it ($0.81 \geq 0.55$).

► 15 warranty checks passed

Evidence still needed
None.

What moved the Python model
Each input replaced by its typical training value; bar = change in the probability of "Invalid Claim".

Damage cause: Water Ingress (typical Manufacturing Defect)

+0.81

Diagnostic confidence: 0.81 (typical 0.58)

+0.07

Purchase price: 1250 (typical 619.22)

-0.06

What the image model saw

Claim summary

CLM-E77387857E · Home Appliances · PCB failure

Within coverage 30x days aft	Reported in time 6 days · limit 6x	Water ingress excluded cause · diag 0.81
Receipt on file	Evidence 3 of 3 supporting	Serial matches registered = evidence
Dates consistent prior date < fault < claim	Invoice unique not seen before	Repairs 0 previous

Purchased 2025-08-24 · claim 2026-09-28
Claim facts vs policy limits. Contains no model output or decision.

The card holds claim facts only — no prediction, confidence or decision. Teal outlines mark the tiles that most drove the image model's answer.

Water Ingress (excluded cause · diag 0.81)

-0.99

Dates consistent (purchase < fault < claim)

-0.96

Invoice unique (not seen before)

-0.91

Pipeline timings (ms): preprocess 5 · python 125 · card 26 · gtm 83 · rules 0 · total 242

Evidence

Each file is fingerprinted (SHA-256); receipts are read by OCR

Receipt

Warranty card

Damage photo

Serial photo

Repair report

Warranty card

warranty_card.png · 10.0 KB · 932a6001ffe0 · 28 Sep 2026

Open

► Extracted details — OCR not available for this file

Damage / fault photo

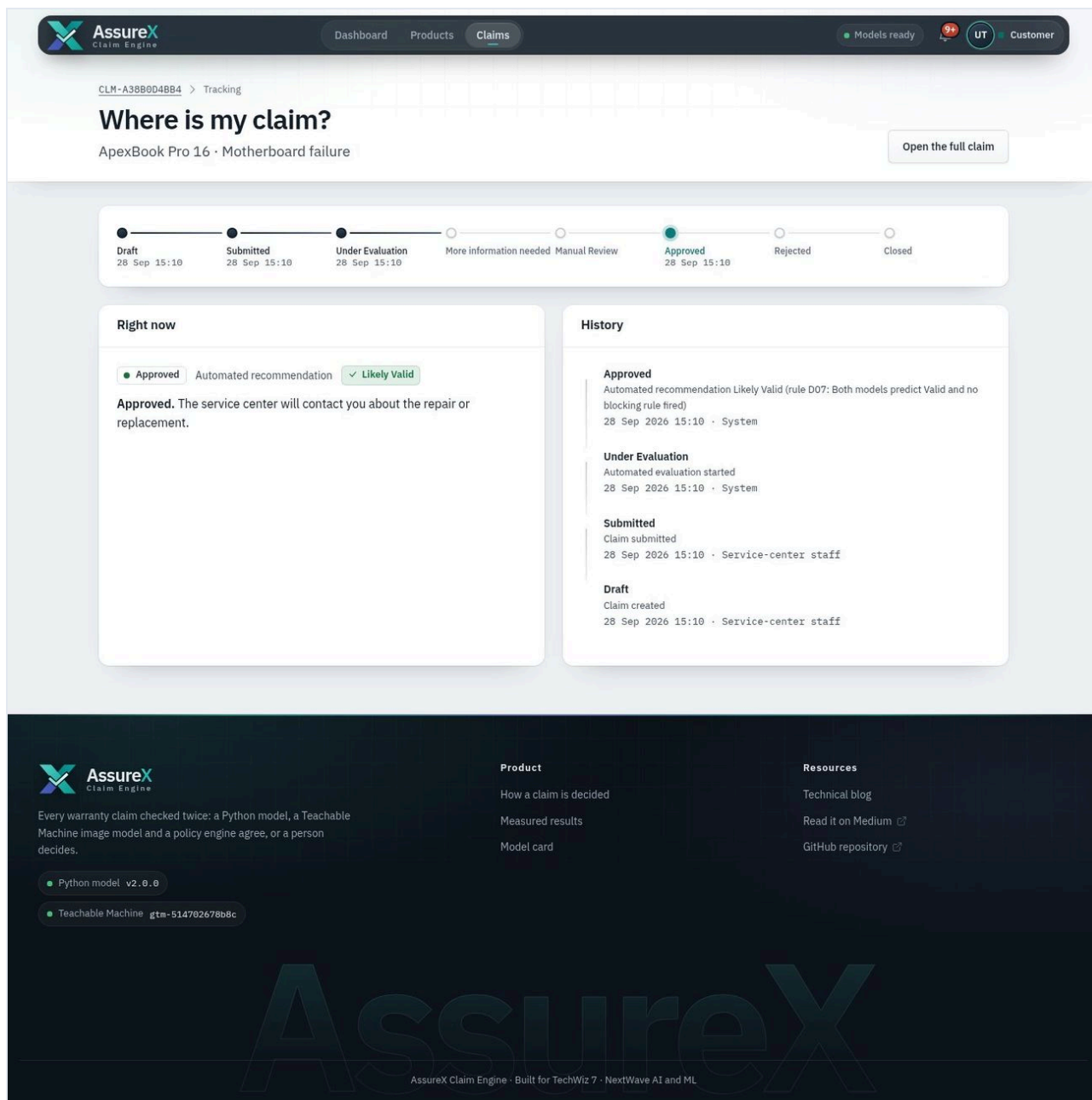
damage_photo.png · 10.4 KB · 624e02016c1b · 28 Sep 2026

Open

Claim details

Product	FrostGuard 450L PRD-1F735B2469
Category	Home Appliances
Serial	SN-FRO-2209381
Purchased	24 Aug 2025 · National Appliance Depot
Warranty	Active until 25 Aug 2027
Fault started	22 Sep 2026

06_claim_likely_invalid_d01.jpg — Claim page, *Likely Invalid* (rule D01): rejected by a hard-fail warranty rule (top of the page)



07_claim_tracking.jpg — Eight-stage claim tracking

Dashboard
Products
Claims

Models ready
9+
UT
Customer

Products and warranties

Every registered product with how much of its warranty is left.

+ Register a product

Search

Category

All categories

Warranty

All
Active
Expiring soon
Expired

Search

6 products.

KitchenPro 30L
Active

KitchenPro · SN-KIT-2201984

611 days left

31 May 2026 ends 31 May 2028

Home Appliances 1 claim

AeroBreeze 1.5T
Active

AeroBreeze · SN-AER-4410297

401 days left

02 Nov 2025 ends 03 Nov 2027

Home Appliances 1 claim

NovaPhone 12
Expired

NovaSound · SN-NOV-5520913

Ended 195 days ago

17 Mar 2025 ends 17 Mar 2026

Consumer Electronics 1 claim

VoltEdge Saw 18V
Active

VoltEdge · SN-VOL-7781204

431 days left

02 Dec 2025 ends 03 Dec 2027

Industrial Tools 1 claim

FrostGuard 450L
Active

FrostGuard · SN-FRO-2209381

331 days left

24 Aug 2025 ends 25 Aug 2027

Home Appliances 1 claim

ApexBook Pro 16
Active

ApexTech · SN-APX-4471823

521 days left

02 Mar 2026 ends 02 Mar 2028

Consumer Electronics 1 claim

Every warranty claim checked twice: a Python model, a Teachable Machine image model and a policy engine agree, or a person decides.

Python model v2.0.0
Teachable Machine gtm-514702678b8c

Product

How a claim is decided
Measured results
Model card

Resources

Technical blog
Read it on Medium
GitHub repository

AssureX

AssureX Claim Engine · Built for TechWiz 7 · NextWave AI and ML

08_products.jpg — Registered products with warranty status

AssureX Claim Engine — Project Documentation · Group NN_DevStorm

Page 60 of 106

Dashboard
Products
Claims

Models ready
98
UT
Customer

Products > PRD-4222DD1221

ApexBook Pro 16

Active
Consumer Electronics
ApexTech - ABP-16
SN-APX-4471823

File a claim

Warranty

ApexTech manufacturer warranty

521

days of cover left

02 Mar 2028

cover ends

24 months

standard term

Started 02 Mar 2026

Ends 02 Mar 2028 - then 7 days grace

Service center

Metro Authorised Service Center

Covered faults

Screen flickering, Motherboard failure, Battery not charging, Speaker malfunction, Touch panel unresponsive, Wi-Fi/Bluetooth failure

Not covered

Liquid ingress or immersion; Screen shattering from drops; Power surge / overvoltage damage; Cosmetic casing wear; Commercial rental use

Report faults within

30 days of noticing them

Repairs

Repairs must be carried out by an authorised service center using genuine parts.

Purchase

Product ID	PRD-4222DD1221
Bought	02 Mar 2026
Retailer	City Electronics Mall
Price	1,899.00
Invoice	INV-2025-48213
Registered	28 Sep 2026

Claims

1 on this product

Claim	Stage	Recommendation	Filed
Motherboard failure CLM-A3880D48B4	Approved	Likely Valid	28 Sep 2026

Product documents

Every claim on this product uses them

Purchase receipt / invoice
receipt_INV-2025-48213.pdf · 38e492957611 · 28 Sep 2026 · text read

Open
Replace
Remove

Document type

Purchase receipt / invoice

File

Choose File | No file chosen

PDF, JPG or PNG, at most 10 MB.

Upload document

Repair history

No repairs on record

Every warranty claim checked twice: a Python model, a Teachable Machine image model and a policy engine agree, or a person decides.

Python model v2.0.0
Teachable Machine gtm-514702678b8c

Product

How a claim is decided
Measured results
Model card

Resources

Technical blog
Read it on Medium
GitHub repository

AssureX

AssureX Claim Engine · Built for TechWiz 7 · NextWave AI and ML

09_product_warranty.jpg — Product and warranty page: coverage, documents and repair history



AssureX

Claim Engine

Claims

Review queue

Models ready

BA

Claim reviewer

Review queue

Claims the automated checks could not settle: low confidence, model disagreement, missing evidence, duplicates, contradictions or a policy review rule.

Unassigned 5

Assigned to me 0

Waiting on the customer 1

Decided

Category

Risk

Model agreement

All categories

Any risk

Any

5 claims

Highest risk first, then oldest

Claim	Why it is here	Python	Teachable Machine	Agreement	Waiting
ApexBook Pro 16 CLM-BC91073EBB Kamran Ali High risk	D02 Contradiction or duplicate indicator found ; possible duplicate	Manual Review 0.85	Manual Review 0.96	Acceptable Match	0 d Take
VividTab 11 CLM-FD180BC3BC Hira Siddiqui High risk	D02 Contradiction or duplicate indicator found 1 contradiction	Manual Review 0.55	Manual Review 0.85	Uncertain Result	0 d Take
CleanCycle 8kg CLM-15580B2CE4 Hira Siddiqui Medium risk	D03 A mandatory document is missing Supporting evidence; missing receipt	Manual Review 0.96	Manual Review 0.98	Strong Match	0 d Take
NovaPods Max CLM-26CC70D39B Hira Siddiqui Medium risk	D04 The models disagree, are not confident, or one is unavailable Grace-period claim	Manual Review 0.94	Manual Review 0.55	Uncertain Result	0 d Take
KitchenPro Oven 45L CLM-FE40F6C0B8 Kamran Ali Medium risk	D04 The models disagree, are not confident, or one is unavailable	Invalid Claim 0.91	Valid Claim 0.97	Model Disagreement	0 d Take



AssureX

Claim Engine

Every warranty claim checked twice: a Python model, a Teachable Machine image model and a policy engine agree, or a person decides.

Python model v2.0.0

Teachable Machine gtm-514702678b8c

Product

How a claim is decided

Measured results

Model card

Resources

Technical blog

Read it on Medium

GitHub repository

AssureX

AssureX Claim Engine - Built for TechWiz 7 - NextWave AI and ML

AssureX

Claim Engine

Claims

Review queue

Models ready

BA

Claim reviewer

Review queue

CLM-26CC70D39B

NovaPods Max • Speaker malfunction

Manual Review

Medium risk

Unassigned

Filed 28 Sep 2026 • waiting 0 d

Shortcuts

Claim page

Next claim

Evidence

4 files

Warranty card

NovaSound NovaPods Max

Synthetic demo image

Warranty card

warranty_card.png • 4bc65a92771a

Open

Damage / fault photo

Open

CLM-26CC70D39B NovaPods Max

Consumer Electronics

Decided in 0.25 s

0.50

0.25

0.75

0.60

0.39

Uncertain Result

Lowest top-class confidence 0.55 is below the 0.60 minimum.

Python model

Teachable Machine

v2.0.0

gtn-514702678b8c

Manual Review 0.94

Manual Review 0.55

Warranty

Ended 4 days before today (inside the 7-day grace period)

24 Sep 2025

ends 24 Sep 2026 + 7 d grace

Fault started

22 Sep 2026

Reported

28 Sep 2026 (6 d later; limit 30)

Cause given

Manufacturing Defect

Diagnosis

0.75 • Service-center technician

Customer's account

Customer

Hira Siddiqui

Other claims

2

Approve

Reject

Request information

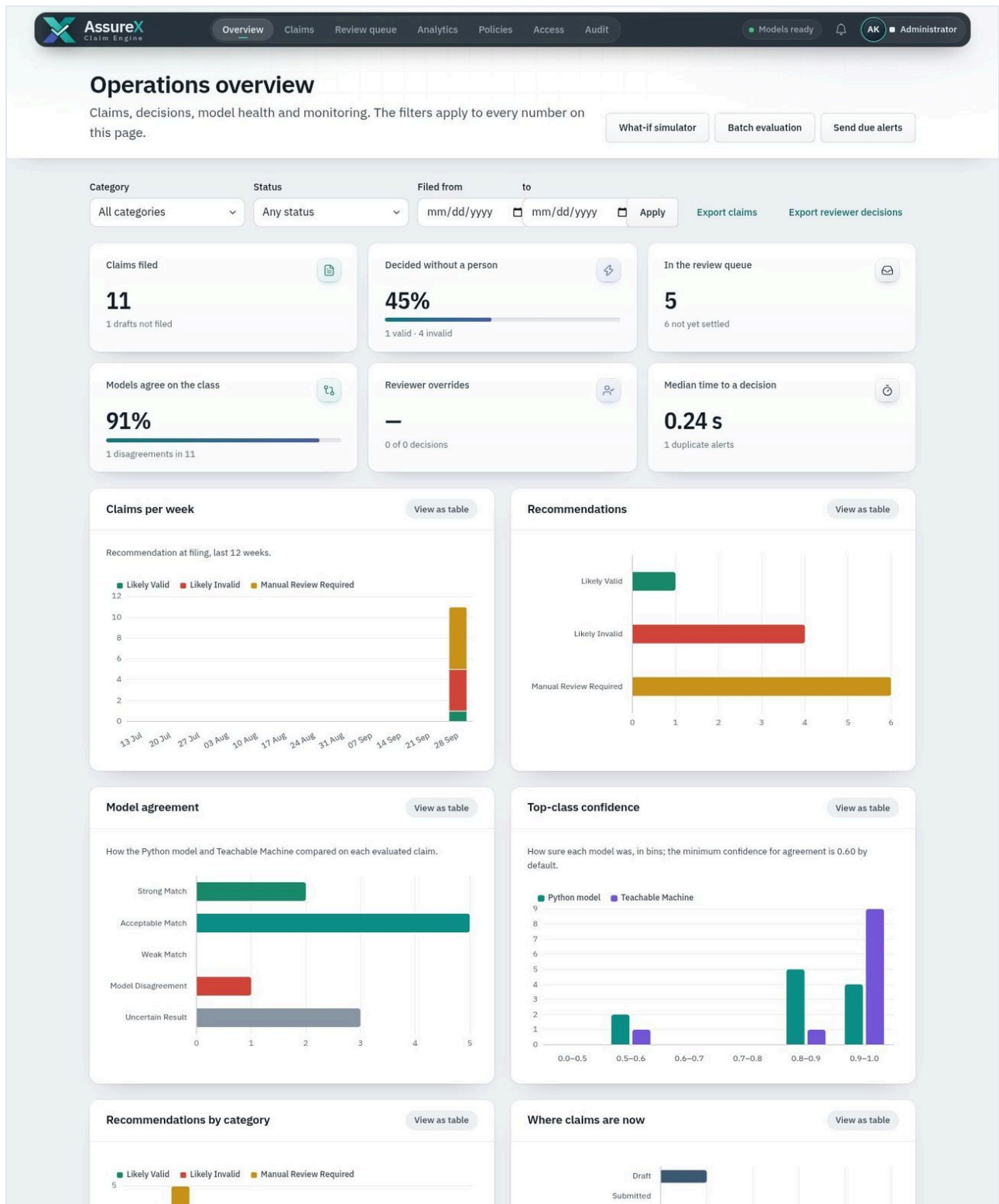
Comment (shown to the customer, kept in the audit trail)

Recommendation: Manual Review Required

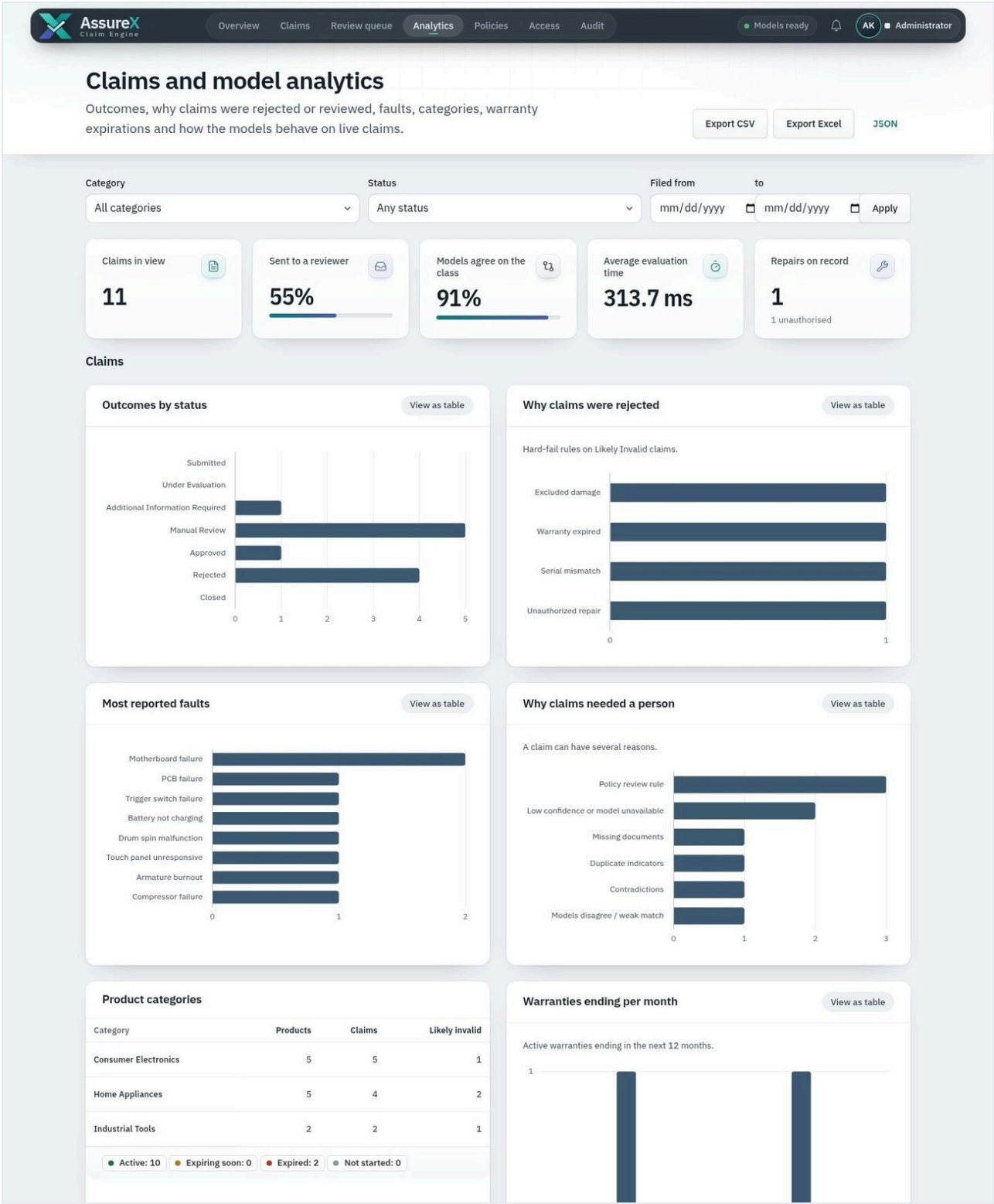
Record decision

Take this claim

11_reviewer_decision.jpg — Reviewer workbench: evidence, the verdict with both models' scores, warranty facts, and the decision bar with keyboard shortcuts



12_admin_overview.jpg — Operations overview: filters, KPI cards, claims per week, recommendations, model agreement and confidence charts (top of the page)



13_analytics_model_evidence.jpg — Analytics and model evidence (top of the page)

AssureX

Claim Engine

Overview

Claims

Review queue

Analytics

Policies

Access

Audit

Models ready

AK

Administrator

Warranty policies and decision settings

Changes apply to new evaluations immediately; recorded results never change. Every save is versioned in the audit trail.

Try thresholds in the what-if simulator

Consumer Electronics

Home Appliances

Industrial Tools

Standard Consumer Electronics Warranty Policy

policies/consumer_electronics.json

Standard coverage (months)

12

Grace period after expiry (days)

on the card

7

Report faults within (days)

on the card

30

Repeat repairs before review

on the card

2

Diagnosis needed to confirm an exclusion

on the card

0.55

An excluded cause with a diagnosis at or above this is a hard fail; below it the claim goes to a reviewer.

Excluded causes

on the card

☐ Manufacturing Defect

☐ Normal Wear and Tear

☒ Electrical Surge

☐ Mechanical Stress

☒ Accidental Drop

☒ Water Ingress

☐ Unknown / Not Sure

Rules and severity

Hard fail: Likely Invalid - Manual review: a reviewer - Warning: shown, never blocks

Rule	Severity
Claim filed after the warranty and its grace period ended WARRANTY_EXPIRED	Hard fail
Claim filed inside the post-expiry grace period GRACE_PERIOD	Manual review
Warranty ends within 14 days of the claim WARRANTY_ENDING_SOON	Warning
Product was repaired by an unauthorised service center UNAUTHORIZED_REPAIR	Hard fail
Damage cause is excluded by the policy and confirmed by diagnosis EXCLUDED_DAMAGE	Hard fail
Damage cause is excluded but the diagnosis is not conclusive EXCLUDED_DAMAGE_UNCONFIRMED	Manual review
Serial number differs from the receipt and serial photo SERIAL_MISMATCH	Hard fail
Serial number mismatch that the evidence cannot confirm SERIAL_UNVERIFIED	Manual review
Fault reported after the policy's reporting deadline LATE_REPORTING	Hard fail
Fault reported in the last week before the deadline NEAR_REPORTING_DEADLINE	Warning
Fault is not in the policy's list of covered faults UNCOVERED_FAULT	Manual review
Cause unknown and diagnostic confidence below 0.50 UNKNOWN_CAUSE	Manual review
Product has reached the repeat-repair review threshold REPEAT_REPAIRS	Manual review
Product was already replaced once under warranty PRIOR_REPLACEMENT	Manual review
Two or more supporting documents are missing INCOMPLETE_EVIDENCE	Manual review
One supporting document is missing SUPPORTING_DOCUMENT_MISSING	Warning

Changes

None yet

Edit a value to see the change here before saving.

Save Consumer Electronics policy

Discard

Version history

Consumer Electronics

No saved changes yet: this is the policy as shipped.

Fixed by the policy file

Covered faults

Screen flickering, Motherboard failure, Battery not charging, Speaker malfunction, Touch panel unresponsive, Wi-Fi/Bluetooth failure

Mandatory

receipt

Supporting

warranty card, serial photo, damage photo

14_policy_editor.jpg — Warranty policy and rule-severity editor, decision settings, version history (top of the page)

AssureX Claim Engine — Project Documentation · Group NN_DevStorm

Page 66 of 106

[Overview](#)
[Claims](#)
[Review queue](#)
[Analytics](#)
[Policies](#)
[Access](#)
[Audit](#)

Models ready

AK Administrator

Overview > Models

Models

Each model is versioned by the hash of its file, and every stored prediction records the version that made it. Installing a new model never changes past results.

Public model card

Python classification model

Running

Version	v2.0.0+514e2fb68286
Algorithm	Hist gradient boosting, sigmoid-calibrated (5-fold)
Trained with	scikit-learn 1.9.1 - seed 42
Inputs	20 features: 8 numeric, 10 yes/no, 2 categories
Test accuracy	89.3% (target: 85%)
Macro F1 · ROC-AUC	89.2% · 0.9475
Calibration (ECE)	0.0818

Retrain with python notebooks/train_python_v2.py.

Google Teachable Machine

Running

Version	gtm=514702678b8c
File	model_unquant.tflite · floating point · input 224x224
Test accuracy	93.8% macro F1 93.8%
Validation accuracy	97.3%
Agrees with Python	92.9% of test claims

Re-run the evaluation on validation and test cards

Train and install the Teachable Machine model

- Download the training cards: 2100 images in three class folders, two variations per training claim. Validation and test cards are left out.

Download training cards (.zip)
- Open teachablemachine.withgoogle.com, then Image Project, Standard image model.
- Name the classes **Valid Claim**, **Invalid Claim** and **Manual Review** and upload each folder into its class.
- Train. 50 epochs, batch 16 and learning rate 0.001 are a good start.
- Export Model, TensorFlow Lite, Floating point, Download, then unzip.
- Upload model_unquant.tflite and labels.txt here and run the evaluation.

Model file (.tflite)

Choose File No file chosen

labels.txt

Choose File No file chosen

Checked before it replaces the current model: TensorFlow Lite header, three labels matching the classes, and a test run. The previous model is archived.

Install the model

Sample Claim Summary Cards

600 × 600 · claim facts only · one renderer for training and live claims

Claim summary

CLM-00004_v1.jpg

Valid Claim

Claim summary

CLM-00006_v1.jpg

Invalid Claim

Claim summary

CLM-00003_v1.jpg

Manual Review

AssureX Claim Engine

Every warranty claim checked twice: a Python model, a Teachable Machine image model and a policy engine agree, or a person decides.

Python model v2.0.0

Teachable Machine gtm=514702678b8c

Product

How a claim is decided
Measured results
Model card

Resources

Technical blog
Read it on Medium
GitHub repository

AssureX

AssureX Claim Engine · Built for TechWiz 7 · NextWave AI and ML

15_models_teachable_machine.jpg — Model status, Teachable Machine installer and evaluation, sample cards

AssureX Claim Engine — Project Documentation · Group NN_DevStorm

Page 67 of 106

[Overview](#)
[Claims](#)
[Review queue](#)
[Analytics](#)
[Policies](#)
[Access](#)
[Audit](#)

Models ready
AK Administrator

Access control

Who can sign in, what each role may do and which records it reaches. A role change signs the person out everywhere so the new limits apply at once.

Customer

3

People with this role

Service-center staff

1

People with this role

Claim reviewer

2

People with this role

Administrator

1

People with this role

People

7 shown

Name or email

Role

Status

Filter

Person	Role	Status	
Areeba Khan (you) admin@assurex.local USR - DA76B1D522	Administrator	Active	Sign out
Bilal Ahmed reviewer@assurex.local USR - 9FDF3E72BE	Claim reviewer	Active	Sign out Disable
Nadia Qureshi reviewer2@assurex.local USR - 5666027353	Claim reviewer	Active	Sign out Disable
Hira Siddiqui hira@example.com USR - CDF1DFA271	Customer	Active	Sign out Disable
Kamran Ali kamran@example.com USR - 47755FAED6	Customer	Active	Sign out Disable
Usman Tariq customer@assurex.local USR - C433C0AD90	Customer	Active	Sign out Disable
Sana Malik staff@assurex.local USR - 78D2F53CB8 · Metro Au	Service-center staff	Active	Sign out Disable

What each role can do

Scope = which records a permission reaches · config/rbac.json

Permission	Customer	Service-center staff	Claim reviewer	Administrator
admin.dashboard	—	—	—	all
alert.dispatch	—	—	—	all
analytics.read	—	—	—	all
audit.read	—	—	—	all
claim.create	own	service center	—	—
claim.read	own	service center	assigned or queue	all
claim.submit	own	service center	—	—
claim.update_draft	own	service center	—	—
document.correct_ocr	own	service center	—	—
document.delete	own	—	—	—
document.read	own	service center	assigned or queue	all

Invite someone

Full name (required)

Work email (required)

Role

Customer

They get a one-time password shown once to you, and must set their own at first sign-in. Public sign-up only creates customer accounts.

Create the account

Recent blocked attempts

No blocked attempts yet.

Service centers

Harbor Tech Repairs

Karachi · +92 21 3555 0200 · SC-D5A3C34C92

Metro Authorised Service Center

Lahore · +92 42 3555 0100 · SC-61C50C31D1

Center name

City

Phone

Add the center

16_access_control.jpg — Access control: people, roles, invitations, service centers, blocked attempts (top of the page)

Audit trail

Every sign-in, upload, OCR correction, prediction, status change, reviewer decision, configuration change and blocked attempt. Nothing here can be edited or deleted.

Export CSV

Export Excel

Action

Role

Actor

Record or details

From

to

All actions

Any role

USR- or email

CLM-, a file hash, a rule

mm/dd/yyyy

mm/dd/yyyy

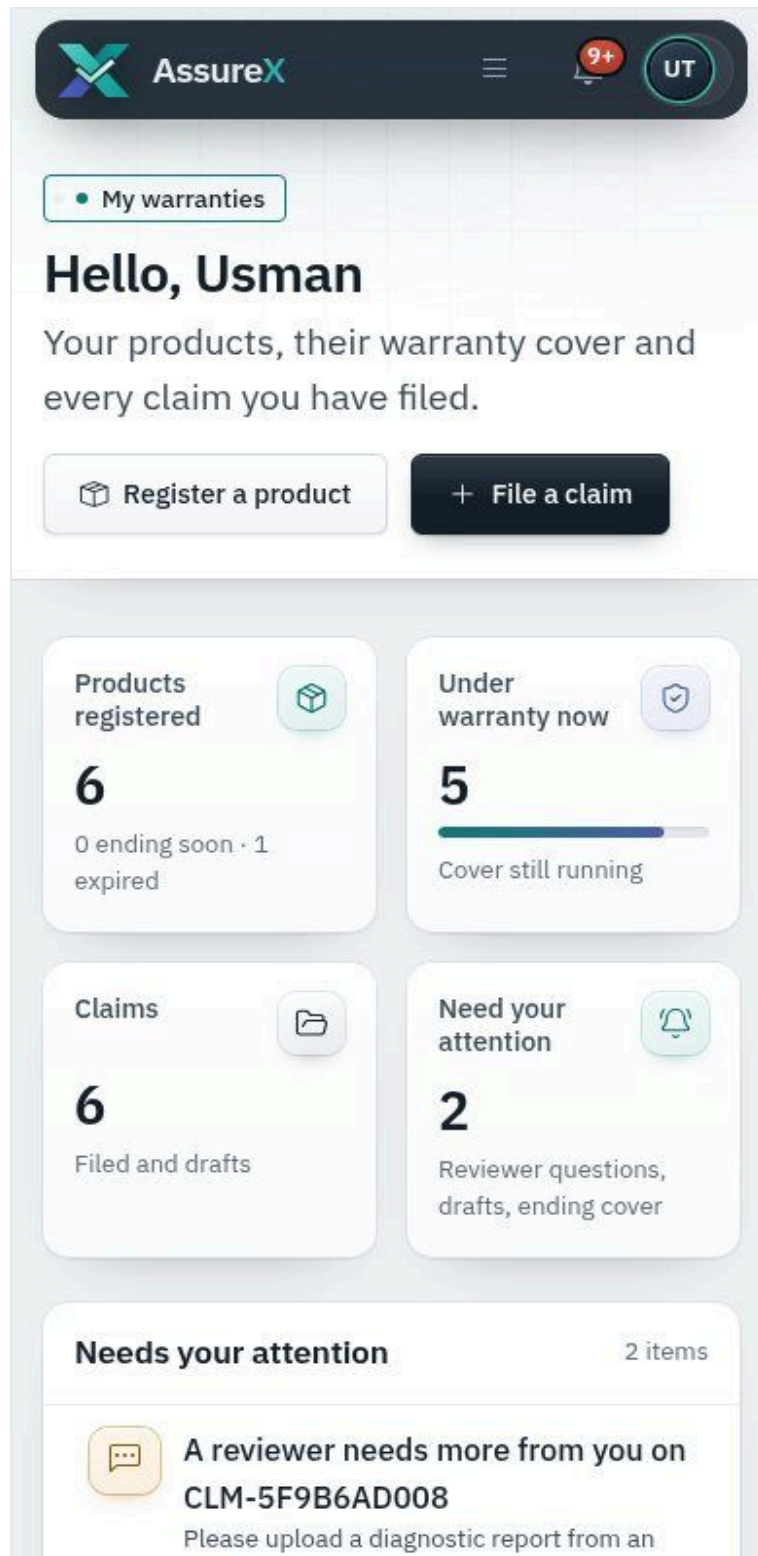
Search

72 events

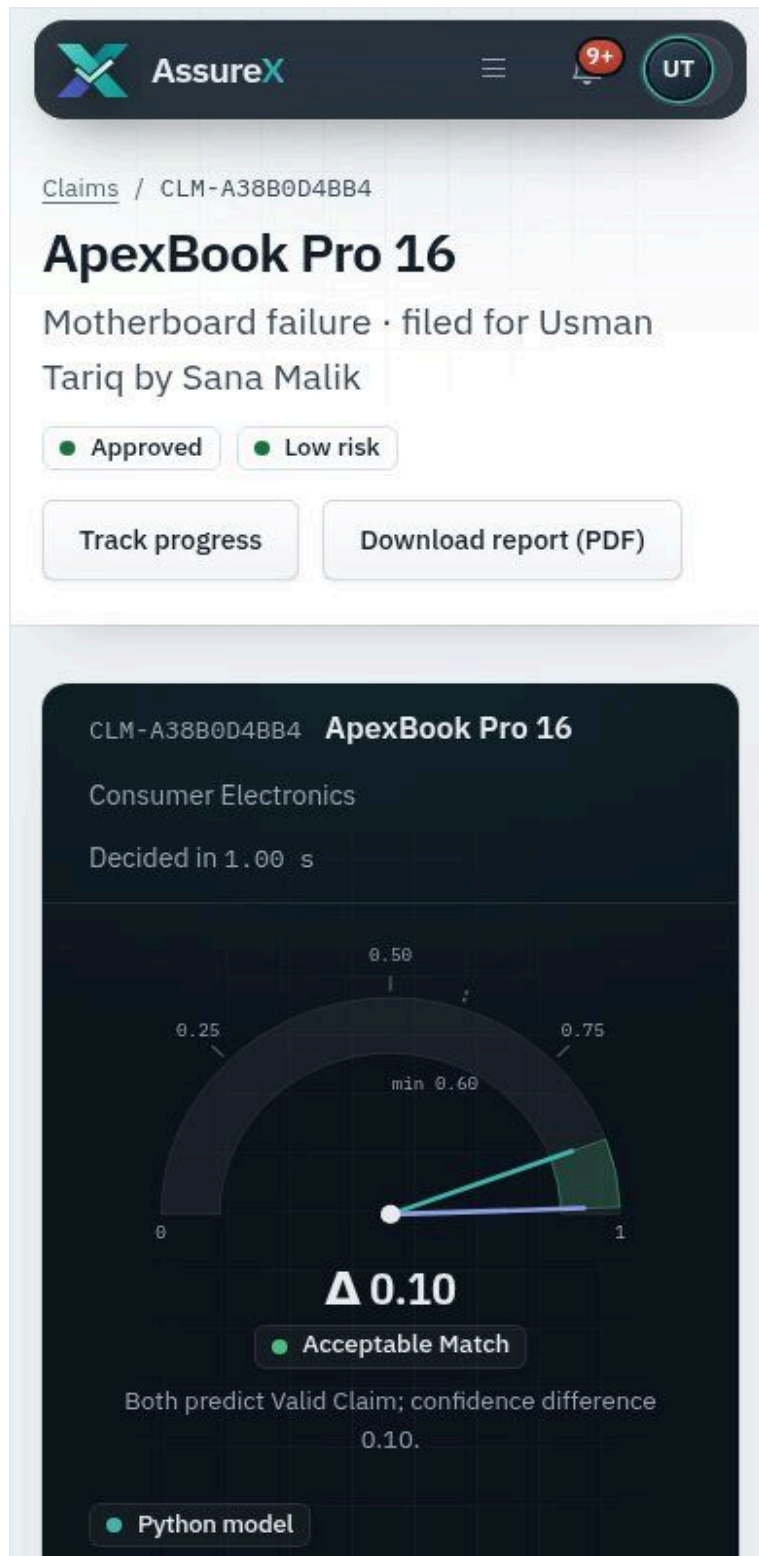
Newest first • times in UTC

Time	Action	Actor	Record	Details	IP	
28 Sep 15:11:55	LOGIN_SUCCESS	Bilal Ahmed Claim reviewer	User USR-9FDF3E72BE		127.0.0.1	
28 Sep 15:11:35	LOGIN_SUCCESS	Areeba Khan Administrator	User USR-DA76B1D522		127.0.0.1	
28 Sep 15:11:26	LOGIN_SUCCESS	Usman Tariq Customer	User USR-C433C0AD90		127.0.0.1	
28 Sep 15:10:08	REVIEWER_DECISION	Bilal Ahmed Claim reviewer	Claim CLM-5F9B6AD008	{ "from_status": "Manual Review", "to_status": "Additional..." }	127.0.0.1	
28 Sep 15:10:08	STATUS_CHANGE	Bilal Ahmed	Claim CLM-5F9B6AD008	{ "from": "Manual Review", "to": "Additional Information Required", ... }		
28 Sep 15:10:08	REVIEWER_ASSIGNED	Bilal Ahmed Claim reviewer	Claim CLM-5F9B6AD008	{ "reviewer": "USR-9FDF3E72BE", "previous": null }		
28 Sep 15:10:08	CLAIM_SUBMITTED	Kamran Ali Customer	Claim CLM-FE40F6C0B8	{ "first_submission": true }		
28 Sep 15:10:08	STATUS_CHANGE	System	Claim CLM-FE40F6C0B8	{ "from": "Under Evaluation", "to": "Manual Review", "note": ... }		
28 Sep 15:10:08	FINAL_DECISION	Kamran Ali Customer	Claim CLM-FE40F6C0B8	{ "decision": "Manual Review Required", "rule": "D04", ... }		
28 Sep 15:10:08	MODEL_PREDICTION	Kamran Ali Customer	Claim CLM-FE40F6C0B8	{ "python": { "predicted_class": "Invalid Claim", "top_confidence": ... } }		
28 Sep 15:10:08	STATUS_CHANGE	System	Claim CLM-FE40F6C0B8	{ "from": "Submitted", "to": "Under Evaluation", "note": "Automated..." }		
28 Sep 15:10:08	STATUS_CHANGE	Kamran Ali	Claim CLM-FE40F6C0B8	{ "from": "Draft", "to": "Submitted", "note": "Claim submitted" }		
28 Sep 15:10:08	CLAIM_SUBMITTED	Hira Siddiqui Customer	Claim CLM-26CC70D39B	{ "first_submission": true }		
28 Sep 15:10:08	STATUS_CHANGE	System	Claim CLM-26CC70D39B	{ "from": "Under Evaluation", "to": "Manual Review", "note": ... }		
28 Sep 15:10:08	FINAL_DECISION	Hira Siddiqui Customer	Claim CLM-26CC70D39B	{ "decision": "Manual Review Required", "rule": "D04", ... }		
28 Sep 15:10:08	MODEL_PREDICTION	Hira Siddiqui Customer	Claim CLM-26CC70D39B	{ "python": { "predicted_class": "Manual Review", "top_confidence": ... } }		
28 Sep 15:10:07	STATUS_CHANGE	System	Claim CLM-26CC70D39B	{ "from": "Submitted", "to": "Under Evaluation", "note": "Automated..." }		
28 Sep 15:10:07	STATUS_CHANGE	Hira Siddiqui	Claim CLM-26CC70D39B	{ "from": "Draft", "to": "Submitted", "note": "Claim submitted" }		
28 Sep 15:10:07	CLAIM_SUBMITTED	Sana Malik Service-center staff	Claim CLM-03538133C3	{ "first_submission": true }		
28 Sep 15:10:07	STATUS_CHANGE	System	Claim CLM-03538133C3	{ "from": "Under Evaluation", "to": "Rejected", "note": "Automated..." }		
28 Sep 15:10:07	FINAL_DECISION	Sana Malik Service-center staff	Claim CLM-03538133C3	{ "decision": "Likely Invalid", "rule": "D01", "policy_version": ... }		
		Sana Malik Service-center	Claim	{ "python": { "predicted_class": "Invalid		

17_audit_trail.jpg — Audit trail with filters and export (top of the page)



18_mobile_dashboard.jpg — Customer dashboard at 390 px



19_mobile_claim.jpg — Claim page at 390 px

Dashboard
Products
Claims

Models ready
99
UT
Customer

Claims / CLM-A3880D4BB4

ApexBook Pro 16

Motherboard failure - filed for Usman Tariq by Sana Malik

Approved
Low risk

Track progress
Download report (PDF)

CLM-A3880D4BB4

ApexBook Pro 16

Consumer Electronics

Decided in 1.00 s

Python model

v2.0.0

Valid Claim 0.89

Valid 0.89

Invalid 0.08

Manual Review 0.03

Teachable Machine

gtm-514702678b8c

Valid Claim 0.99

Valid 0.99

Invalid 0.01

Manual Review 0.00

0.50

0.25

0.75

min 0.60

0

1

Δ 0.10

Acceptable Match

Both predict Valid Claim; confidence difference 0.10.

Likely valid

Why this decision

Both models predict Valid and no blocking rule fired. Rule D07 matched (D01-D06 did not).

D01
D02
D03
D04
D05
D06
D07

► 16 warranty checks passed

Evidence still needed
None.

What moved the Python model
Each input replaced by its typical training value; bar = change in the probability of "Valid Claim".

Purchase price: 1899 (typical 619.22)

Days from fault to claim: 4 (typical 6)

Product category: Consumer Electronics (typical Industrial Tools)

+0.03

-0.02

+0.01

What the image model saw

Claim summary

Within coverage

Reported in time

Manufacturing Defect

Receipt on file

Evidence 1 of 3 supporting

Serial matches registered = evidence

Dates consistent

Invoice unique not seen before

Repairs 0 previous

Purchased 2026-03-02 · claim 2026-09-28
Claim facts vs policy limits. Contains no model output or decision.

The card holds claim facts only — no prediction, confidence or decision. Teal outlines mark the tiles that most drove the image model's answer.

Invoice unique (not seen before)

Serial matches (registered = evidence)

Manufacturing Defect (covered cause · diag 0.86)

-0.98

-0.98

-0.95

Pipeline timings (ms): preprocess 14 · python 823 · card 29 · gtm 128 · rules 1 · total 999

Evidence

Each file is fingerprinted (SHA-256); receipts are read by OCR

Receipt

Warranty card

Damage photo

Serial photo

Repair report missing

Warranty card

warranty_card.png · 10.3 KB · 1225b8f8e122 · 28 Sep 2026

Open

► Extracted details — OCR not available for this file

Damage / fault photo

damage_photo.png · 10.7 KB · 579662361931 · 28 Sep 2026

Open

Claim details

Product

ApexBook Pro 16

PRD-4222001221

Category

Consumer Electronics

Serial

SN-APX-4471823

Purchased

02 Mar 2026 · City Electronics Mall

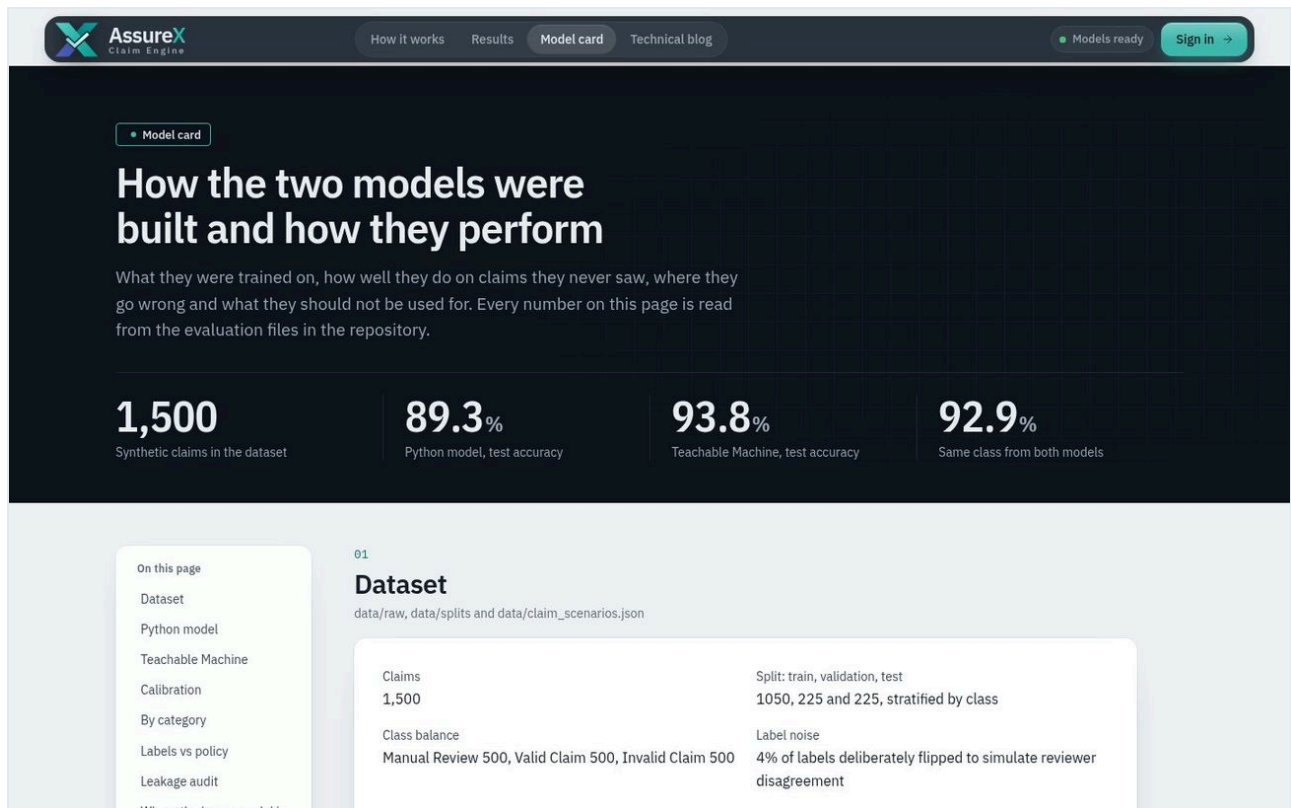
Warranty

Active until 02 Mar 2028

Fault started

24 Sep 2026

26_claim_likely_valid.jpg — Claim page, *Likely Valid* (rule D07): both models agree, no blocking rule fired (top of the page)



27_model_card.jpg — Model card: both models' measured results as a technical report

AssureX
Claim Engine

OverviewClaimsReview queueAnalyticsPoliciesAccessAudit

Models readyAKAdministrator

Overview > What-if

What-if simulator

Change when the two models count as agreeing and see the effect before it touches a single claim. Rule outcomes are replayed as stored; only the agreement check moves.

Decision policy 2026.09.26

On the 225 labelled test claims

Current thresholds

Measure	Current	Proposed	Change
Decided without a person	64.0%	64.0%	±0
Correct among those decisions	94.4%	94.4%	±0
Wrong automated decisions	8	8	±0
Sent to a reviewer	36.0%	36.0%	±0

Accuracy counts an automated "Likely Valid" as right when the true label is Valid Claim, and "Likely Invalid" when it is Invalid Claim. Claims sent to a reviewer are not counted as errors.

Where decisions move

0 of 225 test claims change

Rows: now · columns: with the proposed thresholds

	Valid	Invalid	Review
Valid	74	0	0
Invalid	0	70	0
Review	0	0	81

Live claims

0 of 11 evaluated claims would get a different recommendation

No live claim would get a different recommendation.

Thresholds

Minimum confidence 0.60

Below this, either model counts as unsure: Uncertain Result. Now 0.60.

Strong match: largest gap 0.10

Same class and confidences at most this far apart. Now 0.10.

Acceptable match: largest gap 0.25

Wider than this is a Weak Match and goes to a reviewer. Now 0.25.

Back to currentApply thresholds

Applying saves a new decision-policy version with the change and its simulated effect in the audit trail.

AssureX
Claim Engine

Every warranty claim checked twice: a Python model, a Teachable Machine image model and a policy engine agree, or a person decides.

Python model v2.8.0

Teachable Machine gtm-514702678b8c

Product

How a claim is decided

Measured results

Model card

Resources

Technical blog

Read it on Medium

GitHub repository

AssureX

AssureX Claim Engine · Built for TechWiz 7 · NextWave AI and ML

28_what_if_simulator.jpg — What-if simulator: try consistency thresholds against the stored decisions before applying them

Overview

Claims

Review queue

Analytics

Policies

Access

Audit

Models ready

AK Administrator

Overview > Batch

Batch evaluation

Run up to 500 claim records through both models, the warranty rules and the decision table. Nothing is saved as a claim; results are kept here for download.

Upload

CSV, UTF-8, at most 2 MB and 500 rows

Claim records

Same columns as the training splits. A claim_class column adds accuracy.

Choose File

No file chosen

No file to hand? [Download the 225-claim test split](#) and upload it.

Required columns (29)

Run the batch

Recent batches

No batches yet

Upload a file to run the first one.

Every warranty claim checked twice: a Python model, a Teachable Machine image model and a policy engine agree, or a person decides.

Python model v2.0.0

Teachable Machine gtm-514702678b8c

Product

How a claim is decided

Measured results

Model card

Resources

Technical blog

Read it on Medium

GitHub repository

AssureX

AssureX Claim Engine · Built for TechWiz 7 · NextWave AI and ML

29_batch_evaluation.jpg — Batch evaluation of uploaded claim records through the real pipeline

Dashboard
Products
Claims

Models ready
UT
Customer

Notifications

Claim updates, questions from reviewers and warranties about to end. Opening one marks it as read.

Unread 19
All 19

Unread

More information needed for CLM-5F9B6AD008

Reviewer request: Please upload a diagnostic report from an authorised center confirming the cause.

2m ago - CLM-5F9B6AD008

Open claim
Mark as read

Review complete: CLM-03538133C3

Outcome: Rejected.

2m ago - CLM-03538133C3

Open claim
Mark as read

Claim CLM-03538133C3 rejected

Automated evaluation rejected your claim. Reason: A policy hard-fail rule was triggered

2m ago - CLM-03538133C3

Open claim
Mark as read

Claim CLM-03538133C3: Rejected

Recommendation: Likely Invalid. A policy hard-fail rule was triggered.

2m ago - CLM-03538133C3

Open claim
Mark as read

Claim CLM-03538133C3 received

Your claim for AeroBreeze 1.5T was submitted and evaluated.

2m ago - CLM-03538133C3

Open claim
Mark as read

Review complete: CLM-19E100161E

Outcome: Rejected.

2m ago - CLM-19E100161E

Open claim
Mark as read

Claim CLM-19E100161E rejected

Automated evaluation rejected your claim. Reason: A policy hard-fail rule was triggered

2m ago - CLM-19E100161E

Open claim
Mark as read

Claim CLM-19E100161E: Rejected

Recommendation: Likely Invalid. A policy hard-fail rule was triggered.

2m ago - CLM-19E100161E

Open claim
Mark as read

Claim CLM-19E100161E received

Your claim for NovaPhone 12 was submitted and evaluated.

2m ago - CLM-19E100161E

Open claim
Mark as read

Claim CLM-5F9B6AD008: Manual Review

Recommendation: Manual Review Required. A policy manual-review rule was triggered.

2m ago - CLM-5F9B6AD008

Open claim
Mark as read

Claim CLM-5F9B6AD008 received

Your claim for VoltEdge Saw 18V was submitted and evaluated.

2m ago - CLM-5F9B6AD008

Open claim
Mark as read

Review complete: CLM-E77387B57E

Outcome: Rejected.

2m ago - CLM-E77387B57E

Open claim
Mark as read

Claim CLM-E77387B57E rejected

Automated evaluation rejected your claim. Reason: A policy hard-fail rule was triggered

2m ago - CLM-E77387B57E

Open claim
Mark as read

Claim CLM-E77387B57E: Rejected

Recommendation: Likely Invalid. A policy hard-fail rule was triggered.

2m ago - CLM-E77387B57E

Open claim
Mark as read

Claim CLM-E77387B57E received

Your claim for FrostGuard 450L was submitted and evaluated.

2m ago - CLM-E77387B57E

Open claim
Mark as read

Review complete: CLM-A38B0D4BB4

Outcome: Approved.

2m ago - CLM-A38B0D4BB4

Open claim
Mark as read

Claim CLM-A38B0D4BB4 approved

Automated evaluation approved your claim for ApexBook Pro 16.

2m ago - CLM-A38B0D4BB4

Open claim
Mark as read

Claim CLM-A38B0D4BB4: Approved

Recommendation: Likely Valid. Both models predict Valid and no blocking rule fired.

2m ago - CLM-A38B0D4BB4

Open claim
Mark as read

Claim CLM-A38B0D4BB4 received

Your claim for ApexBook Pro 16 was submitted and evaluated.

2m ago - CLM-A38B0D4BB4

Open claim
Mark as read

Every warranty claim checked twice: a Python model, a Teachable Machine image model and a policy engine agree, or a person decides.

Python model v2.0.0
Teachable Machine gte-5127026788c

Product

How a claim is decided
Measured results
Model card

Resources

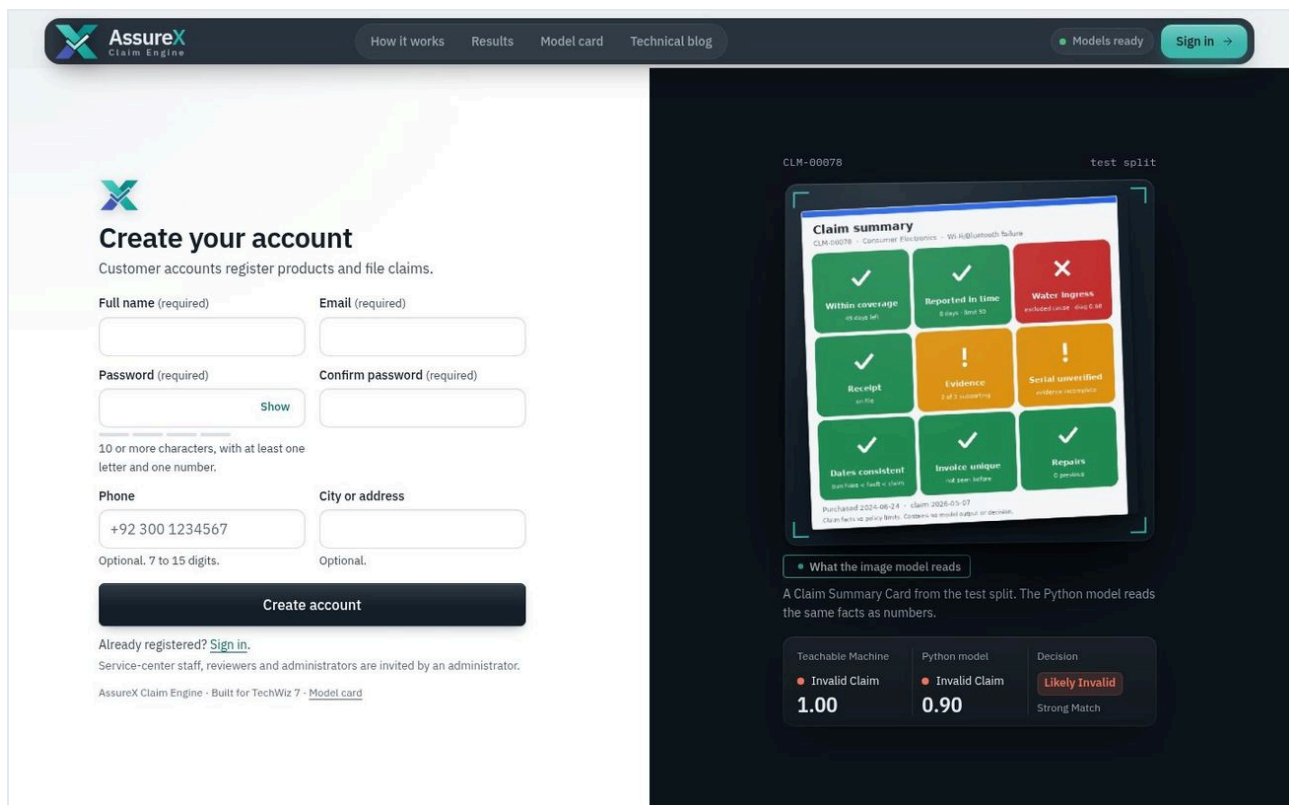
Technical blog
Read it on Medium
GitHub repository

AssureX Claim Engine - Built for TensorFlow 2 - NextWave AI and ML

30_notifications.jpg — Notifications page with unread and all tabs

AssureX Claim Engine — Project Documentation · Group NN_DevStorm

Page 76 of 106



31_register.jpg — Customer self-registration with field rules, password strength and a live match check

10. Deployed application

Item	Value
Public application URL	https://assurex.pythonanywhere.com
Source code	https://github.com/Saba1512006/assurex-claim-engine
Production server	<code>gunicorn wsgi:app</code> (Render: <code>render.yaml</code> ; PythonAnywhere: the WSGI file imports <code>application</code> from <code>wsgi.py</code>)
Health check	<code>/healthz</code> reports whether both models are loaded
Free-tier limitation	The free PythonAnywhere account runs Python 3.10 with a 512 MB disk quota, so both models can show as <i>Unavailable</i> there and claims then go to manual review; the full pipeline runs locally (see the installation instructions) and is shown in the demonstration video

Testing the deployed application

1. Open <https://assurex.pythonanywhere.com>. **Try a real claim** on the landing page runs three sample claims through both models and the rules live, without signing in. **Model card** shows how both models were measured.
2. **Sign in**: one click on an evaluator account on the sign-in page.
3. **Customer**: the dashboard lists what needs attention, product warranty plates and all claims; open a claim for the verdict (agreement gauge, both models, why, what the image model saw); *File a claim* runs the four-step wizard (a receipt PDF from any demo claim's evidence can be reused).
4. **Reviewer**: *Review queue > Start reviewing* opens the workbench. Keys: A approve, R reject, I request information, / comment, N next claim, ? help.
5. **Administrator**: *Overview* (KPIs and charts, each with a table view), *What-if simulator* (move the agreement thresholds and see the effect on 225 labelled claims before saving), *Batch evaluation* (upload the offered test-split CSV), *Policies* (change preview and version history), *Analytics*, *Models*, *Access*, *Audit*, CSV/Excel exports including every reviewer decision next to the models.
6. `/healthz` reports whether both models are loaded.

Evaluator accounts (demo data only)

Role	Email	Password	Can
Customer	customer@assurex.local	CustomerPass123!	Register products, file claims, upload evidence, track
Service-center staff	staff@assurex.local	StaffPass123!	File claims and register products for walk-in customers of <i>their</i> center, record repairs
Claim reviewer	reviewer@assurex.local	ReviewerPass123!	Work the manual-review queue, decide, request information, override with a reason
Administrator	admin@assurex.local	AdminPass123!	Dashboards, analytics, policies, thresholds, models, users, audit trail, exports

Public sign-up creates **customer** accounts only. Other roles are invited by an administrator (*Access* page) and must set their own password at first sign-in.

11. Demonstration video

Video: [demo_video/AssureX_Demo.mp4](https://github.com/Saba1512006/assurex-claim-engine/blob/demo-video/AssureX_Demo.mp4) — one file, 13 min 37 s, 1920 × 1080, 30 fps, H.264 with AAC audio, recorded from the running application against a freshly seeded database. Download: https://github.com/Saba1512006/assurex-claim-engine/blob/demo-video/AssureX_Demo.mp4

- **Narration:** English, generated with the open-source Kokoro text-to-speech model (voice [af_heart](#)) from the script below, and placed at the timestamps shown. Declared in [AI_USAGE.md](#) §3.
- **On screen:** every narrated step also has a caption, and the element being discussed is outlined, so the video can be followed with the sound off.
- **All four roles** are shown signed in: customer (Parts 2–3), claim reviewer (Part 5), service-center staff (Part 6) and administrator (Parts 4, 7 and 8).
- **All 11 SRS demonstration cases** are shown in Part 4, each decided live: valid, invalid, manual review, expired warranty, missing document, duplicate, contradiction, serial mismatch, unauthorised repair, boundary date and model disagreement.
- **Nothing is staged:** every claim, prediction and decision is produced live by the application; the recording is unedited.

SRS §1.10 item 13 — where each required step is shown

Required in the video	Time
User registration or login	1:44
Product registration	2:12
Warranty registration	2:37
Claim creation	2:52
Receipt and document upload	3:06
OCR extraction	2:18
Extracted-data verification	2:29
Data preprocessing	3:54
Python prediction and confidence for all three classes	3:54
Claim Summary Card generation	4:42
Teachable Machine classification and confidences	4:05
Comparison of both predictions and confidence difference	4:15
Warranty-rule execution	4:25
Missing-document detection	6:25
Contradiction detection	6:54
Duplicate-claim detection	6:40
Manual-review routing	8:15
Reviewer comments or override	8:35
Administrator dashboard	10:19
Claim status tracking	4:58
Report generation	5:06
One valid claim	5:35
One invalid claim	5:45
One manual-review claim	5:58

Required in the video	Time
One tricky boundary case	7:34
One model-disagreement case	7:49

Chapters

Time	Chapter
0:04	Title card: Warranty claims, checked twice
0:29	PART 1 · The live check
1:32	PART 2 · Customer journey
3:36	PART 3 · The decision
5:14	PART 4 · The 11 SRS demonstration cases
8:00	PART 5 · Manual review (claim reviewer)
8:54	PART 6 · Service-center desk (service-center staff)
10:03	PART 7 · Administration (administrator)
11:29	PART 8 · Beyond the requirements
13:11	Closing card: Two models. One rulebook.

Script

Time	On screen	Narration
0:05	Title card: Warranty claims, checked twice	Hi, and welcome to Assure X, a warranty claim engine built for TechWiz 7. Every claim is checked twice: once by a Python machine learning model, and once by a Google Teachable Machine image model. A configurable warranty policy engine then turns those results into a decision, and whenever the system is unsure, a person decides.
0:30	PART 1 · The live check	Let's start with the landing page, and its live check.
0:37	The live check runs a real sample claim through the whole pipeline	Right on the home page, this live check runs a real sample claim through the complete pipeline, the same code the product uses.
0:45	Clean fault: both models say Valid, no rule blocks it, so the claim is Likely Valid	Here's a clean fault. Both models predict Valid Claim, and no warranty rule blocks it, so the recommendation is Likely Valid.
0:57	Liquid damage: an excluded cause confirmed by a technician gives Likely Invalid	Now liquid damage. That cause is excluded by the policy, and a technician confirmed it, so a hard-fail rule makes it Likely Invalid.
1:08	Filed four days into the grace period: Uncertain Result, so a person reviews it	And this one was filed four days into the grace period. Confidence drops, the result is uncertain, so the claim goes to a person instead of being auto-decided.
1:22	Behind the card: two models feed the decision core, the policy engine and the outcome	Behind the card, you can see the flow: the Python model and the vision model feed a decision core, which applies the warranty policy and produces the outcome.
1:33	PART 2 · Customer journey	Part two: the customer journey. We'll register, add a product with O C R, and file a claim from start to finish.
1:44	User registration: every field is validated in the browser and on the server	First, a new customer creates an account. Every field is validated, both in the browser and again on the server.
1:56	Sign in with the new account	Now we sign in with the new account.

Time	On screen	Narration
2:02	The customer dashboard: products, warranties, claims and pending actions	This is the customer dashboard. Products, warranties, claims, and anything waiting on the customer, all in one place.
2:12	Product registration: upload the purchase receipt and OCR reads it	To register a product, the customer simply uploads the purchase receipt, and O C R reads it.
2:18	OCR fills in product, model, serial number, purchase date, price, retailer and invoice number	In a moment, the product name, model, serial number, purchase date, price, retailer and invoice number are all filled in automatically.
2:29	The customer checks each extracted field before saving	The customer checks each extracted field before anything is saved.
2:37	The category sets the warranty length: standard or extended cover	The category sets the warranty length, with standard or extended cover.
2:45	The product and its warranty: coverage dates, status and documents	And here's the registered product, with its warranty coverage dates, status and documents.
2:52	Claim creation: choose the product	Now let's file a claim. Step one: choose the product.
2:58	Fault details: category, cause, start date and a description	Step two: the fault. Its category, the cause, when it started, and a short description.
3:06	Upload the receipt: it is read again and compared with the registered product	Step three: documents. The receipt is read again, and compared with the product on record.
3:14	OCR check: the serial number on the receipt matches the registered product	The serial number on the receipt matches the registered product, so this check passes.
3:19	Supporting evidence: warranty card, damage photo and serial-number photo	We add the supporting evidence: the warranty card, a photo of the damage, and a photo of the serial number.
3:29	Review the claim, then submit	Finally, a quick review, and we submit.
3:37	PART 3 • The decision	Part three: the decision. This is the heart of the system.
3:44	The verdict: the recommendation, both models and the consistency check	Within a couple of seconds, the claim is decided. At the top is the recommendation, and the time the whole pipeline took.
3:54	Python model: confidence for Valid, Invalid and Manual Review	On the left is the Python model. The claim's data is pre-processed into features, and the model gives a confidence for each of the three classes.
4:05	Teachable Machine: classifies the Claim Summary Card, with its own three confidences	On the right is Google Teachable Machine. It classifies an image, the Claim Summary Card, and gives its own three confidences.
4:15	Comparison: do the classes match, the confidence difference, and the consistency status	In the middle, the two models are compared: do the predicted classes match, how far apart are their confidences, and are they consistent?
4:25	Why this decision: the warranty rules and the decision-table row that matched	Below that, why this decision. Every warranty rule from the category policy is listed, with the decision table row that matched.

Time	On screen	Narration
4:35	What moved the Python model: each input's effect on the prediction	For explainability, this shows which inputs moved the Python model, and by how much.
4:42	What the image model saw: claim facts only, never a prediction or a decision	And this is exactly what the image model saw. The card shows claim facts only, never a prediction or a decision. The shaded tiles show which parts drove its answer.
4:58	Claim status tracking: eight stages from filing to closing	The customer can track the claim through eight stages, from filing to closing.
5:06	Report generation: a PDF with the claim, evidence, both predictions, rules and decision	And with one click, a P D F report is generated, with the claim, the evidence, both predictions, the rules and the decision.
5:15	PART 4 · The 11 SRS demonstration cases	Part four: all eleven demonstration cases from the requirements, each one decided live by the application.
5:35	Valid claim: both models agree Valid and no rule blocks it: Likely Valid	One. A valid claim. Both models agree it's valid, no rule blocks it, so it's Likely Valid.
5:45	Invalid claim: an excluded cause confirmed by a technician: Likely Invalid	Two. An invalid claim. Water damage is excluded by the policy, and a technician confirmed it, so a hard-fail rule makes it Likely Invalid.
5:58	Manual-review claim: the cause is unknown, so a person has to look at it	Three. A manual review claim. The cause is unknown and the diagnosis is inconclusive, so a review rule sends it to a person.
6:11	Expired warranty: it ended 195 days before the claim, far past the grace period	Four. An expired warranty. The cover ended a hundred and ninety-five days before the claim, far past the seven-day grace period, so it's Likely Invalid.
6:25	Missing documents: the mandatory receipt is missing, so the claim waits	Five. Missing document detection. The mandatory receipt isn't there, so the claim waits for more information, and the customer is told exactly what's needed.
6:40	Duplicate detection: the same documents were already used in another claim	Six. Duplicate detection. The same warranty card and invoice were already used in another claim, so this one is flagged as a possible duplicate and sent for review.
6:54	Contradiction detection: the fault is dated before the purchase date	Seven. Contradiction detection. The fault is dated before the product was even bought, which is impossible, so the claim is routed to review.
7:08	Serial-number mismatch: the receipt shows a different serial than the registered unit	Eight. A serial number mismatch. The serial on the receipt doesn't match the registered unit, so the claim is Likely Invalid.
7:19	Unauthorised repair: the unit was repaired at an unauthorised service center	Nine. An unauthorised repair. The repair history shows work by an unauthorised service center, so the policy's hard-fail rule makes it Likely Invalid.
7:34	Boundary case: filed four days into a seven-day grace period	Ten. A tricky boundary case. It was filed four days into a seven-day grace period. The confidence is low, so the result is uncertain, and a person reviews it.
7:49	Model disagreement: Python says Invalid, Teachable Machine says Valid, so a person decides	And eleven. A model disagreement. Python says invalid, Teachable Machine says valid. When the models disagree, the system never guesses. A person decides.
8:01	PART 5 · Manual review (claim reviewer)	Part five: manual review.
8:15	The reviewer queue: filters, risk levels and waiting time	Claims that need a person land in the reviewer's queue, with filters and risk levels.

Time	On screen	Narration
8:23	The workbench: evidence, both models' scores and the warranty facts side by side	Let's open the disagreement case. The workbench puts the evidence, both models' scores and the warranty facts side by side.
8:35	Approve with a written comment, and a written reason for the override	The reviewer approves it, and writes a comment. Because this overrides the automated result, a written reason is required.
8:46	The decision is recorded, the customer is notified and the status changes	The decision is recorded, the customer is notified, and the claim's status changes. The override is kept in the audit trail.
8:55	PART 6 · Service-center desk (service-center staff)	Part six: the service center desk, the fourth role.
9:08	Service-center staff: only this center's claims and products, with the customer on every row	Service center staff sign in to their own desk. They see only the claims and products of their service center, with the customer's name on every row.
9:21	Staff file claims for walk-in customers, with the technician's diagnosis recorded	Staff can file a claim on behalf of a walk-in customer. This one was filed for Usman Tariq by Sana Malik, and the technician's diagnosis and confidence are recorded with it.
9:38	In the claim form, each product shows its owner, and the diagnostic confidence is required	In the claim form, every product shows its owner. And for staff, the technician's diagnostic confidence is a required field.
9:52	Staff record repairs; an unauthorised repair is flagged and checked by the warranty rules	Staff also record repairs on the product. This unit was repaired at an unauthorised center, so it's flagged here, and the warranty rules take it into account on every future claim.
10:04	PART 7 · Administration (administrator)	Part seven: administration.
10:19	Administrator dashboard: claims, automation rate, model agreement and workload	The administrator dashboard shows claim volume, the automation rate, model agreement and the review workload.
10:29	Charts: claims per week, recommendations, model agreement and confidence	The charts break down claims per week, recommendations, model agreement and confidence.
10:38	Analytics: frequent faults, rejection reasons, category trends and model evidence	Analytics shows the most frequent faults, the reasons claims are rejected, trends per category, and the evidence for both models.
10:49	Warranty policies are configuration: coverage, grace periods, exclusions and rule severity	Warranty policies are configuration, not code. Coverage, grace periods, excluded causes and rule severity can all be changed here, with version history.
11:02	Models: both installed with versions; the Teachable Machine export is uploaded and evaluated here	On the models page, both models are installed with their versions. A new Teachable Machine export can be uploaded and evaluated right here.
11:14	The audit trail: every login, upload, prediction, decision and override	The audit trail records every login, upload, prediction, decision and override.
11:22	Access control: roles, record scopes, invitations and blocked attempts	And access control manages roles, which records each role can see, invitations, and blocked attempts.

Time	On screen	Narration
11:30	PART 8 · Beyond the requirements	Part eight: going beyond the requirements.
11:37	What-if simulator: try a stricter threshold on 225 stored test decisions	The what-if simulator lets an administrator try a stricter confidence threshold on all 225 stored test decisions, before changing anything.
11:49	Automation rate, review load and accuracy update instantly; nothing is saved until Apply	Watch the automation rate, the review load and the accuracy update instantly. Nothing is saved until you press apply.
11:59	Batch evaluation: run uploaded claim records through the real pipeline	Batch evaluation runs a file of claim records through the real pipeline, chunk by chunk.
12:06	30 unseen test claims evaluated: accuracy against the labels, with a downloadable CSV	Thirty unseen test claims, evaluated, with accuracy against their labels, and a downloadable C S V.
12:16	Policy changes are previewed and versioned before they apply	Policy changes are previewed and versioned. And if a change affects a limit shown on the card, the system warns that the image model needs retraining.
12:30	Input safety: names take no digits and phone numbers take no letters	Input safety: names can't contain digits, and phone numbers can't contain letters. This is enforced in the browser and on the server.
12:41	Sign-in protection: five wrong passwords pause the account for one minute	Sign-in is protected too. After five wrong passwords, the account is paused for one minute, with a live countdown.
12:52	The public model card: metrics, confusion matrices and errors for both models	Finally, the public model card. Both models' metrics, confusion matrices and errors, read directly from the evaluation files.
13:03	The technical blog, also published on Medium	And there's a technical blog explaining how it was built, also published on Medium.
13:12	Closing card: Two models. One rulebook.	The Python model reaches eighty-nine point three percent, and Teachable Machine ninety-three point eight percent, on two hundred and twenty-five unseen test claims. Backed by two hundred and ninety-five automated tests. Two models, one rulebook, and a person whenever it matters. Thank you for watching.

12. Technical blog

Field	Value
Title	Building AssureX: two models, one rulebook, and why our first 100% was a bug
Current text	documentation/TECHNICAL_BLOG.md — about 3,000 words; also served in the app at /blog
Medium-ready copy	documentation/medium/ — medium_article.html (paste into Medium) + 5 images + step-by-step guide
Published (current version)	https://medium.com/@sabarajput672/building-assurex-two-models-one-rulebook-and-why-our-first-100-was-a-bug-bb8b9858b11c
Earlier version (v1, superseded)	https://medium.com/@samikhan031027/building-assurex-a-dual-model-warranty-claim-evaluation-system-8a30d3684111
Status	Current text published on Medium on 27 September 2026 (tags: Machine Learning, Python, Teachable Machine, Data Science, Flask). The v1 article describes the leaked 100% model and should be unlisted

SRS topic coverage (page 35–36)

Topic	Section in the blog
Business problem	The business problem
Background and necessity	Background and why it matters
Proposed solution	The proposed solution
Application architecture	Application architecture
Dataset creation	Dataset creation
Dataset challenges	Dataset challenges
Python model development	Python model development
Algorithms compared	Algorithms compared
Google Teachable Machine training	Google Teachable Machine training and the Claim Summary Card
Claim Summary Card generation	same section
Python integration	Python integration
Model prediction comparison	Model prediction comparison
Confidence-score comparison	Confidence-score comparison
Warranty-rule design	Warranty-rule design
OCR and document processing	OCR and document processing
Difficulties encountered	Difficulties encountered
Model errors	Model errors
Model disagreement cases	Model disagreement cases
Testing results	Testing results
Security considerations	Security considerations
Limitations	Limitations
Lessons learned	Lessons learned
Future enhancements	Future enhancements

How we built a warranty-claim engine that combines a Python classifier, a Google Teachable Machine image model and a configurable rule engine — and what we learned when our first version scored suspiciously well.

The business problem

A warranty claim looks simple: a customer says a product broke, the manufacturer pays for the repair. Behind that sentence is a checklist that service centers run by hand thousands of times a month. Was the product bought when the customer says? Is the warranty still active, or inside its grace period? Is the fault covered, or is it water damage the policy excludes? Does the serial number on the unit match the receipt? Has someone already claimed on this invoice? Was the product opened by an unauthorised repair shop?

Every one of those checks is easy on its own. Together they are slow, and different people apply them differently. The same claim can be approved on Monday and rejected on Tuesday. Fraud slips through when a receipt is reused for a second unit. Honest customers wait days for a decision that could have taken seconds. AssureX automates the first pass, explains every recommendation, and sends a claim to a human whenever the evidence is not clear.

Background and why it matters

Warranty cost goes straight to a manufacturer's bottom line, and a badly handled claim costs a customer. Two properties matter more than raw accuracy: **consistency** (the same facts always get the same answer) and **accountability** (anyone can see why an answer was given, and who changed it). That pushed us towards a design where machine learning supports decisions but a readable policy stays in charge, and where nothing happens without leaving a record.

The proposed solution

A customer — or a service-center employee helping a walk-in customer — registers a product with its receipt, then files a claim in four steps: pick the product, describe what happened, attach evidence, review. While they type, a checklist tells them what is missing, whether the reporting deadline is close, and whether the cause they chose is excluded for that product category.

On submission the claim is evaluated twice, independently:

- a **Python classification model** reads the structured claim record;
- a **Google Teachable Machine** image model reads a picture of the same claim — the *Claim Summary Card*.

A **rule engine** checks the category's warranty policy, and detectors look for **contradictions** and **duplicates**. A **decision table** combines everything into one of three recommendations: *Likely Valid*, *Likely Invalid* or *Manual Review Required*. Reviewers work a queue of the uncertain claims; customers follow their claim through eight stages; administrators get dashboards, analytics, exports and an audit trail.

Application architecture

The application is Flask with SQLAlchemy. Requests pass through an access-control layer before they reach a blueprint (auth, products, claims, reviewer, admin). Blueprints call services — claim lifecycle, documents, notifications, alerts, analytics, exports, PDF reports — and the evaluation pipeline:

1. document service (validate by magic bytes, store under a random name, SHA-256, OCR);
2. contradiction and duplicate detectors;
3. feature builder, producing exactly the columns the model was trained on;
4. rule engine driven by `policies/<category>.json`;
5. Python model → probabilities for all three classes;
6. card renderer → Teachable Machine → probabilities for all three classes;
7. consistency status and decision table driven by `config/decision_policy.json`.

Every run writes a new, immutable evaluation row holding the inputs, both model versions (hashes of the model files), the card's hash, the thresholds and the decision trace. Updating a model never rewrites a past result.

Dataset creation

The competition gives no dataset, so we generated one: 1,500 claims, 500 per class, across consumer electronics, home appliances and industrial tools, split 70/15/15 with stratification (1,050 / 225 / 225).

Our first generator worked backwards: *pick a class, then invent a claim that looks like it*. That is the natural way to think about test data and it is exactly wrong for machine learning. The second generator works forwards, the way real claims arrive. It samples each claim's attributes from overlapping, realistic distributions — where in the warranty life the claim falls, how long the customer waited, what caused the damage, how sure the technician is, repair history, which documents were attached — with no knowledge of the class. Then it **derives** the label by applying the warranty policy, and flips 4% of labels to simulate two reviewers disagreeing. Claim IDs are assigned after shuffling.

The scenarios that emerge cover what the brief asks for: normal covered defects, expired warranties, claims inside the grace period, excluded damage, missing documents, contradictory dates, serial mismatches, unauthorised repairs, repeat repairs, possible duplicates and late reporting.

Dataset challenges

Three problems taught us the most.

Label leakage. Version 1 had a column called `damage_type` whose values mapped one-to-one onto the class — "Hardware Defect" was always valid. Any model learned that single column and scored 100%. Worse, the claim IDs were ordered by class (1-500 valid), so even the ID leaked. We now run a *leakage audit* before training: a small decision tree is trained on each feature alone, and training stops if any single feature predicts the class above 90%. The best single feature today, days to expiry, reaches 53%.

Train/serve skew. The web form offered damage types that the training data had never seen, and an encoder configured to "ignore unknown values" silently turned them into zeros. The model then scored claims it had never seen anything like. Now one vocabulary module feeds the generator, the form, the policies and the encoder; unknown values raise an error; and a test renders the claim form and checks that its options equal the training values.

Rare scenarios. Late reporting is only 23 of 1,500 claims. The model sees few examples and misses some of them — which is fine, because the rule engine never misses them.

Python model development

Pre-processing turns a claim into 20 features: eight numeric (price, warranty length, product age, days to expiry, reporting delay, diagnostic confidence, repair count, missing-document count), ten binary (documents present, extended warranty, serial match, unauthorised repair, duplicate invoice, date conflict) and two categorical (category, damage cause). Numbers are standardised for the linear model; categories are one-hot encoded with a fixed vocabulary. The encoder and model are saved as **one** pipeline file, so they can never drift apart.

Algorithms compared

We compared three algorithms with stratified 5-fold cross-validation on the training set, then on the validation set:

Algorithm	CV macro-F1	Validation macro-F1
Logistic regression	0.739	0.726
Random forest	0.922	0.942
HistGradientBoosting	0.913	0.947

The model is chosen on **validation**; the test set is touched exactly once at the end. (Version 1 picked the model by its test score, which quietly turns the test set into training data.) The winner is wrapped in sigmoid calibration so that "80% confident" means right about 80% of the time — essential, because the comparison between our two models is built on those confidence values.

On 225 unseen test claims the calibrated model reaches **89.3% accuracy**, macro-F1 89.2%, ROC-AUC 0.948 and an expected calibration error of 0.082. The SRS asks for 85%. Given the 4% label noise, the practical ceiling is about 96%.

Google Teachable Machine training and the Claim Summary Card

The second model is an image classifier trained in the browser with Google Teachable Machine on pictures of claims. The card had to be redesigned before that could work.

Version 1 cards were 640 × 420 landscape images full of 12-pixel text. Teachable Machine centre-crops every upload to a square, which cut off both side columns; and an image model cannot read small text anyway. The fonts also fell back to a bitmap font on the Linux server, so live cards looked different from training cards.

Version 2 cards are 600 × 600 and use fonts bundled with the application. They drew every fact as a small grey shape in a fixed position: bars for warranty life, reporting delay and diagnostic confidence, hatched tiles for missing documents, pips for category and damage cause. It looked tidy and it failed: the real Teachable Machine model trained on those cards scored 54.2% on the test cards. Teachable Machine does not retrain the image network; it puts a small classifier on top of frozen ImageNet features, and at 224 × 224 pixels thin grey bars look nearly the same whatever their length. We rebuilt Teachable Machine's trainer offline ([notebooks/tm_replica_check.py](#)) and it reproduced the failure (58–66%), which told us the card, not the training run, was the problem.

Version 3 shows each fact against its policy limit as one of nine large tiles in a fixed 3 × 3 grid — coverage, reporting time, damage cause, receipt, supporting evidence, serial number, dates, invoice and repairs. A tile's colour and glyph change with the fact (within coverage, grace period or coverage ended; reported in time or late; covered or excluded cause), and a colour band shows the category. The same replica now scores 92.9–94.2% on the test cards, and the real Teachable Machine model trained on the version 3 cards scored 93.8% (validation 97.3%, macro F1 93.8%), agreeing with the Python model on 92.9% of the test claims. Text is still there, for humans. The card never shows a prediction, a confidence, a rule outcome or a decision.

Each training claim is drawn twice with small, label-preserving variations — background tint, date format, a slight rotation, blur and JPEG quality — giving 2,100 training images. Validation and test cards are rendered once each and never uploaded to Teachable Machine. The exported TensorFlow Lite model runs inside the application; if it is missing, the system says so and sends the claim to a reviewer rather than deciding on one model.

Python integration

The web application never retrains a model and never silently falls back to a different one. The Python model is loaded once per process from the pipeline file; its version string embeds the file's hash. The feature builder that feeds it lives in the application, and the training script imports the

same feature lists. If the file is missing or was pickled by an incompatible library version, the claim is still evaluated by the rules, the model is marked unavailable, and the claim is routed to manual review with a clear message.

Model prediction comparison

For each claim the application shows both predicted classes side by side with all three probabilities, whether the classes match, and the Claim Summary Card that the image model saw. The comparison ends in one of five statuses. If either model's top confidence is below 0.60 the result is an **Uncertain Result**. If the classes differ it is a **Model Disagreement**. Otherwise the gap between the two top confidences decides: up to 0.10 is a **Strong Match**, up to 0.25 an **Acceptable Match**, anything larger a **Weak Match**.

Confidence-score comparison

The confidence difference is $|\text{Python top confidence} - \text{Teachable Machine top confidence}|$. The two models are trained on different representations and will never agree exactly, so the thresholds absorb normal variation. Disagreement, uncertainty and weak matches all send the claim to manual review. All three thresholds live in a configuration file that administrators edit in the app, and each evaluation stores the thresholds it used.

Warranty-rule design

Each product category has a JSON policy: coverage length, standard and extended terms, grace period, reporting deadline, covered faults, excluded damage causes, how sure a technician must be to confirm an exclusion, repeat-repair threshold, repair and replacement conditions, mandatory and supporting documents, and three lists of rules — hard-fail, manual-review and warning. The engine knows sixteen checks. Which list a check is in decides its severity; a check in no list is switched off.

The final decision is a table read top to bottom, first match wins: a hard-fail rule makes a claim *Likely Invalid*; contradictions, duplicates, a missing mandatory document, model disagreement or uncertainty, or a manual-review rule make it *Manual Review Required*; only then do the two models decide — both Invalid gives *Likely Invalid*, both Valid gives *Likely Valid*; anything else goes to a reviewer. Changing the decision logic, adding an exclusion or moving a threshold is a configuration edit, not a code change — which is what the competition's "surprise modification" round asks for, and each of those modifications has a test.

OCR and document processing

Receipts are read with pdfplumber (text PDFs) and Tesseract (photos and scans). We extract invoice number, purchase date, product name, model, serial number, retailer, amount and warranty length with labelled patterns only. The first version also compared text against hard-coded lists of known retailers and products — it worked beautifully on our demo receipts and would have failed on any hidden test receipt, so it is gone. Identifiers must contain a digit, so the word "Invoice" is never mistaken for an invoice number. Every extracted value is shown to the user, who confirms or corrects it; corrections are audited. Files are validated by their first bytes, not their extension: a text file renamed to `.png` is refused.

Difficulties encountered

- Writing a generator that does **not** know the answer was harder than writing one that does.
- Making image-model inputs model-friendly meant designing a picture for a neural network, not a person.

- Keeping one vocabulary across form, dataset, model, policies and cards required a test that renders the form.
- Access control for four roles needed record scope, not just role checks: a service-center employee must see their own center's claims and nothing else, and a reviewer must not approve a claim they filed.

Model errors

On the test split the Python model's weakest spot is Invalid claims predicted as Manual Review (8 of 75). Of the 14 Invalid claims it misses, 5 are late-reporting cases (rare in training), 4 carry deliberately flipped labels, 3 are excluded-damage claims with a diagnosis right at the threshold, and 2 sit on the expiry boundary. Every one of those is a hard-fail rule in the policy, so the rule engine still rejects them correctly. That is the point of combining a model with rules: the model generalises, the rules guarantee.

Model disagreement cases

Disagreement is information, not failure. When the two models disagree the claim goes to a reviewer, who sees both predictions, every rule result and the evidence on one page.

On the 225 test claims the two models predict the same class for 92.9% of them. The comparison statuses are 117 Strong Match, 68 Acceptable Match, 15 Uncertain Result, 13 Weak Match and 12 Model Disagreement — 40 claims that no single model is allowed to settle. The disagreements are instructive. In several late-reporting and excluded-damage claims the Python model hesitates between *Invalid* and *Manual Review* while the image model is confident the claim is invalid: the card puts the red "Reported late" or "excluded cause" tile right in front of it. Some disagreements point at the data rather than the models: in two claims whose labels were deliberately flipped to simulate reviewer disagreement, the image model follows the policy and the Python model follows the flipped label. Whatever the cause, a disagreement never becomes an automatic decision: the decision table sends the claim to a person, usually with a rule result already explaining the problem. Across the whole test split the application's final decision matches the labelled class for 92.0% of claims.

Testing results

197 automated tests run in about 25 seconds: rule boundaries (last day of cover, first day after the grace period, the reporting deadline ± 1 day, the exclusion threshold), contradiction and duplicate detection, OCR patterns, upload validation, both model runtimes, all five consistency statuses, the eleven demonstration cases from the competition brief, full HTTP journeys from registration to approval, and the security controls below. We also render every page for every role and take browser screenshots on desktop and phone widths.

Security considerations

Access is denied by default. Each role holds a list of permissions, each permission a scope — own records, own service center, the review queue, or everything — and each action passes business constraints: reviewers cannot decide claims they filed, claims move only along allowed transitions, overriding the automated recommendation needs a written reason, and administrators cannot demote themselves or the last administrator. A record outside your scope returns "not found", so identifiers cannot be probed. Users are reloaded from the database on every request; a role change or disabled account signs them out everywhere. Sign-in errors never reveal whether an email exists, five failures lock the account, forms carry CSRF tokens, and the Content-Security-Policy forbids inline and third-party scripts. Every sensitive action — including blocked access attempts — goes to an append-only audit log.

Limitations

The dataset is synthetic; real claims will need retraining and recalibration. The Teachable Machine model must be trained in the browser and re-exported whenever the card design changes. Image OCR depends on Tesseract being installed on the server. Duplicate text detection is lexical, not semantic. SQLite suits a single server; larger deployments should move to PostgreSQL.

Lessons learned

- **Distrust a perfect score.** 100% accuracy on generated data is almost always leakage.
- **Select on validation, report on test, once.**
- **Calibrate before you compare confidences** between two models.
- **One vocabulary everywhere**, enforced by tests, prevents the quietest bugs.
- **Rules and models are partners.** Models generalise; rules guarantee the policy; the decision table makes their relationship explicit and editable.
- **Record everything**, including the model versions that produced each answer.

Future enhancements

Retraining on real claims with drift monitoring; learning from reviewer overrides; image-based damage detection on customer photos; semantic duplicate detection; email and SMS notifications; an API for service-center systems; and multi-manufacturer tenancy.

13. Security and access-control design

Model

Every request passes three checks, in order. Any failure denies.

1. **Permission** — the role holds the permission (`config/rbac.json`). Nothing is implied: deny by default.
2. **Scope** — the record is inside the role's reach: `own`, `service_center`, `assigned_or_queue`, `all`. Out-of-scope records return **404**, so IDs can't be probed.
3. **Constraints** — separation of duties, claim state machine, edit locks, admin self-protection.

The same scope drives list queries (`scoped_claims`) and single-record checks (`authorize_object`), so a record hidden on the detail page can never leak through search, dashboards or CSV export.

Roles

Role	Reaches	Can't
Customer	own products, claims, documents	see others' records, edit a submitted claim, review
Service-center staff	records of their service center	other centers, decisions, policies
Claim reviewer	unassigned queue + claims assigned to them	decide a claim they filed, skip states, override without a reason
Administrator	everything, read	decide claims (config owners don't adjudicate), change own role, remove the last admin

Loopholes found in v1 and how v2 closes them

#	v1 behaviour	v2
1	<code>/register</code> accepted <code>role=Admin</code>	public sign-up creates customers only; others are invited
2	role read from the session cookie; role change / disable took effect only after logout	user reloaded from DB each request; <code>session_version</code> revokes old sessions
3	<code>/claims/documents/<id>/correct-data</code> had no ownership check (any user could edit any OCR data)	<code>authorize_object("document.correct_ocr", doc)</code>
4	object checks existed only for customers (<code>if role == customer and curr_user...</code>)	scope check for every role
5	staff and reviewers saw every product, claim and document	service-center and assignment scopes
6	reviewer could approve own claim, re-open closed claims, override without reason	SoD + transition table + 20-char reason
7	login said whether an email exists; no lockout; session not rotated	one generic message, 5-try lockout, <code>session.clear()</code> on login
8	a new route was public unless someone added a decorator	before-request hook blocks any unguarded endpoint
9	no CSRF on state-changing forms	Flask-WTF CSRFProtect (<code>src/app_security.py</code>)

Evidence

[tests/test_rbac_policy.py](#) (32 cases, pure) and [tests/test_rbac_routes.py](#) (app-level: unguarded-route inventory, escalation, enumeration, session revocation, role x endpoint matrix, IDOR).

14. AI tool usage declaration

Required by SRS §1.8 (item 9) and deliverable 15. Every AI tool used during development is listed below. **The final claim decision is never produced by a generative-AI service:** it comes from the team's Python classification model, the Google Teachable Machine model, the warranty rule engine and the decision table running inside this application. No external AI API is called at runtime.

Correction. An earlier version of this file stated that no AI code generators were used. That is not accurate for the v2 upgrade described below, so this declaration replaces it.

1. Google Teachable Machine (required by the SRS)

Field	Declaration
Tool	Google Teachable Machine (teachablemachine.withgoogle.com), Google LLC
Purpose	Image classification of Claim Summary Cards into Valid Claim / Invalid Claim / Manual Review
Type of assistance	Browser-based transfer-learning of an image model on the team's training cards; TensorFlow Lite export
Files / modules affected	<code>model/teachable_machine/</code> (export + <code>labels.txt</code> + <code>evaluation.json</code>), consumed by <code>src/core/gtm_classifier_v2.py</code>
Status	Trained by the team in the browser on the v3 cards and installed as <code>gtm-514702678b8c</code> (test accuracy 93.8%); see <code>documentation/GTM_EVIDENCE.md</code>
Testing	<code>notebooks/evaluate_gtm.py</code> / Admin > Models > Run evaluation on the 225 validation and 225 test cards

2. Claude Code (Anthropic) — v2 upgrade

Field	Declaration
Tool	Claude Code (Anthropic), an AI coding assistant
Date	26 September 2026
Purpose	Integrate the team's v2 upgrade kit (leakage-free dataset, calibrated model, square cards, RBAC) into the application, fix the defects the kit identified, redesign the web interface, write tests and update documentation
Prompt / type of assistance	"Complete the project using the attached v2 upgrade kit, model card and SRS; implement everything fully with a top-quality design." Code generation, refactoring, test writing, documentation drafting, running the training scripts and the test suite
Files / modules affected	Almost every module: <code>src/</code> (core pipeline, rules, security, services, api), <code>templates/</code> , <code>static/</code> , <code>config/</code> , <code>policies/</code> , <code>dataset_generator/</code> , <code>notebooks/</code> , <code>database/seed.py</code> , <code>tests/</code> , <code>reports/</code> , <code>documentation/</code> , <code>README.md</code> . Generated data: <code>data/raw</code> , <code>data/splits</code> , <code>data/summary_cards</code> , <code>model/python_model/</code>
What it did not do	Train the Teachable Machine model (requires the browser tool); decide any claim at runtime; invent metrics (every number in the docs is read from files the scripts produce)
Testing performed by the tool	192 automated tests (<code>python -m pytest -q</code>), a page-by-page render check of every screen for all four roles, browser screenshots, and an end-to-end seed that submits the 11 demonstration claims through the live pipeline

2b. Claude Code (Anthropic) — frontend rebuild

Field	Declaration
Tool	Claude Code (Anthropic), an AI coding assistant
Date	27 September 2026

Field	Declaration
Purpose	Rebuild the web interface from the team's written brief, add the explainability, simulation and batch features it asked for, and test them
Prompt / type of assistance	The team's <i>AssureX Frontend Master Prompt</i> (PDF, 10 parts, prepared by Muhammad Azhar Nawaz), plus the instruction to implement all of it together with three model-explanation additions. Code generation, refactoring, test writing, running the test suites, browser screenshots, accessibility and Lighthouse audits
Files / modules affected	templates/ (every page), static/css/ , static/js/ , src/api/api.py , src/api/errors.py , src/services/ (verdict , whatif , batch , model_card_service , receipt_scan , paging), src/core/explain_models.py , src/core/pipeline.py (stored verdict payload and stage timings), src/app_security.py , tests/ (contract , route-matrix and tests/e2e/ browser tests), scripts/subset_icons.py , README.md , documentation/PROJECT_REPORT.md
What it did not do	Change the Claim Summary Card design or either model (tests re-render the test cards and fail if Teachable Machine accuracy could drop; the batch evaluator reproduces 89.3% and 93.8% on the test split); decide any claim at runtime
Testing performed by the tool	225 automated tests including a route x role matrix over every guarded route; 13 browser journeys on a live seeded server with axe-core accessibility checks on 21 pages; Lighthouse on the public pages; screenshots at 1440, 1024 and 390 px

2c. Claude Code (Anthropic) — styling pass, input rules and deployment fixes

Field	Declaration
Tool	Claude Code (Anthropic), an AI coding assistant
Date	27–28 September 2026
Purpose	Visual polish requested by the team, form input rules, sign-in lockout countdown, landing-page hero background, dependency fixes for deployment, and updating the documentation to the final state
Prompt / type of assistance	The team's instructions, given one change at a time (for example "no numbers in name fields", "show a timer when sign-in is locked", "premium navbar and footer", "preloader like the logo"). Code generation, CSS, tests, browser screenshots, running the test suites
Files / modules affected	templates/ (navbar, footer, base, landing, auth, dashboards, claim detail, workbench, notifications), static/css/ , static/js/ (app.js , instrument.js , hero_engine.js , hero_engine_math.js), static/img/brand/ , src/rules/validator.py , src/api/auth.py , src/api/access.py , src/api/products.py , src/app.py , config/rbac.json (lockout minutes), requirements.txt , tests/ (test_input_rules.py , test_hero_engine_math.py , tests/js/ , tests/e2e/test_hero_engine.py), README and documentation/
What it did not do	Change either model, the dataset or the Claim Summary Card (the accuracy guards re-score both models on every test run: 89.3% and 93.8%); decide any claim at runtime
Testing performed by the tool	295 automated tests and 34 browser tests (overlap, contrast and pixel checks of the hero background at five widths, input rules, lockout countdown, accessibility with axe)

Required team entries (SRS: modifications, testing and verifier name)

The SRS requires the **team** to review, modify, test and understand AI output, and to name the member who verified it:

Module	Modifications made by the team	Tests run by the team	Verified by (team member)
Dataset generator & cards (dataset_generator/)	Saba Noor reviewed the generated dataset (class-independent attributes, 4% label noise, stratified 1,050 / 225 / 225 split) and the v3 card design adopted after the first Teachable Machine run	Checked that no validation or test claim appears in training (CSV or image) and that each card matches its CSV record, Claim ID and label	Saba Noor
Python model training (notebooks/train_python_v2.py)	Saba Noor ran the training script, reviewed the three-algorithm comparison and the selection on the validation split	Checked the test metrics (89.3% accuracy, 89.2% macro F1), the leakage audit and the saved-model accuracy guard in the test suite	Saba Noor
Teachable Machine training & evaluation	Saba Noor trained the image model in the browser on the 2,100 training cards (epochs 50, batch 16, learning rate 0.001), retrained after the card redesign (v2 → v3), and installed the export through Admin > Models	Evaluated on the 225 validation and 225 test cards (93.8% test accuracy, 97.3% validation) and reviewed the misclassified samples	Saba Noor
Evaluation pipeline (src/core/)	Muhammad Sami wrote the first pipeline (OCR, comparison, decision engine) before the v2 upgrade; the team kept the v2 decision table and thresholds after review	Deployed and ran the app on PythonAnywhere and locally; Saba Noor compared the live-check results with the model card figures	Muhammad Sami, Saba Noor
Rule engine & policies (src/rules/ , policies/)	Muhammad Sami wrote the original rule set, multi-category policies, serial verification and contradiction/duplicate checks; kept as the basis of the config-driven v2 rules	Muhammad Sami created the sample_claims/ suite covering the SRS demonstration cases; Muhammad Sami ur Rehman ran all 11 through the app and reported the defects found	Muhammad Sami, Muhammad Sami ur Rehman
Security & access control (src/security/ , config/rbac.json)	Saba Noor reviewed the role permissions and asked for the one-minute lockout and the server-side input rules	Signed in as each of the four roles and checked what each can open; tested the lockout countdown and the rate-limit page	Saba Noor
Web interface (templates/ , static/)	Muhammad Sami tuned the live check card height and hero padding (28 Sept); Saba Noor requested and reviewed every page change	Saba Noor ran the app locally on Windows and checked each page for all four roles; Muhammad Sami ur Rehman tested the pages in Chrome and on mobile; Muhammad Ghanyan checked the mobile view	Saba Noor, Muhammad Sami ur Rehman
Frontend rebuild features (verdict, workbench, what-if, batch, model card)	Built from the team's written brief; Muhammad Ghanyan's Figma designs for the login and dashboard pages and colour palette guided the look	Muhammad Sami ur Rehman ran the 11 sample claims through the app and tested the pages in Chrome and on mobile; Muhammad Ghanyan checked the mobile view	Saba Noor, Muhammad Sami ur Rehman
Styling pass, input rules, lockout countdown, hero background (27–28 Sept)	Saba Noor specified each change and asked for corrections after testing (menu scrolling, notifications link, footer layout, hero placement, text clipping, lockout time)	Typed digits into name fields and letters into phone fields, locked an account with five wrong passwords and watched the countdown, checked the landing page at several window sizes	Saba Noor
Documentation & report	Saba Noor corrected links, figures and wording; the blog was published on Medium from her account	Checked every figure against reports/ and the model files	Saba Noor

3. Other tools

Tool	Use
scikit-learn, pandas, NumPy, Pillow, ReportLab, pdfplumber, Tesseract (pytesseract)	Libraries used by the application (not AI assistants)
Chart.js, Bootstrap Icons, Alpine.js (CSP build), IBM Plex fonts	Front-end libraries and fonts, self-hosted in static/vendor/
Playwright, axe-core (axe-playwright-python), Lighthouse	Browser testing, accessibility and performance audits (development only)
Mermaid	Rendering the report diagrams from documentation/diagrams/*.mmd
Kokoro text-to-speech (open-source, voice af_heart)	English narration of the demonstration video, generated from the script in documentation/DEMO_VOICEOVER.md ; the team wrote and checked the script against the application

Images: the demo "damage" and "serial" photos created by [database/seed.py](#) are synthetic drawings made with Pillow, labelled as such on the image. No AI-generated imagery is used in the application.

15. Team contribution record and development log

The SRS asks for a record of each member's contribution and for commits from every team member. The commit counts below come from `git shortlog -sne`; the contribution rows are the team's statement of who did what, and each member should be able to explain the modules listed against their name.

Group

Group	NN_DevStorm
Project	AssureX Claim Engine — NextWave AI and ML (Aptech Limited)
Batch	2609E2'
Faculty	Sir Minhaj

Student ID	Name	Role in the project
Student1525913	Saba Noor	Team lead · integration, frontend direction, QA, documentation
Student1524867	Muhammad Sami	Backend and ML pipeline developer · deployment
Student1509222	Muhammad Ghanyan	Frontend design (Figma, colour palette, mobile checks)
Student1505677	Muhammad Sami ur Rehman	Testing (sample claims, Chrome and mobile)

Commits on `main`

GitHub account	Commits
Saba1512006 (Saba Noor)	68
sami2515 (Muhammad Sami)	52

Counted on 28 September 2026 (120 commits, 23–28 September); per-day activity is in [documentation/DEVELOPMENT_LOG.md](#).

Regenerate before submitting: `git shortlog -sne main`. Every member should have commits under their own account.

Contributions

Team member	Role	Main contributions (modules, documents, experiments)	Evidence (commits, files)
Saba Noor	Team lead · integration, frontend direction, QA, documentation	Led the v2 upgrade: integrated the leakage-free dataset, calibrated Python model, v3 Claim Summary Cards and RBAC into the application (AI-assisted, declared in AI_USAGE.md). Recorded and documented the Teachable Machine v3 results (93.8% test accuracy), the comparison report and the accuracy guards. Directed the frontend rebuild and the styling pass page by page (navbar, sign-in/register, dashboards, claim detail, workbench, footer, preloader, hero background), tested every change in the browser on Windows and reported the defects that were fixed (notifications link, menu scrolling, input validation on the Access page, logout message, hero text clipping). Specified the form input rules and the one-minute logout countdown. Published the technical blog on Medium; maintained the README, project report, test cases, installation guide, development log and AI declaration; merged the team's fork and pinned the model dependencies for release.	68 commits (26–28 Sept) · src/ , templates/ , static/ , tests/ , documentation/ , reports/ , AI_USAGE.md
Muhammad Sami	Backend and ML pipeline developer · deployment	Built the first complete version of the application (23–26 Sept): multi-role registration and profiles; product registration and the common warranty interface; receipt/invoice upload with OCR extraction and verification; warranty tracking and expiry alerts; claim registration, fault evidence and repair history; document organisation, data validation and pre-processing; the algorithm comparison matrix; Claim Summary Card endpoint and Teachable Machine preview; confidence comparison and consistency thresholds; the warranty rule set, multi-category policies, four-source serial verification, contradiction, missing-document and duplicate detection; the decision engine, manual-review workflow and reviewer override audit trail; 8-stage tracking and notifications; admin analytics and search; reporting, audit and model-version telemetry; the first RBAC implementation. Deployed to PythonAnywhere and fixed its start-up errors; set up Render/gunicorn files; tuned the live check layout and the Linux/Windows TFLite requirements on 28 Sept.	52 commits (23–26, 28 Sept) · src/ , templates/ , policies/ , sample_claims/ , Procfile , render.yaml , requirements.txt
Muhammad Ghanyan	Frontend design	Designed the login and dashboard pages in Figma and chose the colour palette used as the visual reference for the interface; checked the mobile view of the pages.	Figma designs shared with the team (no commits under a personal GitHub account)
Muhammad Sami ur Rehman	Testing	Ran the 11 SRS sample claims (sample_claims/) through the application and checked each result; tested the pages in Chrome and on mobile; reported the defects found to the team for fixing.	Test runs reported to the team (no commits under a personal GitHub account)

Work sessions / meetings (optional)

Date	Members present	Work done

See also [AI_USAGE.md](#) (AI tools used and who verified their output) and [documentation/DEVELOPMENT_LOG.md](#) (dated log of work, problems and fixes).

Development log

Entries record work completed, problems encountered, model failures, changes made and tests performed. They are written from the Git history ([git log](#)); team members can add their own notes under each day.

Commit activity by day

Day	Commits	Accounts
23 September 2026	3	Muhammad Sami
24 September 2026	18	Muhammad Sami
25 September 2026	15	Muhammad Sami
26 September 2026	19	Muhammad Sami (13), Saba1512006 (6)
27 September 2026	59	Saba1512006
28 September 2026	6	Muhammad Sami (3), Saba1512006 (3)

23 September 2026 — first working version

Work completed - Deployment files (unicorn, Procfile, [render.yaml](#)); architecture diagram; database state synced. - Product lookup mismatch fixed in the claim wizard; document upload inside the wizard. - LICENSE, the [sample_claims/](#) suite, screenshots and export fixes; password confirmation on sign-up.

24 September 2026 — SRS functional requirements

Work completed - Multi-role registration, profile management and navigation (SRS 1.6 i-ii); product registration and the common warranty interface (iii-iv); receipt/invoice upload and OCR extraction (v-vi); extracted-data verification and warranty tracking (vii-viii); expiry alerts and claim registration by staff and users (ix-x); claim information, fault evidence upload and repair history (xi-xiii). - Document organisation, data validation and pre-processing (xiv-xvi); algorithm comparison on the admin dashboard (xviii); Claim Summary Card endpoint, Teachable Machine preview and prediction comparison UI. - Confidence comparison with consistency thresholds; the warranty rule set; multi-category policies, serial verification from four sources and contradiction detection; missing-document and duplicate checks.

Problems encountered and fixed - Quick-login reference error; a Jinja2 syntax error in the claim inspection page; demo claims not kept for the reviewer workbench; test clean-up leaking between tests.

25 September 2026 — decisions, dashboards and deployment fixes

Work completed - Claim summary, preparation assistance and the final decision engine; decision explanation, manual-review workflow and reviewer override audit trail; claim tracking and notifications; admin duplicate alerts, search and analytics; reporting and audit coverage; model version tracking and error handling. - UI clean-up (navbar, signed-in welcome card, user-facing labels).

Problems encountered and fixed - PythonAnywhere startup failures: a missing `typing.Any` import and the `reportlab` import at start-up. - Cross-platform model loading; strict OCR receipt validation (unreadable invoice images refused); a 500 error in the intake wizard; OCR mock fallback.

26 September 2026 — v2 upgrade (AI-assisted, see [AI_USAGE.md](#))

Work completed - Replaced the dataset generator: attributes sampled independently of the class, labels derived from the policy files, 4% label noise, IDs assigned after shuffling, stratified 1,050/225/225 split. - Retrained the Python model: three algorithms, selection on validation, sigmoid calibration, leakage audit, permutation importance, model card. Test accuracy 89.3% (macro-F1 89.2%, ROC-AUC 0.948). - Redesigned the Claim Summary Card (600×600, bundled fonts, visual encoding of every feature) and rendered 2,100 training / 225 validation / 225 test cards. - Replaced the scikit-learn stand-in for Teachable Machine with a TFLite runtime, an admin installer and an evaluation script. The browser training itself is still to be done. - Rule engine rewritten around config: 16 checks, severity per category in `policies/*.json`; decision table and routing in `config/decision_policy.json`; admin editors for both. - Integrated role- and record-scoped access control, login hardening, CSRF, rate limits, CSP, upload sniffing. New UI for every page.

Problems encountered - v1 scored 100%: `damage_type` was a relabelled class and claim IDs were ordered by class (leakage). - v1 selected the model on the test split; v1 probabilities were near 1.000, so every comparison was a "Strong Match". - Form options did not match training values; unknown values were silently encoded as zeros. - OCR relied on hard-coded retailer/product lists; `Invoice` was read as an invoice number. - Landscape cards were centre-cropped by Teachable Machine; fonts differed on Linux.

Model failures observed - Invalid claims predicted as Manual Review (8/75 on test); 5 of 14 missed Invalid claims are the rare late-reporting scenario. Covered by the LATE_REPORTING hard-fail rule. - Boundary claim (4 days into grace) gets 57% confidence → *Uncertain Result*; correctly routed to review.

Changes made after testing - Warnings made advisory (blocking rules belong in `manual_review_rules`); receipt made the only mandatory document, with rules for missing supporting documents — aligns the decision table with the policy. - Receipts with readable text but no invoice details are refused; scanned PDFs without text are accepted.

Tests performed - 192 automated tests passing; every page rendered for all four roles; browser screenshots desktop/mobile; seed run submitting the 11 demonstration claims through the live pipeline.

Teachable Machine, first run (card v2) - Trained in the browser on the 2,100 v2 cards (epochs 50, batch 16, learning rate 0.001) and installed as `gtm-ec95b4aa49b5`. Test accuracy 54.2% (validation 52.9%, macro F1 45.1%): below the 85% target. - The TensorFlow 2.20 package on Windows no longer ships `tf.lite.Interpreter`; the runtime uses `ai-edge-litert`, which installed fine on Windows. - Root cause found with an offline replica of Teachable Machine's trainer (`notebooks/tm_replica_check.py`): frozen MobileNet features cannot separate the v2 card's thin grey shapes (replica 58–66%). - Card redesigned (v3, policy-relative colour tiles); replica test accuracy 92.9–94.2%. Training cards regenerated.

Teachable Machine, second run (card v3) - Retrained in the browser on the 2,100 v3 cards with the same settings; installed as `gtm-514702678b8c`. - Test accuracy 93.8% (macro F1 93.8%), validation 97.3%, agreement with the Python model 92.9%. - Installing over the running model returned a 500 on Windows (memory-mapped `.tflite` could not be archived). Fixed by loading models from bytes; regression test added.

Accuracy guards - Tests now re-score both saved models on every run: Teachable Machine on test cards rendered with the current card code and policy files (must stay $\geq 85\%$ and equal to the recorded 93.8%), the Python model on the test split ($\geq 85\%$, equal to 89.3%). A deliberate one-colour change to the card dropped the image model to 77.8% and failed the test, as intended. - The policy editor warns when a limit drawn on the card is changed (the image model then needs retraining).

Requirement audit against SRS §1.10 - Added the standalone label encoder, preprocessing pipeline, feature schema, processed train/val/test CSVs, labels file, scenario definitions and all 225 sample test predictions (extracted from the saved model; a test proves they match it). - Excel (.xlsx) export next to CSV for claims, analytics, audit and search, with the same formula guard. - Test results file ([reports/test_results.txt](#)), team contribution record template, demo-video script, deployed-app testing guide in the README; a seeded demonstration claim where the two real models disagree. - Teachable Machine error analysis: 12 of its 14 test errors are deliberately flipped labels.

27 September 2026 — frontend rebuild, styling and input rules (AI-assisted, see [AI_USAGE.md](#))

Work completed - Teachable Machine v3 results recorded (93.8% test accuracy) with evaluation screenshots, misclassified samples and the comparison report; accuracy guards for both models in the test suite; technical blog published on Medium and linked; seeded model-disagreement case; demo video script. - Frontend rebuilt from the team's written brief: design tokens and app shell, verdict screen with agreement meter, landing live bench, model card, claim wizard with autosave and OCR split view, reviewer workbench, admin overview, what-if simulator, batch evaluator, policy editor with history, analytics, audit, access and models pages. - Styling pass: premium navbar, sign-in and register with the card scanner, dashboards with KPI cards and validated chart colours, claim detail and workbench, new AssureX logo, footer, back-to-top button, animated preloader, and a hero engine background that follows the live check. - Input rules on every form: no digits in names, cities, brands or retailers; no letters in phone or number fields; strict email format — enforced in the browser and on the server. - Sign-in pause shortened to one minute after five wrong passwords, with a live countdown on the sign-in and rate-limit pages.

Problems encountered and fixed - The Notifications item in the account menu opened the profile page; the account menu did not scroll on short screens; the admin menu had lost the Models, What-if and Batch links. - Training-card download failed on Windows. - A blurred, translucent card over the animated background made the live check slow enough to fail its browser test; the blur was removed. - The rate-limit (5 per minute) message conflicted with the "15-minute lockout" hint; both now say one minute.

Tests performed - 295 automated tests and 34 browser tests (route $\times$ role matrix, accessibility with axe, no overlap of the hero background with text at five screen widths, text contrast, input rules and lockout countdown).

28 September 2026 — deployment

Work completed - Live check card height and hero padding tuned on the landing page (Muhammad Sami). - Requirements: [tflite-runtime](#) on Linux and [ai-edge-litert](#) on Windows (Muhammad Sami). - The rate-limit page made compatible with older Flask-Limiter releases; scikit-learn pinned back to 1.9.1 so the saved Python model loads (Saba1512006). Work from a separate fork was merged into the main repository.

Problems encountered - On the PythonAnywhere free account both models showed as *Unavailable*: the account runs Python 3.10, whose newest scikit-learn (1.7.2) cannot load the model trained with 1.9.1, and the TFLite runtime was not installed. Creating a Python 3.11 virtualenv failed on the account's system image, and installing the TFLite runtime hit the 512 MB disk quota. - Render needed

a payment card even for the free instance, so it was not used. - Claims on the live site therefore go to manual review (the designed fallback). The full pipeline runs locally with the pinned requirements; see *Known limitations* in the README.

Tests performed - 295 automated tests and 34 browser tests pass locally with Python 3.11 and scikit-learn 1.9.1 ([reports/test_results.txt](#)).

Open items

- Record and link the demonstration video (.mp4).
- Complete the team rows in [AI_USAGE.md](#) and [documentation/TEAM_CONTRIBUTIONS.md](#).
- Add the Teachable Machine project link to [documentation/GTM_EVIDENCE.md](#).

16. Submission checklist

Group **NN_DevStorm** · AssureX Claim Engine — NextWave AI and ML · Batch 2609E2' · Faculty: Sir Minhaj

Each row follows SRS §1.10 *Project Deliverables* (items 1–16). Paths are relative to the project folder.

Start here: [documentation/AssureX_Project_Documentation.pdf](#) contains every item below in 16 chapters (report, dataset, both models' evidence, comparison report, policies, test cases and results, installation, execution guide with screens, deployment, video, blog, security, AI usage, team record, this checklist).

16. Final submission checklist

SRS item	Where
Complete documentation (all deliverables in one file)	documentation/AssureX_Project_Documentation.pdf (106 pages) and .docx — built by <code>python reports/build_documentation.py</code>
Project report	reports/AssureX_Project_Report.pdf (source: documentation/PROJECT_REPORT.md); also chapter 1 of the complete documentation
Public GitHub repository URL	https://github.com/Saba1512006/assurex-claim-engine
Complete source code	src/ , templates/ , static/ , database/ , config/ , wsgi.py
Structured warranty claim dataset	data/raw/common_warranty_claims_1500.csv ; splits data/splits/{train,val,test}.csv
Claim Summary Card image dataset	data/summary_cards/train/{valid,invalid>manual_review}/ (700 each), data/summary_cards/val/ (225), data/summary_cards/test/ (225)
Python model files	model/python_model/ (model, label encoder, preprocessing pipeline, feature schema, model card, sample predictions)
Google Teachable Machine model files	model/teachable_machine/model_unquant.tflite , labels.txt , evaluation.json
Warranty-policy files	policies/consumer_electronics.json , home_appliances.json , industrial_tools.json
Model-comparison report	reports/model_comparison_report.md and .csv (225 unseen test claims)
Test cases and results	documentation/TEST_CASES.md , tests/ , results in reports/test_results.txt
Installation instructions	documentation/INSTALLATION.md
Execution instructions	README.md (<i>Quick start</i> , <i>How to use it</i>)
Deployment URL	https://assurex.pythonanywhere.com (free-tier limitation in README.md › <i>Known limitations</i>)
Demonstration video	demo_video/AssureX_Demo.mp4 ; also https://github.com/Saba1512006/assurex-claim-engine/blob/demo-video/AssureX_Demo.mp4 ; chapters in documentation/DEMO_VOICEOVER.md
Technical blog link	https://medium.com/@sabarajput672/building-assurex-two-models-one-rulebook-and-why-our-first-100-was-a-bug-bb8b9858b11c
AI_USAGE.md declaration	AI_USAGE.md
Team contribution record	documentation/TEAM_CONTRIBUTIONS.md ; dated log in documentation/DEVELOPMENT_LOG.md

Detail by deliverable

#	Deliverable	Where
1	Project report (all 35 required sections, with DFD, use case, activity, sequence and decision-flow diagrams)	documentation/PROJECT_REPORT.md , reports/AssureX_Project_Report.pdf , diagrams in documentation/diagrams/
2	Required folders: README.md, AI_USAGE.md, requirements.txt, LICENSE, src/, templates/, static/, data/, notebooks/, model/, policies/, dataset_generator/, database/, tests/, sample_claims/, documentation/, screenshots/, reports/, config/	All present at the project root
2	Dataset-generation, pre-processing and training scripts	dataset_generator/generator_v2.py , src/core/features.py , notebooks/train_python_v2.py
2	Claim Summary Card generation	src/core/card_v2.py , dataset_generator/build_cards_v2.py
2	Rule engine, OCR and document processing	src/rules/ , src/ocr/ , src/services/documents.py , src/services/receipt_scan.py
3	Scenario definitions, labels, statistics, data dictionary, Claim ID ↔ image mapping	data/claim_scenarios.json , data/labels.csv , data/dataset_statistics.json , data/data_dictionary.json , data/claim_id_to_card_mapping.csv
4	Python model evidence (three algorithms compared, cross-validation, confusion matrix, per-class metrics, feature importance)	documentation/PYTHON_MODEL_EVIDENCE.md , model/python_model/model_card_v2.json
5	Teachable Machine evidence (classes, samples, configuration, observations, misclassified samples, retraining, screenshots)	documentation/GTM_EVIDENCE.md , screenshots/20_*-25_* , reports/gtm_misclassified_*.csv
6	Model prediction and confidence comparison (≥ 30 unseen claims; match status, confidence difference, consistency, rules, final decision)	reports/model_comparison_report.md / .csv (225 claims)
7	Three configurable warranty policies (coverage, start conditions, covered faults, exclusions, reporting period, repair and replacement conditions, authorised centers, grace period, mandatory documents, hard-fail / warning / manual-review rules)	policies/*.json , editable in Admin > Policies
8	Test cases (functional, integration, boundary, negative, security, database, OCR, both models, comparison, rules, contradiction, missing-document, duplicate, serial mismatch, low confidence, disagreement, hidden-test readiness)	documentation/TEST_CASES.md , tests/
8	The 11 demonstration claims	sample_claims/01_... to 11_... , loaded by python database/seed.py ; shown in the video, Part 4
9	Installation instructions (prerequisites, OS, Python version, virtual environment, dependencies, database, seed, model placement, OCR, environment variables, folders, run and test commands, default administrator, troubleshooting)	documentation/INSTALLATION.md

#	Deliverable	Where
10	Execution instructions (every step from registration to report export and tests)	README.md > <i>How to use it</i>
11	GitHub requirements (public, commit history, screenshots, test results, evaluator logins, limitations, assumptions, report, blog and video links)	README.md , screenshots/ , reports/test_results.txt
12	Deployed application with evaluator and administrator logins	https://assurex.pythonanywhere.com , accounts in README.md > <i>Evaluator accounts</i>
13	Demonstration video (.mp4)	demo_video/AssureX_Demo.mp4 , documentation/DEMO_VOICEOVER.md
14	Technical blog ($\geq 2,000$ words)	Medium link above; text in documentation/TECHNICAL_BLOG.md
15	AI tool usage declaration	AI_USAGE.md

Known gaps, stated openly

- **Commits from all team members** (SRS §1.10 item 11): commits exist under two GitHub accounts (Saba1512006, sami2515). Muhammad Ghanyan (Figma designs) and Muhammad Sami ur Rehman (testing) contributed outside the repository; see [documentation/TEAM_CONTRIBUTIONS.md](#).
- **Deployment:** the free PythonAnywhere account runs Python 3.10 with a 512 MB disk quota, so both models can show as *Unavailable* there and claims then go to manual review. The full pipeline runs locally ([documentation/INSTALLATION.md](#)) and is shown working in the demonstration video.

Submission package (faculty layout)

```
AssureX-Claim-Engine-Submission/
├─ Source_Code/          the project (README.md, AI_USAGE.md, requirements.txt, LICENSE, src/ ...
config/)
├─ Documents/           Project_Report.pdf, Python_Model_Report.pdf,
Teachable_Machine_Report.pdf,
|                        Model_Comparison_Report.pdf, Test_Cases.xlsx, Test_Results.pdf,
Installation_Guide.pdf,
|                        User_Guide.pdf, Technical_Blog_Link.txt, Team_Contributions.md,
Deployment_URL.txt
└─ Videos/             Demonstration.mp4
```

- The documents are generated from the project's sources into `submission/Documents/`: `python reports/build_submission_documents.py`.
- On Windows, one command builds the whole folder and its zip on the Desktop (the video must first be in `demo_video/AssureX_Demo.mp4`): `powershell -ExecutionPolicy Bypass -File scripts\make_submission.ps1`. It leaves out `venv/`, `.git/`, caches, the local database and `.env`.

Sharing the project folder

Before uploading the folder (for example to Google Drive), leave out `venv/` (machine-specific, several hundred MB) and `__pycache__` folders. Everything else, including `demo_video/AssureX_Demo.mp4`, belongs in the submission. The evaluator creates a fresh virtual environment with the *Quick start* commands in [README.md](#).